Faculty of Science and Engineering

School of Computer Science, Electrical and Electronic Engineering, and Engineering Mathematics

MSc Cyber Security (Infrastructures Security)

# Reactive SDN for Securing ICS Environments

## A Criticality- and Operation-Aware Response Framework (CARS)

Amit Kiran Deb - 2711250

---

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Science in the Faculty of Science and Engineering.

---

Thursday 3rd September, 2026

Supervisor: Dr Joseph Gardiner

# Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Science and Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Author.

Amit Kiran Deb - 2711250, Thursday 3rd September, 2026

# Contents


**1 Introduction** **1**

**2 Background** **3**
2.1 Industrial control systems and their security priorities . . . 3
2.2 Software-defined networking and OpenFlow . . . 3
2.3 Reactive intrusion response with SDN in ICS . . . 4
2.4 Trusting the control plane and its forwarding integrity . . . 4
2.5 Stateful security in the data plane . . . 5
2.6 Asset criticality and consequence-based prioritisation . . . 5
2.7 Operation awareness and the false-data-injection threat . . . 5
2.8 Industrial control system testbeds . . . 6

**3 Design and Implementation** **7**
3.1 Threat and attacker model . . . 7
3.2 CARS: architecture and principles . . . 8
3.3 The enforcement pipeline . . . 9
3.4 The criticality and operation model . . . 12
3.5 Reference implementation . . . 14
3.6 Portability and scalability . . . 18
3.7 Testbed instantiation . . . 18
3.8 Deployment . . . 21

**4 Critical Evaluation** **24**
4.1 Method and metrics . . . 24
4.2 Decision conformance and false positives . . . 24
4.3 Criticality-graded response . . . 26
4.4 Wire-level campaign: armed versus unprotected . . . 28
4.5 Cross-layer validation . . . 30
4.6 Reaction window and latency . . . 33
4.7 Anatomy of a defended attack: two pivots end to end . . . 35
4.8 Does the defence disturb the process? . . . 37
4.9 Process-layer defence in depth . . . 39
4.10 Long-run and stress-test campaign . . . 40
4.11 Comparison with related work . . . 42
4.12 Threats to validity . . . 43
4.13 Ethics . . . 47

**5 Conclusion** **48**
5.1 Contributions . . . 48
5.2 What the evaluation establishes, and what it does not . . . 48
5.3 Future work . . . 49

**A Supporting Material** **54**
A.1 Full OpenFlow tables . . . 54
A.2 Deployed decision logic . . . 56
A.3 Detection rules . . . 58
A.4 Runtime state . . . 60

A.5 Complete evaluation matrix . . . . . 60
A.6 Decision logs and events . . . . . 60
A.7 Packet-level and wire-level analysis . . . . . 61
A.8 Scenario testing in full . . . . . 63
A.9 Scripts and code . . . . . 65
A.10 The CARS intelligence dashboard . . . . . 70
A.11 Gap remediation evidence . . . . . 71
A.12 Process-transparency and availability runs . . . . . 73
A.13 Statistical visualisations of the evaluation . . . . . 74

# List of Figures


3.1 The CARS framework . . . 9
3.2 The reproducible system architecture . . . 10
3.3 A packet's path through the CARS fabric . . . 11
3.4 A malicious write's path, armed and disarmed . . . 11
3.5 Detection to response, and self-heal . . . 15
3.6 The criticality-scaled timeout ladder . . . 17
3.7 The SDN-ICS logical topology . . . 19
3.8 The physical process cell . . . 20
3.9 The fabric from `ovs-vsctl show` . . . 21
3.10 The hardware-in-the-loop . . . 22
3.11 The live process . . . 22
3.12 The Factory IO S7-1200 driver . . . 23
3.13 The controller status interface . . . 23
3.14 The deployed controller running live . . . 23

4.1 Criticality read from the controller . . . 27
4.2 Process devastation, disarmed . . . 28
4.3 Armed isolation on the dashboard . . . 29
4.4 Armed enforcement at the datapath . . . 29
4.5 FDI injection, armed versus disarmed . . . 30
4.6 The armed drop on the wire . . . 32
4.7 The malicious operation on the wire . . . 32
4.8 Anatomy of the 7.6 ms reaction window . . . 34
4.9 Reaction window over 100 trials . . . 35
4.10 The two pivots from one Kali . . . 36
4.11 Reaction window versus attacker cadence . . . 38
4.12 Framework flow-table behaviour under load . . . 43
4.13 The first-packet residual . . . 45

A.1 Operation-aware coverage boundary . . . 59
A.2 GUARD anti-spoof counters . . . 60
A.3 The THROTTLE meter . . . 60
A.4 Modbus operation on the wire . . . 61
A.5 S7 write reaches the PLC, disarmed . . . 62
A.6 Attack frame amid legitimate loop traffic . . . 62
A.7 Sensor-spoof deception, disarmed . . . 63
A.8 Reconnaissance blocked, armed . . . 64
A.9 Identity spoofing dropped at GUARD . . . 64
A.10 Reaction window over 100 trials . . . 64
A.11 Dashboard discovered topology . . . 70
A.12 Normal OT traffic baseline . . . 74
A.13 Decision conformance at scale . . . 74
A.14 Operation- and source-graded response . . . 75
A.15 Flow-integrity: poll vs event-driven monitor . . . 75
A.16 Process transparency: armed vs disarmed . . . 76
A.17 Reaction window against background load . . . 76
A.18 Reconnection jitter on the legitimate conduit . . . 77

A.19 Reaction window, long-run campaign . . . . . 78
A.20 Per-vector reaction window . . . . . 79
A.21 Criticality-scaled timeout, 120 probes . . . . . 79
A.22 S7-1200 connection-pool limit . . . . . 80
A.23 Process stability under continuous attack . . . . . 81

# List of Tables


3.1 The criticality model . . . 12
3.2 The rulebook . . . 13
3.3 The seven-response ladder . . . 16

4.1 The decision matrix . . . 26
4.2 Per-class conformance summary . . . 26
4.3 Testbed criticality assignment . . . 27
4.4 The campaign: armed versus unprotected . . . 30
4.5 Cross-layer proof of each response . . . 31
4.6 The drop shown on the wire . . . 31
4.7 Operation-awareness on one conduit . . . 33
4.8 Where each attack class terminates . . . 36
4.9 The same attack from two positions . . . 37

5.1 Aims and objectives against outcomes. . . . 49

A.1 Map from the report to the published repository (the project GitHub repository). . . . 54

# List of Listings


3.1 The three-table admission a packet meets, in the switch's priority order: identity binding, then the stateful policy with the reactive responses on top, then forwarding. . . . . 10
3.2 The CARS operation decision. A tier is fixed by first match in the rulebook, a sensitive operation on a critical asset is elevated, and the tier, the criticality weight and the arrival rate select the graded response. . . . . 13
3.3 Turning a response into an OpenFlow rule with a criticality-scaled timeout, and the self-heal on expiry that releases it unless the attack renews the detection. . . . . 17
4.1 The flow-table saturation harness (`flowtable_stress.py`): a flood of distinct sources posted to the controller, each forcing one source-scoped isolate, sampled as the reactive table fills and then self-heals on timeout. . . . . 41
A.1 Cell-1 switch `ovs1`, tables 0 to 2. . . . . 54
A.2 Gateway switch `ovsgw`, tables 0 to 2. . . . . 55
A.3 Cell-2 switch `ovs2`, tables 0 to 2. . . . . 56
A.4 The deployed `RULEBOOK` (first match wins). . . . . 56
A.5 The A2 allowlist and default-deny (`a2_policy.json`). . . . . 57
A.6 S7 and Modbus operation-class rules from `cars.rules`. . . . . 58
A.7 The evaluation matrix, `cars_eval_matrix.csv`. . . . . 60
A.8 Representative controller decisions across the campaign (extract of the decision log; times `08-05`). . . . . 61
A.9 Modbus read and write on the same conduit; the function code sits at offset 7. . . . . 61
A.10 The reactive isolate as an `OFPT_FLOW_MOD`, decoded from `of_control.pcap` (OpenFlow 1.3, tcp/6653). . . . . 62
A.11 The controller decision path (`cars_engine.py`): first-match rulebook classification. . . . . 65
A.12 Grey-zone elevation and criticality-scaled block duration. . . . . 65
A.13 Criticality-graded, flood-aware response selection. . . . . 65
A.14 Source quarantine: a self-healing drop on every switch (`REACTIVE_COOKIE = 0x00CA`). . . 65
A.15 Table 0 identity binding and anti-spoof drop. . . . . 66
A.16 Token gate on the control endpoints, in pseudocode. . . . . 66
A.17 Operation recovery from the Snort alert, forwarded to the controller, in pseudocode. . . . 66
A.18 Flow identity and the drift model, in pseudocode. . . . . 66
A.19 Anomaly-triggered last-good restore over S7, in pseudocode. . . . . 67
A.20 Single-clock mean-time-to-mitigate measurement, in pseudocode. . . . . 67
A.21 The FDI overflow injection (writes to the S7 output area). . . . . 67
A.22 Driving each case through the engine and scoring it against intent, in pseudocode. . . . . 67
A.23 Fire a controlled case, and heal the tier-scaled reactive rule between fires. . . . . 67
A.24 Arm/disarm and the three-point capture used across the campaign. . . . . 68
A.25 The three capture points and the control-plane vector. . . . . 68
A.26 The four independent ground-truth layers per scenario. . . . . 68
A.27 Two synchronised capture points: what the detector sees versus what reaches the PLC. . 68
A.28 Fire each operation on the same conduit and await the new decision, in pseudocode. . . . 68
A.29 The output-area write (actuation) and the sensor-spoof variant. . . . . 68
A.30 Crafting an arbitrary Modbus function code (FC at offset 7). . . . . 69
A.31 The bang-bang control law, the dynamics, and the interference health metric, in pseudocode. 69
A.32 Stream reassembly, and the offset-to-relative rule change. . . . . 69
A.33 Sending the Write-Var whole or split at byte N (0x05 pushed into the second segment). . 69
A.34 One authorised HMI stop; and pinning the level sensor high. . . . . 69
A.35 The high-side envelope, rate and liveness checks (alarm-only), in pseudocode. . . . . 69

A.36 Gap A: before (fragmented write evades) and after (caught), from the controller audit. . . 71
A.37 Gap A: `cars.conf` diff, stream reassembly added on ports 102 and 502. . . . . 71
A.38 Gap A: `cars.rules` diff, S7 rules re-anchored PDU-relative (rev 1 to rev 2). . . . . 71
A.39 Gap B: the flood backstop and the two guardian alarms. . . . . 72
A.40 Gap B: the guardian alarm feed (`cars_guardian.jsonl`), stall record from the fresh run. 72
A.41 Gap C: transient (mostly missed) versus persistent (always caught) flow-integrity status, with the thirty-trial totals. . . . . 72
A.42 Gap C remedy: event-driven monitor on the same transient test. . . . . 72
A.43 Gap 2: shared-identity conduit BLOCK versus single-host source ISOLATE, deployed engine. 72
A.44 Gap D: silent-drop baseline, the redirect, and the decoy receiving the diverted traffic. . . . 73
A.45 Transparency: online-only level statistics per phase, from `interleaved.csv`. . . . . 73
A.46 Block-and-maintain under attack, then self-heal, from the armed run. . . . . 73
A.47 Reconnection jitter: the two planes contrasted. . . . . 73
A.48 The isolate is source-scoped: the reactive drop matches the attacker alone, and the legitimate Factory IO conduit is untouched during the isolation (captured live on `ovs1`). . . . . 81

# Abstract

Automated intrusion response is rarely switched on in operational technology, and the reason is trust rather than detection. A reactive defence that blocks traffic can cut the very process it is meant to protect, and an operator will not arm a measure that might trip a running plant. This project addresses that barrier with CARS, a criticality- and operation-aware software-defined-networking intrusion-response framework for industrial control systems. Instead of acting uniformly, CARS conditions its response on two properties that plain blocking ignores: the criticality of the asset under attack, and the industrial operation, recovered from the traffic by a deep-packet-inspection sensor, so that reading a value from a controller is treated differently from writing one. Its active enforcement responses are graded across a seven-rung ladder, each bounded in time and reversible, while the safety-critical loop is held on an alert-only rung that never cuts the process; every decision is recorded. The contribution is this design as a portable framework, a controller core separated from a per-site configuration so that the same engine applies to any plant rather than to one testbed; the testbed described below is one instantiation of it, used to validate the design against a real physical process.

That design was validated on a physical hardware-in-the-loop testbed rather than a pure network simulation. Two Siemens S7-1212C programmable logic controllers, their operator panels and the software-defined fabric are real hardware running live S7comm and Modbus/TCP, and one of them drives a tank-level process that is co-simulated in Factory IO, a hardware-in-the-loop arrangement that keeps the controller and the industrial protocol real while modelling only the plant, with an engineering workstation present as in a small plant. The system is realised as a three-table OpenFlow pipeline: identity binding against spoofing, a stateful policy stage carrying the criticality- and operation-aware rulebook, and a learning switch, supported by a flow-integrity self-check and an authenticated control interface. Detection is delegated to the packet sensor, while the software-defined network layer makes the graded decision and enforces it as OpenFlow rules. Every capability was validated at the level of individual packets, protected against unprotected.

The central result is that the armed defence refused every illegitimate operation while cutting none of the legitimate process traffic: a false-positive rate of zero on the live process, no illegitimate operation permitted across the decision space and a 2,078-case corpus, with enforcement shown on the wire rather than inferred from a log, and a reaction window with a median of 7.6 milliseconds. A false-data injection that overflowed the tank when unprotected could not move the process when the defence was armed: its S7 session was severed at the first write, at most a single frame reaching the PLC before the cut, and the tank held its band. The evaluation carries its boundaries honestly, including the trusted-insider case that a network layer must not act on. The finding is that the trust barrier which keeps such defences switched off is not fundamental: a reactive network defence can be made criticality- and operation-aware, bounded and reversible enough for an operator to arm on a running process.

# Supporting Technologies

The following third-party hardware and software were used to build and evaluate CARS. The contribution of this project is the CARS design, its controller engine, and the evaluation; the items below are the platform it was built on.

**Hardware**

- Three workstations forming the testbed: one runs the Open vSwitch fabric (`ovs1` and `ovsgw`), the Snort sensor and the supporting services, and hosts the supervisory and attacker endpoints; one runs the CARS controller; one runs the second cell (`ovs2`).
- Two identical Siemens teaching boxes, each holding an S7-1212C programmable logic controller (model 6ES7 212-1BE40-0XB0, firmware 4.2.3, confirmed by scanning during evaluation). The Cell-1 controller drives the hardware-in-the-loop tank process; the Cell-2 box provides the second cell used to validate the cross-cell and NAT-clone identity scheme, and does not drive a separate physical process.
- Two Siemens KTP700 operator panels, one in each teaching box, as the human-machine interface on each cell.
- A hAP access point providing the isolated wired management network that carries the OpenFlow control plane, kept separate from the operational-technology data plane.
- A Windows PC acting as the engineering workstation, running TIA Portal and Factory IO.

**Software**

- TIA Portal V19, used to program the PLCs and the OB30 tank-control logic.
- Factory IO with its S7-1200 driver, providing the hardware-in-the-loop tank-level process wired to the live PLC.
- Open vSwitch 3.3.4, providing the software-defined data plane (the switches `ovsgw`, `ovs1` and `ovs2`) under OpenFlow 1.3 control.
- os-ken 2.8.1, the OpenFlow 1.3 controller framework hosting the CARS engine (`cars_engine.py`); the engine and the supporting tools are written in Python.
- Snort (2.9 series), used as the deep-packet-inspection sensor that recovers the industrial operation from S7 and Modbus traffic; the deployed configuration uses its `stream5` TCP stream-reassembly preprocessor and offset-based rule syntax.
- python-snap7, the S7 client library used by the process model, the remediation agent and the attack clients.
- Linux network namespaces, used to isolate the supervisory endpoints (SCADA, Modbus operator, historian, remediation agent) on the fabric-and-sensing node, each bound to a switch port.
- Kali Linux, used as the external attacker virtual machine attached to the operational segment, together with nmap for reconnaissance and a set of custom Python clients for the S7 and Modbus write, false-data-injection and fragmentation tests.
- tcpdump, with Wireshark and tshark, used to capture and analyse traffic at the wire points for the packet-level evaluation.

# Notation and Acronyms

The acronyms used in this report are listed below, followed by a note on the S7 addressing referred to in the design and evaluation chapters.

| Acronym | Meaning |
|---|---|
| CARS | Criticality- and Operation-Aware Response Framework (this project) |
| ICS | Industrial control system |
| OT | Operational technology |
| IT | Information technology |
| PLC | Programmable logic controller |
| HMI | Human-machine interface |
| EWS | Engineering workstation |
| SCADA | Supervisory control and data acquisition |
| SDN | Software-defined networking |
| OVS | Open vSwitch |
| DPI | Deep packet inspection |
| IDS | Intrusion detection system |
| FDI | False data injection |
| HIL | Hardware-in-the-loop |
| NAT | Network address translation |
| GUARD | The identity-binding, anti-spoof stage of the pipeline (Table 0) |
| MTTM | Mean time to mitigate (the reaction window from detection to enforcement) |
| REST | Representational state transfer (the controller's control interface) |
| TCP | Transmission control protocol |
| FC | Modbus function code |
| OB | Organisation block (PLC program unit) |
| DB | Data block (PLC memory area) |
| S7 | Siemens S7 communication protocol |
| PDU | Protocol data unit |
| TPKT | ISO transport packet header on TCP (the `03 00` bytes) carrying S7comm |
| COTP | Connection-oriented transport protocol (ISO 8073), the layer beneath S7comm |
| MBAP | Modbus application protocol header (carries the Modbus function code) |
| DPID | OpenFlow datapath identifier (per switch) |
| REAL | S7 IEEE 754 single-precision (32-bit) floating-point data type |
| TP, TN, FP, FN | True positive, true negative, false positive, false negative |

S7 addressing follows the Siemens convention used throughout: `%I` for process inputs and `%Q` for process outputs at the bit level, `%ID` and `%QD` for the corresponding double-word (32-bit) analogue channels, which on this process carry REAL (IEEE 754 single-precision floating-point) values rather than double integers, so the level and valve values pass as native floats without `SCALE`/`NORM_X` conversion, and `%M` for internal memory bits (merkers). For example, `%ID100` is the analogue level input from the tank sensor, `%QD100` and `%QD104` drive the fill and discharge valves, and `%M0.1` and `%M0.2` are the HMI start and stop bits.

# Acknowledgements

I thank my supervisor, Dr Joseph Gardiner, for his guidance and feedback throughout this project. I am grateful to the staff of the University of Bristol Cyber Security group for access to the laboratory and its equipment. I also thank my family for their support during the work.

# Chapter 1

# Introduction

Industrial control systems (ICS) run the physical processes behind electricity generation, water treatment, gas distribution and manufacturing. They differ from ordinary information technology in one respect that shapes every security decision: safety and the continuity of the process come first. The NIST guide to operational-technology security states that OT security objectives typically prioritise integrity and availability, followed by confidentiality, and must treat safety as an overarching priority [1]. A controller that stops driving its process can therefore cause a physical hazard rather than only a loss of data. A programmable logic controller (PLC) that loses contact with its sensors part way through a cycle may overfill a tank or trip a turbine. Any defensive measure placed in front of such a device therefore carries a risk of its own.

Software-defined networking (SDN) separates the control of a network from the switches that forward its traffic, so a central controller can install or withdraw forwarding rules as conditions change. This makes SDN an attractive base for reactive intrusion response in ICS: when a threat is detected, the controller can act on the offending traffic across the whole network in a coordinated way. The open question is what that action should be. A survey of SDN-based intrusion-response systems for ICS organises the field into three families of response strategy, of which the most direct, dynamic traffic filtering, acts by dropping packets or blocking flows [2]. None of these strategies conditions its response on the criticality of the device being protected. Cutting traffic to a safety-critical PLC in the middle of its control loop can be as harmful as the attack it is meant to stop. This is why automated response is still rarely enabled in operational technology. The barrier is not detection accuracy but trust: an operator will not switch on a defence that might trip the process itself.

The threats such a defence must answer are well documented and physical in their consequences. An attacker who reaches the control network can enumerate the PLCs, impersonate a trusted engineering station, or inject false sensor data so that the controller drives the process to an unsafe state while its own readings appear normal [3]. Because the SDN controller holds the forwarding policy for the whole network, a compromise of the control plane is itself a recognised attack, able to insert or remove rules silently [4]. A response system that reacts to these threats with a single blunt action inherits the trust problem described above; one that reacts without regard to which asset it is protecting cannot claim to be safe for the process.

This project addresses that gap with CARS, a criticality- and operation-aware SDN intrusion-response framework for ICS. The central idea is to condition the response on two properties that uniform blocking ignores: the criticality of the asset being protected, and the nature of the operation being attempted. Criticality follows a consequence-based tiering drawn from established guidance, in which each asset is rated by the physical and operational consequence of its compromise [5, 6]. The operation is recovered from the traffic itself, so that reading a value from a PLC is treated differently from writing one, and an engineering download is treated differently again. Responses are graded rather than binary, and each is designed to be bounded in time, reversible, and to leave an audit record. The intent is a defence an operator can enable, because it will refuse an unsafe action on a critical asset before it will ever interrupt the process that keeps that asset running.

The work is grounded in a physical hardware-in-the-loop testbed rather than a pure simulation. Two real Siemens S7-1212C PLCs (model 6ES7 212-1BE40-0XB0, firmware 4.2.3, confirmed by scanning the device during evaluation), their operator panels and the SDN fabric are physical hardware running live S7comm and Modbus/TCP; the tank-level process one of them drives is co-simulated in Factory IO, so the fluid dynamics are modelled in software while the control and network planes are real; a physical

water tank was not available, and this hardware-in-the-loop arrangement was preferred to a pure software simulation because it keeps the controller, its control logic and the industrial protocol real and co-simulates only the plant. An engineering workstation is present as it would be in a small plant. The hardware-in-the-loop process still allows a response to be judged against its real consequence: whether the tank stays within its safe band, or overflows. The central challenges the project had to resolve were to define a criticality and operation model that is precise enough to enforce, to realise it as an OpenFlow pipeline that does not add meaningful latency to the control path, and to show, at the level of individual packets, both that the defence stops the attacks and that it never blocks the legitimate process.

## Aims, objectives and achievements

The aim of the project was to design, build and evaluate an SDN intrusion-response framework for ICS whose response is conditioned on asset criticality and operation, so that automated response can be enabled without risking the physical process. This was pursued through the following objectives:

- Design a response model that grades enforcement by asset criticality and operation class, and that is bounded, reversible and evidence-generating.
- Implement the model as an OpenFlow controller with a layered pipeline, together with integrity checks that detect tampering with the forwarding state and refuse an unauthenticated disarm of the defence, so that control-plane tampering is caught rather than trusted (a fully subverted controller is out of scope, Section 3.1).
- Keep the design portable, separating a controller core from the site-specific configuration of assets, roles and conduits, so that the same engine applies to any plant rather than to one testbed.
- Build a representative hardware testbed with real PLCs, an operator panel and a live hardware-in-the-loop process, and instantiate CARS on it to validate the design against real physical consequences.
- Evaluate decision conformance and false positives, and validate each capability at the packet level against realistic attacks, comparing protected with unprotected operation.

The main achievements of the project, each evidenced in the chapters that follow, are:

- CARS itself: a criticality- and operation-aware SDN intrusion-response framework, realised as a three-table OpenFlow pipeline (identity binding against spoofing, a stateful policy stage carrying the criticality- and operation-aware rulebook, and a learning switch), a graded seven-level response ladder, a flow-integrity checker and an authenticated control interface.
- A portable design in which the engine is separated from a site configuration of assets, roles, conduits and tiers, verified to reproduce the deployed policy, and shown to run unchanged in a hardware-free emulation as well as on the physical testbed.
- A three-node SDN-ICS hardware testbed with two real S7-1212C PLCs and a live hardware-in-the-loop tank process, on which CARS was instantiated and validated against a real physical consequence.
- A decision-conformance evaluation across the range of role, operation class and asset criticality in which every legitimate operation was permitted and every illegitimate one was refused, with no false positives against the live process traffic.
- A packet-level validation of each capability, protected against unprotected, spanning reconnaissance, identity spoofing, out-of-state traffic, control-plane and flow tampering, operation-aware commands, and a false-data injection that overflowed the tank when unprotected yet, when the defence was enabled, was severed at its first write, a single frame reaching the PLC before the cut, so no write moved the process.

# Chapter 2

# Background

This chapter sets out the technical basis the project depends on and the work it builds on. It moves from the security priorities of industrial control systems, through the use of software-defined networking to react to attacks in these environments, to the more specific problems of trusting the control plane, keeping security decisions stateful in the data plane, recognising the operation an attacker is attempting, and grading a response by the consequence of acting on an asset. The intent is that a reader who is technically competent but new to the topic can follow, and fairly assess, the design and evaluation presented in later chapters.

## 2.1 Industrial control systems and their security priorities

Industrial control systems (ICS) supervise and drive physical processes in sectors such as electricity, water, gas and manufacturing. A programmable logic controller (PLC) reads sensors, runs a control program and drives actuators such as pumps and valves, often within a cycle measured in milliseconds. This coupling to the physical world changes what security has to protect. The NIST guide to operational-technology security states that OT security objectives typically prioritise integrity and availability, followed by confidentiality, and must treat safety as an overarching priority [1]. Etxezarreta et al. give the same priority from the network side, noting in their survey that availability is a priority in industrial control systems, whose concerns include fault tolerance and human safety, so that an unexpected system outage is not acceptable [2]. A controller that is prevented from driving its process can therefore cause a physical hazard rather than only a disclosure of data. A defence for such an environment must therefore be judged not only on whether it stops an attack, but on whether the act of defending disturbs the process.

A brief note on how such a controller is programmed helps the reader who comes from networking rather than automation. A PLC runs on a repeating scan: each cycle it copies the field inputs into a process image, executes its logic, and writes the outputs. That logic lives in organisation blocks that the runtime calls automatically, such as OB30, the cyclic block that holds this project's tank control. Memory is addressed by area: `%I` and `%Q` are the input and output images at the bit level, `%ID` and `%QD` the 32-bit analogue channels, and `%M` internal memory bits. So a level sensor might arrive at `%ID100`, a valve be driven from `%QD100`, and an operator button be held in a `%M` bit. These conventions recur throughout the design and evaluation chapters.

## 2.2 Software-defined networking and OpenFlow

Software-defined networking (SDN) separates the decision about how traffic is forwarded, the control plane, from the switches that carry it, the data plane. A central controller holds a global view of the network and installs forwarding rules into the switches, most commonly through the OpenFlow protocol [7], so the behaviour of the network can be changed in software rather than by reconfiguring each device by hand [8]. For intrusion response this programmability is attractive, because when a threat is detected the controller can add or remove rules across the network in a coordinated way [2]. The same centralisation is also a weakness, discussed in Section 2.4, since the controller becomes a single point through which the whole forwarding policy can be altered. Securing SDN itself has been surveyed across its control and data planes, spanning traditional, machine-learning and moving-target-defence approaches [9], and the same programmability has been used to enforce per-flow security policy dynamically in other domains,

for example robotic systems [10].

## 2.3 Reactive intrusion response with SDN in ICS

Etxezarreta et al. survey how SDN has been used to build intrusion-response strategies for ICS and organise the field into a taxonomy of three families [2]. The first is dynamic traffic filtering, in which a detection technique such as deep packet inspection, signature matching, anomaly detection or machine learning identifies malicious traffic and the controller drops the offending packets or blocks the flow so that it does not reach the target device. The survey's own example of this family detects payload alteration on the path between a SCADA server and a PLC and drops the malicious control packets before they arrive [2]. The second family is network survivability, which uses the global view of the network to detect a failure or attack and reconfigure paths so that operation continues, as in the self-healing SDN communication layer for microgrids of Jin et al. [11]. The third is cyber deception, which includes moving-target defence, where addresses or paths are randomised to raise the attacker's uncertainty [12], and honeypots.

Concrete systems in the filtering family show the pattern directly. Kabasele Ndonda and Sadre pair a P4 flow and Modbus whitelist with a deep-packet-inspection layer that updates that whitelist through the controller [13], the closest structural precedent for the layered pipeline used here. Sainz et al. demonstrate the detection half of this pattern directly, coupling a signature-based deep-packet-inspection sensor to an SDN controller that drops the flagged flows, and validate it on a scaled industrial testbed against payload alteration and flooding [14]; this sensor-to-controller arrangement is the one on which CARS builds its own detection. Brugman et al. build an SDN-based detection-and-prevention framework for ICS [15], and Radoglou-Grammatikis et al. combine anomaly detection with active prevention for DNP3 SCADA traffic in DIDEROT [16]. The response can itself be virtualised: Murillo Piedrahita et al. realise it as virtualised, SDN-based incident-response functions for control systems [17]. The same detect-then-react loop recurs in machine-learning detection of denial-of-service on SDN-based SCADA [18] and, outside the industrial setting, in reactive SDN mitigation that blocks a flood at its source [19].

Two observations follow that motivate this project, and they hold across the concrete systems above rather than only in the abstract taxonomy. First, the enforcement action in each is a uniform cut. The survey's payload-alteration example drops the offending control packets [2], the two-level system blocks any flow outside its Modbus whitelist [13], DIDEROT couples anomaly detection with active prevention of the flagged flow [16], and the denial-of-service work blocks the flood at its source once detected [19, 18]. What advances between these systems is the detection, by whitelist, learned model, signature or classifier, while the response itself is the same drop applied to any offender. Second, and more consequential, none conditions that response on what it is acting upon. A blocked flow to a rogue scanner and a blocked flow to a PLC in the middle of its control loop are the same action, even though the second can be as harmful as the attack it answers; and where the industrial operation is recovered at all it is used to judge whether traffic is malicious, not to grade how the defence should respond. This is, on the evidence of the systems surveyed above, the practical reason automated response is rarely armed in operational technology: the barrier is not the accuracy of detection but trust that the response will not itself trip the process, and a uniform cut offers an operator no assurance on that point. CARS addresses this validated gap by conditioning the response on the two axes the surveyed systems leave flat, the criticality of the protected asset and the class of operation attempted, and by making that response bounded, reversible and evidence-generating so that it can be armed on a live process rather than left switched off.

## 2.4 Trusting the control plane and its forwarding integrity

Because the SDN controller sets the forwarding policy for the whole network, its compromise is a distinct and serious threat. Gardiner et al. show that the centralisation of control in SDN creates a single point of failure, and demonstrate controller-in-the-middle attacks in which a compromised or interposed controller manipulates the network in an ICS setting [4]. A response system that trusts its own control plane without question cannot claim to be safe against this class of attack.

The complementary defence is to verify the forwarding state itself. Melis et al. present a toolkit, built as plug-ins for the ONOS controller, that lets an administrator express security policies for an industrial SDN and check them formally against the installed rules. Their checker detects compromised forwarding caused by bogus injected flow-rules, inner loops and black-holes, which they note are hard to find through ordinary network scans, as well as the replacement or removal of legitimate rules [20]. The flow-integrity

checker in CARS follows this line of work: it holds a trusted baseline of the security policy and reports any drift from it, so that tampering with the forwarding rules is caught even when the controller itself is the source of the change.

## 2.5 Stateful security in the data plane

Early SDN security work observed that pushing every decision to the controller is itself exploitable. Shin et al. introduce AVANT-GUARD, which extends the OpenFlow data plane with two mechanisms. Connection migration has the switch complete the TCP handshake and involve the controller only once a connection is established, which sharply reduces the traffic sent to the control plane during scanning or denial-of-service attacks. Actuating triggers allow conditional actions to be taken within the data plane in response to network conditions [21]. The relevant principle for this project is that security decisions can carry connection state in the data plane rather than treating each packet in isolation. The stateful stage of the CARS pipeline follows that principle: it tracks connection state so that only properly established flows are admitted, and out-of-state packets that a stateless rule would pass are refused. More recent work pushes security functionality further into the data plane itself, using programmable data planes to raise the digital resilience of OT networks [22].

## 2.6 Asset criticality and consequence-based prioritisation

The idea that a response should depend on the value of what is being protected is the basis of consequence-driven security engineering for control systems. The Idaho National Laboratory's Consequence-driven Cyber-informed Engineering examines high-consequence risk in critical infrastructure by first identifying the most damaging events an adversary could cause, then the key devices and components that enable them, the plausible cyber attack paths to those devices, and concrete mitigations and tripwires [5]. Operational guidance from CISA and international partners complements this at the inventory level. It recommends classifying operational-technology assets both by criticality, where a critical asset is one whose failure or compromise would have the most significant impact, and by function, such as control, monitoring, engineering or management [6]. CARS applies both ideas. Each protected asset is assigned a criticality tier according to the consequence of its compromise, and that tier, together with the class of operation, determines both the decision and the strength and duration of the response. The specific tiering used in this project is defined in Chapter 3.

## 2.7 Operation awareness and the false-data-injection threat

Traffic filtering in ICS can act on more than the source and destination of a flow. Because industrial protocols carry the operation being requested, deep packet inspection allows a defence to distinguish, for example, a read of a value from a write of one, and to treat an engineering download differently again; the survey lists deep packet inspection among the detection techniques used in the filtering family [2]. Protocol-specific detection builds on this: model-based intrusion detection for Modbus/TCP, for example, learns a deterministic model of each highly periodic HMI-PLC channel and flags deviations from it [23]. This matters because some of the most damaging ICS attacks are not floods but precise manipulations of control or sensor data. In a false-data-injection attack the adversary alters the values the controller reads or writes so that the process is driven to an unsafe state while its own readings appear normal; Etxezarreta et al. identify false-data injection as one of the main threats to ICS and propose a replica-based moving-target defence against it [3]. Network reconfiguration that exploits the attacker's uncertainty has been applied to the same threat in SCADA settings [24]. Reconnaissance is the usual first stage of such campaigns, and defending against it in ICS has itself been studied as a moving-target-defence problem [25]. The asset-discovery tools that reconnaissance depends on have themselves been systematised into a taxonomy of industrial scanning tools, contrasting how deeply each fingerprints a device [26], which is the capability CARS denies an unregistered scanner at the network layer. In these defences, where the operation is recovered it informs the detection, whether a request is judged malicious, rather than the response. Grading the action by the operation, permitting a read while refusing a write, and elevating that write again when its target is a critical asset, is the axis CARS adds on top of detection. CARS recovers the operation from the traffic and uses it in the decision, so that a read from a PLC and a write to it are not treated alike.

## 2.8 Industrial control system testbeds

Evaluating ICS security work requires a realistic environment, and building one is itself a recognised research problem. Antonioli and Tippenhauer present MiniCPS, a toolkit built on the Mininet emulator for reproducible security research on cyber-physical system networks, supporting simulated devices and industrial protocols [27]. Gardiner et al. observe that ICS security work often cannot be evaluated on real systems for safety and operational reasons, and distil design principles and lessons from building ICS security testbeds of their own [28]. This project takes the higher-fidelity option. Rather than a pure emulation, it uses a hardware testbed with two real Siemens PLCs driving a live physical process through the SDN fabric, so that a response can be judged against its actual effect on the process. The design of that testbed is described in Chapter 3.

# Chapter 3

# Design and Implementation

This chapter presents CARS as a design and then its realisation, arranged from the general to the specific. It opens with the threat and attacker model the design answers, then sets out CARS as an architecture, the layered enforcement pipeline, the criticality and operation model at its centre, and the reference implementation of the detection, the responses and the self-checks. It then shows that this design is portable, a core separated from site-specific configuration rather than tied to one plant, before describing the concrete testbed on which CARS was instantiated and the deployment that produced the results of Chapter 4. The design is the contribution; the testbed is one instantiation of it, used to validate it against a real physical process.

## 3.1 Threat and attacker model

### 3.1.1 Assumptions and capabilities

In keeping with the setting of the project, the attacker is assumed to have already reached the operational-technology network, whether by moving laterally from an enterprise network or by direct connection, which is the starting point taken by related work on defending ICS against reconnaissance [25]. Three attacker positions are considered, and all three are exercised in the evaluation; their concrete instantiation on the testbed is given in Section 3.7. The first is an unregistered external host that is present on the OT segment but holds no legitimate role. The second is a trusted device that has been compromised, so that its legitimate network identity is used to issue illegitimate commands. The third is a compromised control plane, in which the SDN controller itself, or a party interposed on its channel, manipulates the forwarding rules, which is the threat identified by Gardiner et al. for software-defined ICS networks [4]. CARS answers this third position only in part, and the boundary is drawn deliberately. Its self-checking layer detects tampering with the forwarding state against a trusted baseline, and its authenticated control interface refuses a silent disarm (Section 3.5); but a controller that is fully subverted, in command of its own decision logic and aware of the defence's own internal markings, is beyond what a self-checking control plane can catch. That case is therefore out of scope as something CARS solves: CARS detects and resists control-plane tampering, it does not defend against a controller that has been turned entirely. From its position the attacker can send arbitrary traffic, including forged and out-of-state packets, and can spoof the network-layer and link-layer identity of other hosts.

### 3.1.2 Attacker goals

The attacker's objective is to disrupt or seize control of the physical process, or to gather the information needed to do so. The goals exercised in the evaluation, in the order an intrusion tends to follow, are the following.

- **Reconnaissance.** Discover the hosts on the segment and identify the PLCs, including model and firmware, to select an exploit [25].
- **Identity spoofing.** Impersonate a trusted host at the network or link layer to obtain the access that the identity carries.
- **State manipulation.** Inject forged or out-of-state traffic to slip past a stateless control or to disturb an established session.

- **Control-plane and flow tampering.** Insert, remove or rewrite forwarding rules, or attempt to disarm the defence, from a compromised control plane.
- **Unauthorised control.** Issue an operation that a given role should not perform on a given asset, such as writing an actuator or downloading a program to a safety-critical PLC.
- **False-data injection.** Falsify sensor or control values so that the PLC drives the process to an unsafe state while its own readings appear normal [3].

### 3.1.3 Attacker knowledge and scope

Attacker knowledge depends on position. The external host begins with no knowledge of the roles or addresses on the segment and must discover them, whereas the compromised trusted host already holds a legitimate identity and knows the conduits it is permitted to use. Knowledge of the defence itself is treated by Kerckhoffs's principle, not by obscurity: the attacker is assumed to know that CARS is present and how it works in general, its layered pipeline, its grading by asset criticality and industrial operation, and its bounded self-healing responses. The design does not depend on the attacker's ignorance of it, and this is why the assumption is safe to grant: the identity binding, the conduit allowlist and the default-deny refuse an unregistered or forged source whether or not the attacker knows they are there, and the reactive stage acts on the operation recovered from the packet rather than on the attacker failing to anticipate it. Two secrets are withheld, and neither is obscurity of the mechanism: the authentication token that gates the control interface, and the internal cookie markings that separate the proactive policy from the reactive responses for the flow-integrity check. No in-scope attacker holds either; the markings would help only a party that can install flow rules, which is the control-plane position, and a controller subverted far enough to know them is placed out of scope in Section 3.1. The model is bounded deliberately. Physical attacks on the devices, supply-chain compromise, and attacks on authentication mechanisms are out of scope, since the focus is network-borne attacks on the process once the perimeter has been breached, following the same scoping as related ICS reconnaissance work [25]. Two residual boundaries are carried honestly through the evaluation. An endpoint that is fully compromised while acting only within the operations its role is permitted is inside the trust the design grants that role. A fully subverted controller that knows the defence's own internal markings is outside what a self-checking control plane can detect. Both are examined in Chapter 4.

## 3.2 CARS: architecture and principles

CARS is a response framework rather than a fixed configuration. Stated in general terms, before any particular plant, it is a controller that governs a software-defined fabric carrying industrial traffic and conditions its response on two axes a uniform block ignores: the criticality of the asset under attack, and the industrial operation recovered from the packet. The response it enforces is bounded in time, reversible, and evidence-generating, which is the property that lets an operator arm it on a live process. Everything specific to a site, the assets and their tiers, the roles, the conduits and the rulebook, is configuration supplied to this framework, not part of it (Section 3.6).

CARS is placed as the enforcement point in the SDN fabric, between the operational hosts and the assets it protects, as shown in Figure 3.1. Its design is layered. A proactive layer, always in force, binds each host's identity and admits only the connections on a short allowlist, so an unregistered host is refused before any operation is considered. A reactive layer recovers the operation from the traffic by deep packet inspection, decides its tier from the source role, the operation and the criticality of the target, and applies a graded response. A self-checking layer verifies the forwarding state against a trusted baseline and authenticates the control interface, so that a compromise of the control plane is caught rather than trusted.

Two separations shape the design. The controller, which decides, is separate from the switches, which enforce, the standard software-defined split that lets one policy govern the whole fabric. The decision, which is a tier, is separate from the response, which is an action, so that a single judgement drives a graded ladder rather than one blunt action. Every response is bounded in time, reversible, and leaves an audit record, which is what allows an operator to enable it. The remaining sections detail each layer: the pipeline that enforces the policy (Section 3.3), the criticality and operation model that decides (Section 3.4), and the reference implementation of the detection, the responses and the self-checks (Section 3.5); Section 3.6 then shows the design is portable across sites, and Section 3.7 describes the testbed on which it was instantiated and validated.

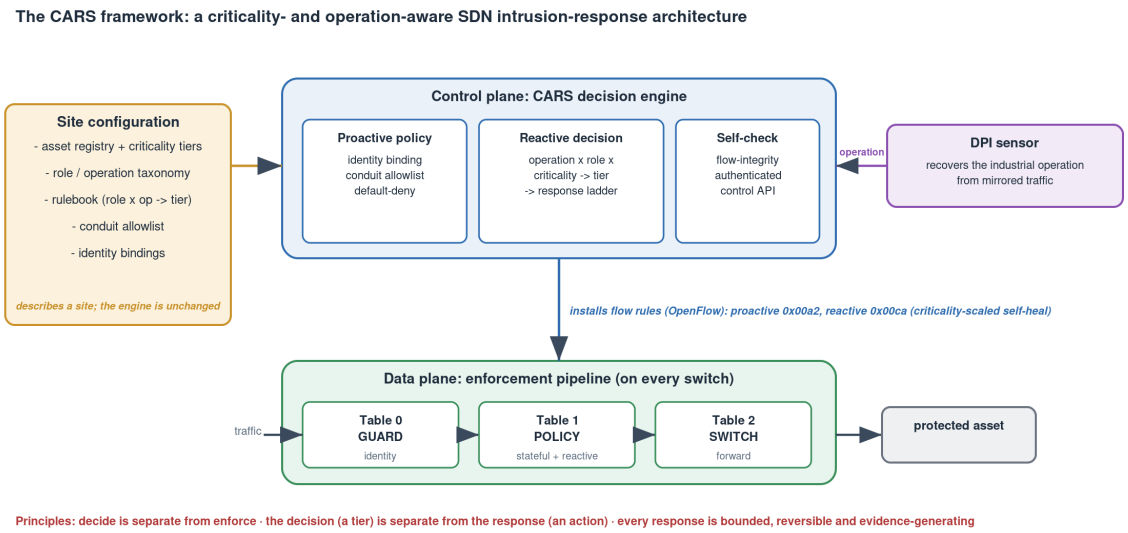


Figure 3.1: The CARS framework, independent of any particular plant. A control-plane decision engine holds the three layers, proactive policy, the reactive criticality- and operation-aware decision, and the self-check; a deep-packet-inspection sensor supplies the operation; and the engine installs rules into a data-plane enforcement pipeline, GUARD then stateful POLICY then SWITCH, on every switch it governs. Everything specific to a deployment is a site configuration the engine loads; the design principles are stated below the figure. The concrete testbed instantiation of this architecture is Figure 3.7.

## 3.3 The enforcement pipeline

Before the pipeline itself, Figure 3.2 gives the general shape of the system as a site-independent reference architecture. A single CARS controller, the portable decision engine, is loaded with a site configuration and governs the OpenFlow switches over OpenFlow 1.3; a deep-packet-inspection sensor mirrors switch traffic to recover the industrial operation; and the same three-table pipeline is installed identically on every switch, in front of the protected assets of the operational-technology segment. Nothing in the figure is tied to a plant: the switches are `1` to `n`, the assets are generic, and each carries a criticality tier `w`. This is the portable component view; the abstract framework was Figure 3.1 and the concrete testbed instantiation is Figure 3.7.

CARS enforces its policy in the data plane as a pipeline of three OpenFlow tables, installed on every switch in the fabric so that the same rules apply in each cell. A packet reaches its destination only if it passes all three stages; failing any stage drops or diverts it. The pipeline, and the reactive decision that installs rules into it, are shown together in Figure 3.3. The flow rules quoted below are read from the running switches rather than from the source, and the full tables are reproduced in Appendix A, so that the description matches the deployed system exactly.

Table 0 performs identity binding, the defence against spoofing. Each protected host is bound to the switch port, MAC address and IP address from which it is permitted to speak. A packet whose source matches its binding, for both IP and ARP, is passed to Table 1; traffic arriving on an inter-bridge link, which has already been checked at the far switch, is passed on as well. A packet that claims a protected address but arrives on any other port matches no binding and is dropped, for both IP and ARP, while traffic that claims no protected address is passed on. The effect is that an attacker cannot borrow the network identity of a trusted host: a forged source is dropped at ingress before it reaches the policy stage. The concrete match priorities that order these rules are read from the running system in the reference implementation (Section 3.5) and reproduced in full in Appendix A.

Table 1 is the stateful policy stage, and it carries connection state in the data plane in the manner introduced by AVANT-GUARD [21]. An untracked packet is first submitted to connection tracking and re-enters the table. A packet belonging to an already established connection is admitted, which is what allows a protected device such as the HMI to receive the replies to connections it began while remaining shielded from new inbound connections. New connections are then judged against the proactive policy. A small allowlist of conduits, each naming a source, a destination and a service port, is permitted and committed to the connection table; each deployment supplies its own conduit set by configuration, and the

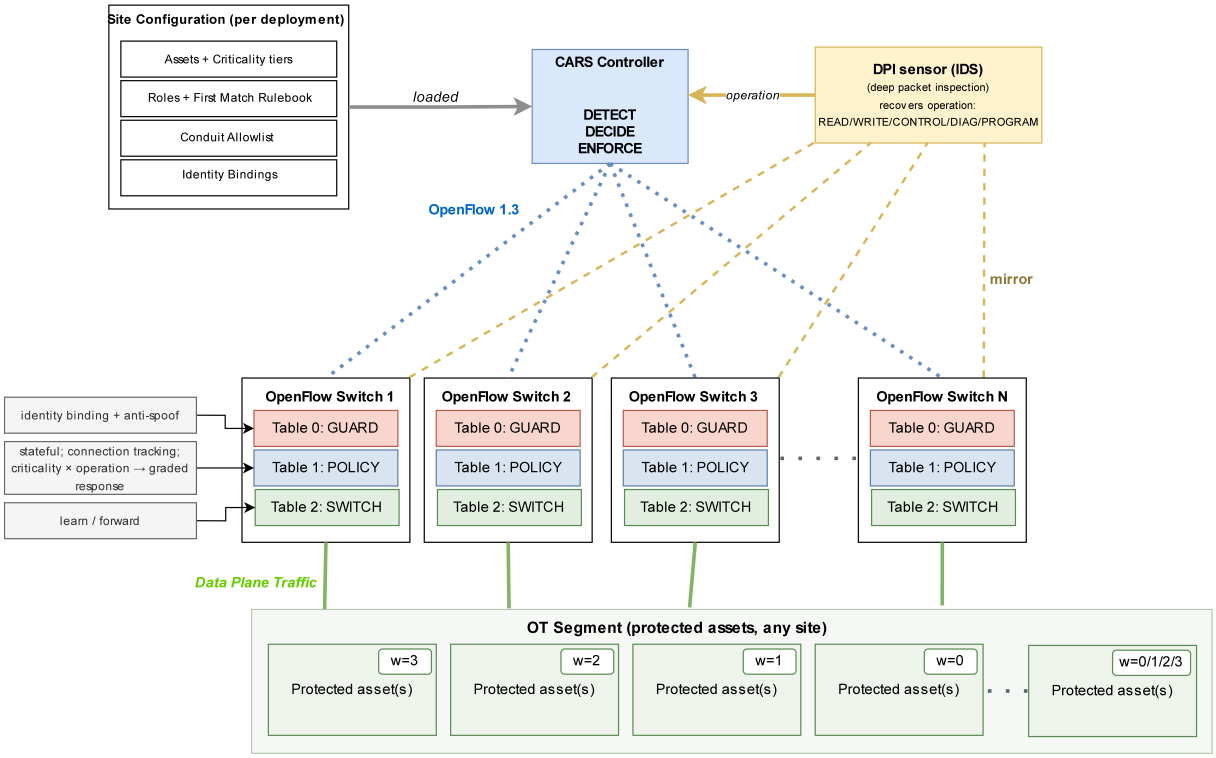


Figure 3.2: The system architecture as a site-independent reference. The CARS controller, the portable decision engine (detect, decide, enforce), is loaded with a per-site configuration and programs the OpenFlow switches over OpenFlow 1.3 (blue dotted); a deep-packet-inspection sensor mirrors switch traffic to recover the industrial operation (amber dashed); and process traffic crosses the switches (green) to the protected assets. Every switch runs the same three-table pipeline: GUARD (identity binding and anti-spoof), the stateful POLICY (connection tracking and the criticality- and operation-graded decision), and SWITCH (learn and forward). The switches are `1` to `n` and the assets are generic, each carrying a criticality tier `w` (3 critical to 0 low); the engine is identical across sites, only the configuration changes. The concrete testbed instantiation of this architecture is Figure 3.7.

concrete list installed on the testbed is enumerated in Appendix A.2. A new connection to a protected asset that matches no conduit is dropped, the default-deny that closes an asset to everything except its named conduits, and a new connection to any other asset is committed and passed. The reactive responses described in Section 3.4 are installed into this same table by the controller at higher priority, so that an isolate or a block takes precedence over the proactive rules for as long as an attack persists; the concrete priority bands are read from the running system in the reference implementation (Section 3.5) and reproduced in full in Appendix A.

Table 2 forwards. It floods broadcast traffic, sends a packet whose source and destination MAC pair has been learned to the correct port, and refers an unknown destination to the controller so that the source can be learned. Separating the three concerns, identity at Table 0, connection state and policy at Table 1, and forwarding at Table 2, means each is enforced on its own, and a packet cannot reach forwarding by evading an earlier stage. The admission logic a packet meets is stated as pseudocode in Listing 3.1, in the priority order the switch evaluates.

```
on packet P at a switch, ingress port p:
    // Table 0 - identity binding (anti-spoof)
    if (p, P.mac, P.ip) is a registered binding:  goto Table 1   // bound identity passes
    else if p is an inter-switch uplink:           goto Table 1   // already checked upstream
    else if P.ip is a protected address:           DROP           // claimed a protected identity
      from the wrong port
    else:                                           goto Table 1

    // Table 1 - stateful policy, highest matching priority wins
    if a reactive rule matches P.src:              DROP or meter  // active isolate/block/throttle
    else if P is an established connection:         goto Table 2   // return traffic, allowed session
    else if (P.src, P.dst, P.port) is allowlisted: commit; goto Table 2   // a named conduit
```

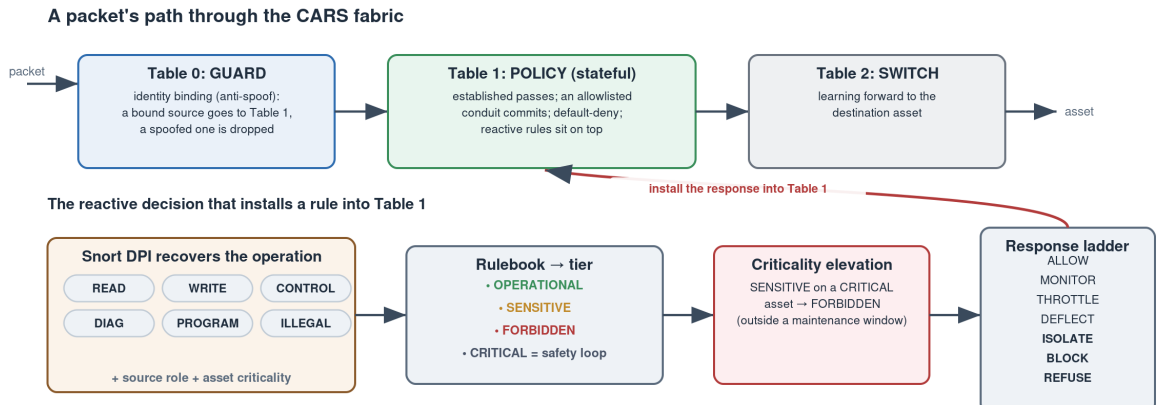


Figure 3.3: A packet's path through the CARS fabric, and the reactive decision that installs rules into it. Above, the three-table pipeline: a packet is admitted only if its identity binds at Table 0 (GUARD), its connection is permitted at the stateful Table 1 (POLICY), and it is then forwarded at Table 2 (SWITCH). Below, the decision that produces the reactive rules on Table 1: deep packet inspection recovers the operation, the rulebook maps the source role, operation and destination to a tier, a sensitive operation on a critical asset is elevated, and the tier, weight and rate select the graded response, installed into Table 1 with a criticality-scaled self-heal. Drawn for this report from the deployed pipeline, rulebook and deep-packet-inspection rules.

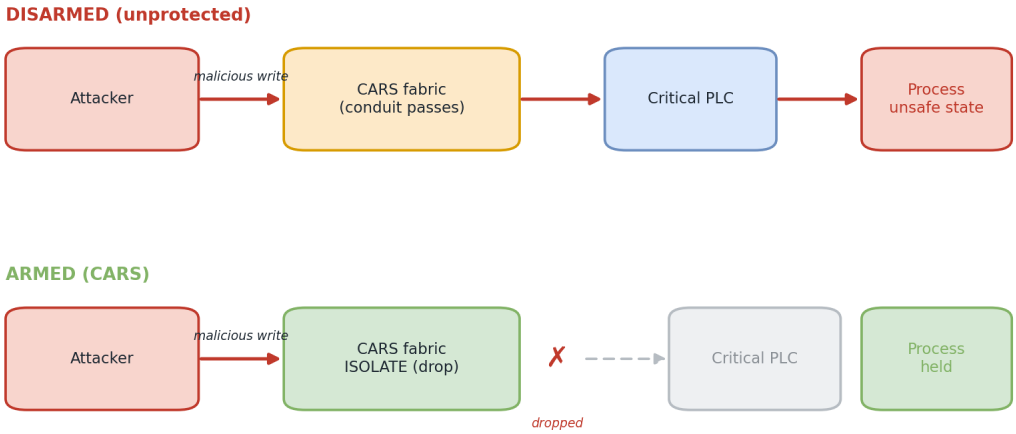


Figure 3.4: The path of a malicious write in each state of the defence. Disarmed, the write crosses the fabric, reaches the critical PLC and drives the process to an unsafe state. Armed, the reactive rule drops it in the fabric, so it never reaches the PLC and the process is held. The armed-versus-disarmed contrast is measured in Chapter 4.

```
    else if P.dst is a protected asset:          DROP          // default-deny
    else:                                         commit; goto Table 2

    // Table 2 - learning switch: forward to the learned port, else flood and learn
```

Listing 3.1: The three-table admission a packet meets, in the switch's priority order: identity binding, then the stateful policy with the reactive responses on top, then forwarding.

The proactive and reactive rules carry distinct cookies, so the flow-integrity checker of Section 3.5 can tell the fixed policy from the responses installed at run time; the concrete cookie values are given there and in Appendix A. What this pipeline does to a single malicious write, in each state of the defence, is shown in Figure 3.4.

| Tier | $w$ | Assigned to an asset whose compromise... | Response effect |
|---|---|---|---|
| CRITICAL | 3 | directly endangers safety or the running process, such as a PLC on a live control or safety loop | hold $30{+}15{\cdot}3 = 75$ s; escalates soonest |
| HIGH | 2 | causes serious operational disruption, such as an operator HMI or a secondary controller | hold $30{+}15{\cdot}2 = 60$ s |
| MEDIUM | 1 | has operational value but a recoverable loss, such as a historian or data collector | hold $30{+}15{\cdot}1 = 45$ s |
| LOW | 0 | is of limited consequence, such as an auxiliary or low-value unit | hold $30{+}15{\cdot}0 = 30$ s; escalates latest |

Table 3.1: The criticality model, defined generally for any OT asset set. An asset is placed in a tier by the consequence of its compromise, and the tier's weight $w$ scales the response: it sets the block or isolate duration to $30 + 15w$ seconds and brings escalation forward at higher weight. The tiers are independent of any particular plant; the concrete assignment for the testbed on which CARS is evaluated is given in Chapter 4, Table 4.3.

## 3.4 The criticality and operation model

CARS makes two kinds of decision. At the connection level a proactive policy fixes which parties may open a connection to which asset at all. At the operation level each permitted exchange is judged by the role of its source, the operation it carries, and the criticality of the asset it targets. This section defines the model; the pipeline of Section 3.3 enforces it. The values quoted are those of the testbed instantiation (Section 3.7) and are taken from the deployed controller; the model itself is defined by role, operation and consequence, and applies to any asset set (Section 3.6).

### 3.4.1 Asset criticality

Every protected asset is assigned a criticality tier by the consequence of its compromise, following the consequence-based approach set out in Section 2.6 [5, 6]. Table 3.1 lists the criticality tiers and their weights. Four tiers are used rather than more or fewer because grading consequence into four levels is the established convention in this domain. Industrial functional-safety practice defines four safety-integrity levels, SIL 1 to SIL 4, the risk reduction required of a safety function, allocated by the risk a hazard poses and so rising with its consequence [29]; and the four-level Low, Medium, High and Critical naming that CARS adopts follows the four non-zero ratings of the CVSS qualitative severity scale, above its None baseline [30]. CARS grades each asset by the consequence of its compromise on this scale, a different axis from the security levels of IEC 62443, whose four levels grade the capability of the attacker a zone must resist rather than the consequence of the asset [31]. Four is also the fewest that separates the distinct consequence classes this deployment presents, and each asset's tier follows the consequence of its compromise rather than its device type: a controller on the safety-critical process is CRITICAL, an operator panel or secondary controller whose loss is a serious operational disruption is HIGH, a data collector whose loss is recoverable is MEDIUM, and a low-value auxiliary unit is LOW; the same device type can therefore sit at different tiers where its consequence differs. The weights are the integers three down to zero so that they scale the response linearly and carry no finer numeric meaning than that ordering. The weight does two things in the response. It sets the duration of a block or an isolate, computed as thirty seconds plus fifteen for each unit of weight, so a critical asset holds an attacker out for seventy-five seconds against thirty for a low one, and it brings escalation forward, so a higher-consequence asset reaches a cut sooner. At the lowest tier both effects vanish by construction: at weight zero the fifteen-second term is zero and the escalation stays at its slowest, so a LOW asset receives the base response unchanged and the criticality mechanism adds nothing to it. This is deliberate rather than incidental, since an asset left untiered also resolves to LOW: the tier is the framework's neutral default, the behaviour it shows when asset criticality contributes nothing, and the higher tiers are defined by what they add above this floor. Criticality therefore shapes not only whether CARS acts but how hard and for how long. Why these durations take the values they do, the thirty-second base set to outlast one automated TCP reconnect and the fifteen-second step chosen to space the tiers rather than tuned to a measurement, is justified in the response-ladder subsection (Section 3.5.4) and shown as the measured ladder in Figure 3.6.

| Source role | Destination | Operation | Tier |
|---|---|---|---|
| hmi, plc (the loop) | plc, hmi | any | CRITICAL |
| remediation | plc | any | OPERATIONAL |
| ews | plc | read, write, control | OPERATIONAL |
| any | plc, hmi | control, diag, program, illegal | FORBIDDEN |
| ews | hmi | any | SENSITIVE |
| historian, scada, supervisory | plc, hmi | write | SENSITIVE |
| historian, scada, supervisory | plc, hmi | any | OPERATIONAL |
| any | any | any | FORBIDDEN |

Table 3.2: The rulebook, evaluated first match wins (rows with a shared tier grouped for presentation; the exact deployed order is in Appendix A). A SENSITIVE operation on a CRITICAL asset, outside a maintenance window, is elevated to FORBIDDEN.

### 3.4.2 Roles and operations

Each host has a role in the registry: *plc*, *hmi*, *historian*, *scada*, *ews* (engineering workstation), *remediation*, or *gateway*. A host with no registry entry, such as the external attacker, is *unknown*. The operation carried by a packet is recovered from the industrial protocol by deep packet inspection (Section 3.5): a read of a value, a write of one, a control action on the process image, a diagnostic request, a program download, or a malformed request, together with the plain connection setup. Treating the operation as a first-class input is what lets CARS permit an engineer to read a PLC while refusing that same engineer a program download.

### 3.4.3 The rulebook and criticality elevation

The tier of an operation is decided by a rulebook of (source role, destination role, operation, tier) tuples, evaluated first match wins. First match is used so that the row ordering itself encodes precedence: a specific exception, such as the pre-authorised control rows for the engineering workstation and the remediation agent, is placed ahead of the blanket rule that would otherwise catch it, and a single ordered pass then yields a deterministic verdict without conflict-resolution logic. Table 3.2 gives the rules; the full deployed ordering is reproduced in Appendix A. The operator loop between HMI and PLC is tier CRITICAL and is never enforced against, a safety invariant so that the defence can never cut the loop that keeps the process safe. The dangerous operations are FORBIDDEN, subject to a deliberate ordering. A diagnostic, a program download or a malformed request is refused to a PLC or HMI from every source. A control write is refused from every source as well, except the two that are pre-authorised for it and whose rows are placed before the block, the engineering workstation and the remediation agent, so their legitimate actuation of the PLC classifies as OPERATIONAL rather than FORBIDDEN. Routine reads from the supervisory roles are OPERATIONAL, and an actuating write from a supervisory, historian or scada role is SENSITIVE. On top of the first match, one rule adjusts the tier by criticality: a SENSITIVE operation aimed at a CRITICAL asset, outside an authorised maintenance window, is elevated to FORBIDDEN, because nothing but the control loop and an authorised engineer may actuate the safety-critical process. The decision is drawn, with the pipeline it feeds, in Figure 3.3. The resulting tier and the observed rate then select a graded response, which is described in Section 3.5.

The decision, from the source role and operation through the criticality elevation to the graded response, is stated as pseudocode in Listing 3.2, independent of the implementation language; the deployed code that realises it is annotated in Appendix A.2.

```
function DECIDE(source, dest, operation, rate):
    role  <- registry role of source          // 'unknown' if unregistered
    tier  <- FORBIDDEN                          // default for the unknown
    for each (r, d, op, t) in RULEBOOK:         // first match wins
        if r matches role and d matches role(dest) and op matches operation:
            tier <- t
            break
    weight <- criticality weight of dest        // 3, 2, 1, 0 for CRITICAL..LOW
    if tier = SENSITIVE and weight = 3 and not in maintenance window:
        tier <- FORBIDDEN                       // elevate on a critical asset
    flood <- rate >= FLOOD_RATE and source not flood-exempt
    return SELECT_RESPONSE(tier, weight, flood, offences of source)
```

```
function SELECT_RESPONSE(tier, weight, flood, count):
    if tier = CRITICAL:
        return REFUSE                          // safety loop: mirror and alert, never cut
    if tier = OPERATIONAL:
        if not flood:  return ALLOW
        if weight = 3: return BLOCK            // a flood on a critical asset is cut
        return THROTTLE, then BLOCK on persistence
    if tier = SENSITIVE:
        if flood or weight = 3:  return BLOCK
        return THROTTLE, then BLOCK on persistence
    // tier = FORBIDDEN: a single command can harm
    if source is a shared (NAT) identity:  return BLOCK   // cut the conduit, not the host
    if count < escalation-threshold(weight):  return BLOCK
    return ISOLATE
```

Listing 3.2: The CARS operation decision. A tier is fixed by first match in the rulebook, a sensitive operation on a critical asset is elevated, and the tier, the criticality weight and the arrival rate select the graded response.

### 3.4.4 Proactive policy

Alongside the per-operation decision, a proactive policy fixes which connections may exist at all, independently of any live detection. It is a short allowlist of conduits, each a source, a destination and a service port, together with a default-deny on the protected assets. The policy is loaded from a run-time file and installed as the allowlist and default-deny flows of the stateful stage (Section 3.3); the concrete priority bands are given in the reference implementation (Section 3.5) and Appendix A. It consists of a small set of role-to-asset conduits, the operator, engineering, remediation and historian paths that a given asset is allowed, paired with a default-deny that refuses every other new connection to a protected asset. The conduit set is a property of the deployment rather than of the framework; the concrete list installed on the testbed, with its addresses and service ports, is enumerated in Appendix A.2. An unregistered host is therefore refused at the connection level before any operation is considered, while the rulebook governs what the permitted parties may then do. This allowlist-with-default-deny realises a software-defined firewall on the industrial fabric, the approach also taken for Industry 4.0 manufacturing networks [32], here narrowed to named conduits and paired with the stateful and operation-aware stages that the rest of this chapter describes.

## 3.5 Reference implementation

The design of the preceding sections is realised as an os-ken OpenFlow 1.3 controller [7], an Open vSwitch fabric and a Snort sensor, together with the pipeline of Section 3.3. The mapping of these components onto the physical nodes is given in the Deployment section (Section 3.8). This section describes how each mechanism is built. The values quoted are read from the running system.

### 3.5.1 Identity binding and connection state

Table 0 realises the anti-spoof binding as OpenFlow rules. Each protected host has a rule, at priority 200, that matches its ingress port, source MAC and source address, for both IP and ARP, and passes the packet on; a companion priority-100 rule drops any packet that claims the same address from any other port. The bindings are pinned to fixed ofports, so that re-cabling cannot silently move a host and break its binding, a fault observed and corrected during development. Table 1 carries connection state using the switch's connection-tracking action: an untracked packet is submitted to conntrack and re-enters the table, an established connection is admitted, and only a handshaked new connection that matches a conduit is committed. Because state is held in the data plane rather than at the controller, an out-of-state packet that a stateless rule would pass is refused without a round trip to the control plane, in the manner of AVANT-GUARD [21]. The full flow tables dumped from all three switches are reproduced in Appendix A.1, the deployed decision logic in Appendix A.2, the Snort detection rules in Appendix A.3, and a snapshot of the live runtime state in Appendix A.4.

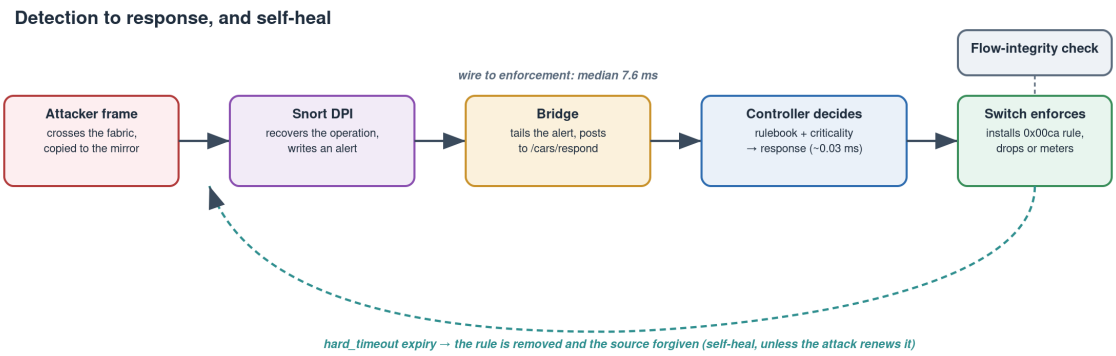


Figure 3.5: Detection to response, and self-heal. A mirrored attacker frame is classified by Snort, the bridge forwards the alert to the controller, which decides against the rulebook and criticality and installs the enforcing flow rule; the wire-to-enforcement window has a median of 7.6 ms. On `hard_timeout` expiry the rule is removed and the source forgiven, so the response self-heals unless the attack renews it, and a flow-integrity check watches the tables throughout. Drawn for this report from the deployed detection path.

### 3.5.2 Operation classification

The operation carried by a packet is recovered by deep packet inspection. Snort is used as the sensor because it is a mature signature-based inspection engine whose rule language matches the function field of these industrial protocols directly, and it runs passively on a mirror of the gateway rather than inline, so recovering the operation can neither delay nor drop the process traffic it inspects. The Snort sensor reads a mirror of the gateway switch and matches the function field of each industrial request. For S7comm to a PLC on port 102, the function byte that follows the protocol header distinguishes a read (`0x04`), a write or control action (`0x05`), and the diagnostic control and stop functions (`0x28` and `0x29`). For Modbus to the simulated unit on port 502, the function code distinguishes a register read (`0x03`), a coil write taken as a control action (`0x05`), a register write (`0x06`), the multi-register writes (`0x0F`, `0x10`), a diagnostic (`0x08`) and the encapsulated-interface program function (`0x2B`); any function code above the defined range is flagged as malformed. These are the operation classes the decision of Figure 3.3 reasons about. Each match raises an alert that names the operation, which is what allows the decision of Section 3.4 to treat the same connection differently according to what it carries.

### 3.5.3 From detection to response

Detection and response are joined by the chain shown in Figure 3.5. Snort writes each alert to a file; a bridge process follows the file and forwards each alert, with its source, destination and recovered operation, to the controller's `/cars/respond` interface; the controller classifies the operation against the rulebook and criticality, selects a response, and installs the corresponding flow rule. The controller's own decision time is about 0.026 ms on average (measured in Section 4.6), so the path from a packet on the wire to an installed rule completes in the low milliseconds.

### 3.5.4 The response ladder

The controller selects one of seven graded responses and realises it in OpenFlow. ALLOW and MONITOR admit the traffic and record it. THROTTLE attaches a rate-limiting meter to the conduit. DEFLECT rewrites the destination to a honeypot and installs the forward and reverse rules for it. ISOLATE drops all traffic from the offending source at priority 110, and BLOCK drops the single offending conduit at priority 100. REFUSE is reserved for the safety-critical loop and only mirrors and alerts, never enforcing, so that the operator's control of the process can never be cut by the defence. The seven responses, what each does on the wire and when it is selected, are summarised in Table 3.3. Of these, DEFLECT is provided by the engine for completeness but is not exercised in the evaluation of Chapter 4; the responses

| Response | What it does on the wire | Selected when |
|---|---|---|
| ALLOW | forwards the traffic and records it; installs no reactive rule | the operation is legitimate for the source, operation and asset |
| MONITOR | forwards but raises an alert for the record | a permitted operation is worth watching |
| THROTTLE | attaches a rate-limiting meter to the conduit | a SENSITIVE actuation, or a non-critical flood, that should be slowed rather than cut |
| DEFLECT | rewrites the destination to a honeypot and installs the forward and reverse rules | the source is to be drawn off to a decoy rather than dropped (provided by the engine; not exercised in this evaluation) |
| ISOLATE | drops all traffic from the offending source | a FORBIDDEN operation from a source that persists; quarantines the whole host |
| BLOCK | drops the single offending conduit | a FORBIDDEN first offence, a shared NAT identity, or a flood on a critical asset; cuts the one flow, not the host |
| REFUSE | mirrors and alerts only; installs no rule, never enforces | the operator control loop (tier CRITICAL); the safety cap that the defence must never cut |

Table 3.3: The response ladder: each graded response, its OpenFlow realisation, and the condition that selects it (Listing 3.2). The scaled quantity is the timeout, $30+15w$ seconds, applied to whichever reactive rule ISOLATE, BLOCK or THROTTLE installs; ALLOW, MONITOR and REFUSE install no reactive rule.

the evaluation drives are ALLOW, MONITOR, THROTTLE, ISOLATE, BLOCK and REFUSE. Every reactive rule carries the cookie `0x00ca` and a criticality-scaled timeout of $30 + 15w$ seconds, so it expires and self-heals unless the attack persists.

The thirty-second base is set to outlast an automated reconnect: a default TCP retransmission sequence spans roughly thirty-one seconds, so a thirty-second quarantine survives a full reconnect attempt rather than releasing the source mid-retry, while staying short enough that a mistaken block on a legitimate source clears in well under a minute. The fifteen-second step per weight makes the hold proportional to consequence, producing the 30, 45, 60 and 75 second ladder across the four tiers of Figure 3.6, confirmed live in Section 4.3. The fifteen-second step is a chosen design parameter, not a derived one: the design requires only that the hold rise with consequence and that the four tiers stay clearly separated, and any positive step meets that, so fifteen seconds was chosen to give a well-spaced ladder, a 2.5-fold spread from the thirty-second low tier to the seventy-five-second critical tier. Its value is not load-bearing; a larger step would only punish a false positive harder and a smaller one would blur the tiers.

This timeout-driven design is a deliberate choice over an alert-only or an operator-driven one, and it answers the trust problem the project set out to solve. An operator-driven response, in which the defence only raises an alarm and a human then installs a rule by hand, is the posture that already prevails in operational technology, and it is too slow for the attack it must answer: the false-data injection of Section 4.4.1 needs a single write to reach the PLC, and the reaction window is milliseconds, so by the time an operator has read an alarm and acted the write has landed. Automation is therefore necessary at the attack's timescale. What makes the automation safe to arm is not that a human is kept in the loop but that the automated action is bounded and reversible: the hard timeout makes every response temporary, so a response installed in error, a false positive on a legitimate source, reverts itself within at most seventy-five seconds with no operator action. It is this self-correction, rather than a longer human review, that lets an operator leave the defence armed. The operator is not removed from the loop; every decision is logged as evidence, and the manual controls, a direct block or unblock, a defence disarm, and an authorised maintenance window, remain available for the judgement the automation should not make.

The timeout does not open a window an attacker can wait out. A reactive rule is renewed on every continued detection: each forbidden operation re-installs the rule and resets its timer, so a source that keeps attacking stays cut for as long as it persists, and the timeout expires only when the attack actually stops. An attacker that pauses for the hold to lapse and then resumes is re-detected and re-cut on its very next forbidden packet, within the same millisecond reaction window, so the interval between expiry and re-isolation admits at most the single first-packet leak already bounded in Section 4.6, which does

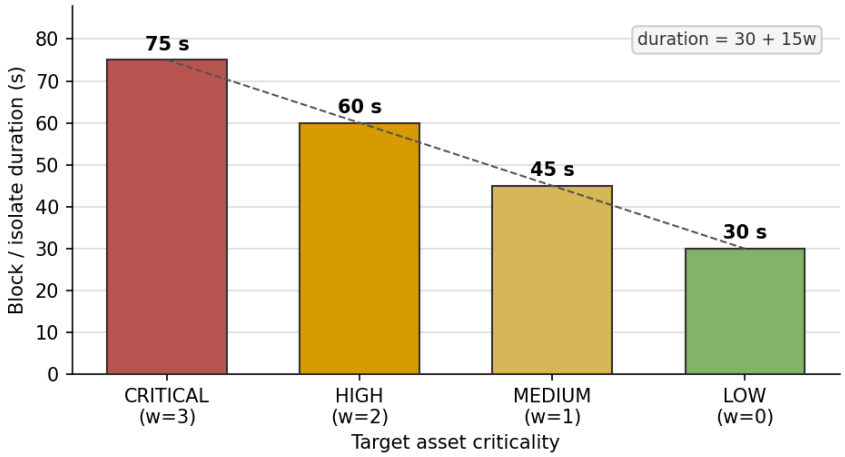


Figure 3.6: The criticality-scaled timeout ladder $30 + 15w$: an ISOLATE or BLOCK against a target of weight 3, 2, 1 and 0 holds for 75, 60, 45 and 30 seconds. Read from the deployed engine, holding the actor and operation fixed and varying only the target's criticality tier, so the figure shows the design's timeout scaling as the running system produces it.

not move the process. The expiry is therefore a release for a stopped attacker or a mistaken block, not an opening for a waiting one. Requiring an operator to confirm each release, rather than letting the hold expire on its own, would defeat the property that makes the automation armable in the first place: a response installed in error would then stay in force until a human noticed and cleared it, turning a bounded, seconds-long false positive into an outage of unbounded length, and it would add nothing against a real attacker, who is re-cut by renewal whether or not anyone is watching. Release is for that reason automatic on expiry, with the manual block, unblock, disarm and maintenance window always available for the judgements an operator does want to make.

The length of the hold is tuned to this trade-off, not to operator reaction time. A longer timeout would punish a false positive harder, holding a wrongly-cut legitimate source out for minutes rather than seconds, which is the outcome the design most wants to avoid; and it is unnecessary, because a genuine attacker is held by renewal for as long as it persists, and the operator can extend or lift any response by hand regardless of the timer. The base is set at thirty seconds only to outlast one automated TCP reconnect, and the criticality step lengthens the hold on higher-consequence assets; beyond that, duration is a cost to legitimate work under a false positive rather than a benefit.

How a chosen response becomes a flow rule, and how it self-heals, is stated as pseudocode in Listing 3.3.

```
function ENFORCE(response, source, dest, weight):
    timeout <- 30 + 15 * weight                    // 75 / 60 / 45 / 30 s for CRITICAL..LOW
    if response = ISOLATE:
        install drop  matching (src = source)               // quarantine the source
    else if response = BLOCK:
        install drop  matching (src = source, dst = dest)   // cut the one conduit
    else if response = THROTTLE:
        install meter matching (src = source, dst = dest)   // rate-limit, do not cut
    stamp the rule with cookie 0x00ca and hard_timeout = timeout

on hard_timeout expiry of a reactive rule:
    remove the rule and clear the source's offence count    // self-heals unless renewed
```

Listing 3.3: Turning a response into an OpenFlow rule with a criticality-scaled timeout, and the self-heal on expiry that releases it unless the attack renews the detection.

### 3.5.5 Forwarding integrity and control authentication

Two mechanisms protect CARS against a compromised control plane, the threat identified by Gardiner et al. [4]. A flow-integrity checker holds a trusted baseline of the proactive policy, distinguished by its cookie `0x00a2`, and compares the live tables against it every ten seconds; a rule that is missing, added or altered, other than the reactive rules under cookie `0x00ca`, is reported as drift, catching bogus injected

rules, black-holes and removals in the manner of the policy checker of Melis et al. [20]. The ten-second interval is the knee of a trade-off: it bounds the time to catch a persistent injection at one poll while the cost of dumping three small flow tables every ten seconds is negligible, and a shorter interval narrows but cannot close the window, since a rule injected and withdrawn inside one poll still evades it. Separately, every state-changing control operation, such as disarming the defence, requires an authentication token, so a host that reaches the control interface cannot silently switch CARS off; an unauthenticated attempt is refused and logged.

## 3.6 Portability and scalability

The design of the preceding sections is deliberately independent of the plant it protects. The pipeline, the response ladder, the criticality model and the rulebook are a specification; the assets, roles, conduits, bindings and tiers that fill them are configuration. The two are separated in the implementation: the engine's decision logic, its pipeline install and its response repertoire are the portable core, and everything specific to a deployment, the asset registry and its tiers, the conduit allowlist, the rulebook rows, the identity bindings and the fabric's uplinks, is expressible as a site file the same engine consumes. A site is described, not coded. Standing up CARS on a different plant means writing that file, not changing the engine: a new asset is a registry line with a tier, a new permitted path is a conduit line, a new response weighting is a tier weight. That the configuration extracted for the testbed reproduces the deployed policy exactly is checked by a parity test against the running engine, so the separation is verified rather than asserted. The backbone shown for the testbed in Section 3.4, the criticality tiers, the role and operation taxonomy, the decision matrix and the flow tables, is therefore not specific to this testbed; it is one instantiation of a portable model, and the same engine drives any site whose configuration it is given.

Scale follows from the same separation. The pipeline is installed identically on every switch the controller governs (Section 3.3), so extending the fabric is adding switches, which the controller discovers and programs without code change, and adding assets is extending the registry and the allowlist. The bounds of the reactive stage were measured directly in Section 4.10.1: it scales to tens of thousands of concurrent responses on the deployed hardware, and the criticality-scaled timeout bounds that state without operator action.

Independence from the hardware is demonstrated, not merely asserted. The same engine, the same pipeline and the same rulebook run in a hardware-free emulation in which the physical PLC is replaced by a software PLC and the fabric by software switches, so the full defended attack, both response layers, can be reproduced on a single machine with no PLC, switch or cabling. The testbed of Section 3.7 gives the real, physically-grounded validation; the emulation shows the design is not tied to that particular hardware. Both are published so the result can be reproduced either way.

## 3.7 Testbed instantiation

To validate CARS against a live process on real control hardware, the design was instantiated on a hardware testbed rather than a pure emulation, so that a response can be judged against its effect on a running process driven by real PLCs, with the tank itself a hardware-in-the-loop co-simulation. The PLC, the S7 traffic, the control logic and the enforcement are all real; only the plant, the tank and its fluid, is co-simulated, in Factory IO. A physical water tank was not available for this project, so the plant is modelled in software; hardware-in-the-loop was preferred to a pure software simulation for a technical reason as well, in that it keeps everything the defence acts on real, the controller, its control logic and the industrial protocol, and co-simulates only the physical process, so the evaluation exercises a real PLC and real S7 traffic rather than an emulated stand-in. It also lets the process be driven past its safe limit to overflow, so the defence can be measured against a genuine physical-harm attempt. The instantiation fills the site configuration of Section 3.6 with the assets, roles and conduits this bench presents: four machines and two real PLCs, arranged as the two-cell topology of Figure 3.7. The physical process cell, with its PLC and operator HMI, is shown in Figure 3.8.

### 3.7.1 Nodes and devices

The testbed is built on physical machines rather than an emulation. A dedicated controller node runs the CARS controller as an os-ken 2.8.1 OpenFlow 1.3 application. os-ken was chosen because it is the actively

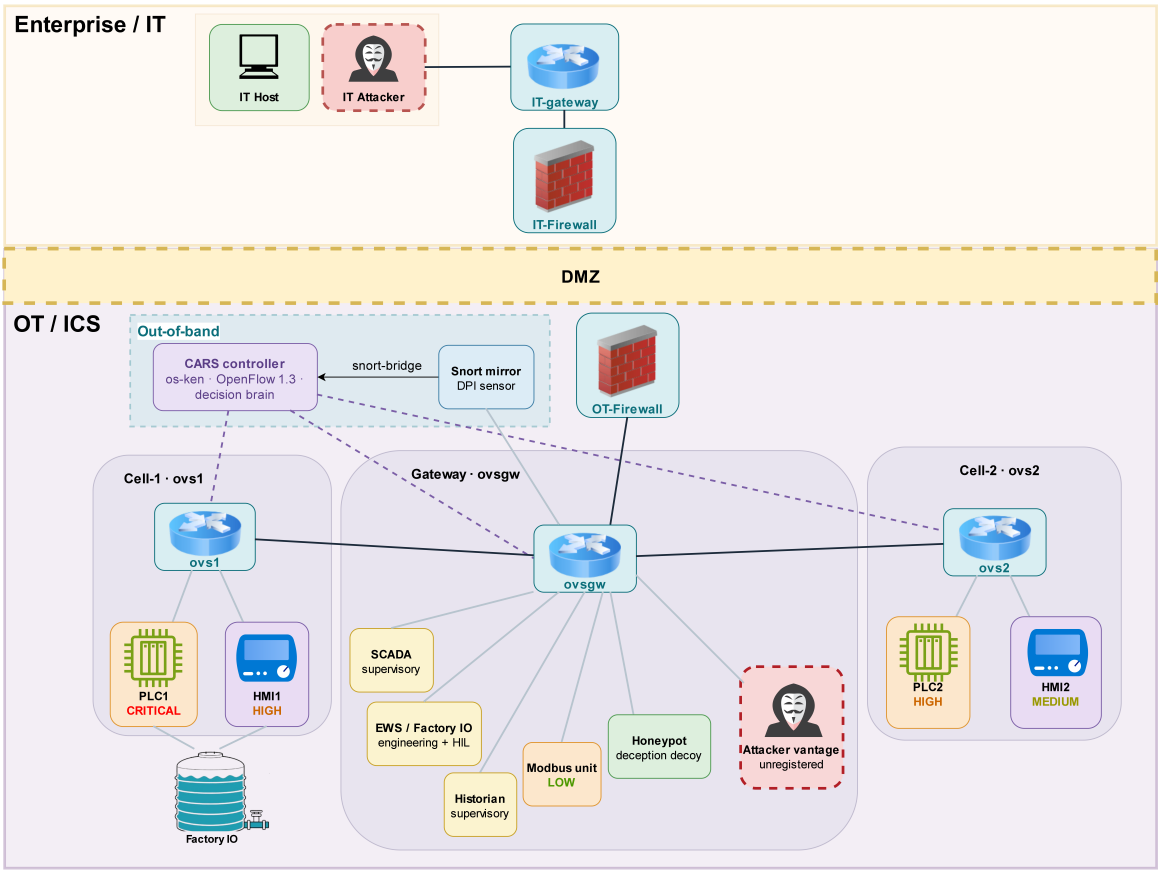


Figure 3.7: The SDN-ICS logical topology. Three OpenFlow switches sit under a single CARS controller: Cell-1 on `ovs1`, the gateway on `ovsgw`, and Cell-2 on `ovs2`. Above the OT fabric is the modelled perimeter exercised in Chapter 4: an enterprise/IT zone and a DMZ separated from the process network by an enterprise and an OT firewall, the latter source-NATing every north-to-south crossing onto the gateway identity. This is distinct from the Cell-2 NAT clone, through which `ovs2` is addressed on its own subnet. Link types are distinguished by style: *control-plane* links (dashed purple) carry OpenFlow between the controller and the three switches on an out-of-band management network; *data-plane* links (solid dark) carry process traffic across the fabric; and the *mirror* link (grey) copies gateway traffic out-of-band to the Snort sensor. Protected assets are shown by functional role and criticality tier. The topology is that of the running testbed; the host addresses and the full port and binding map are read from the live fabric (`ovs-vsctl show`, Figure 3.9) and listed in Appendix A.

maintained fork of Ryu, whose own development had ended, and it keeps Ryu's Python OpenFlow 1.3 application model, in which the whole decision logic of Section 3.4 runs as a single controller program rather than being spread across a heavier controller framework. The fabric is three Open vSwitch 3.3.4 bridges speaking OpenFlow 1.3 to that controller: `ovs1` carries Cell-1, `ovsgw` the gateway, and `ovs2` carries Cell-2. Open vSwitch was used because it is the standard open, programmable software switch and it supports the two data-plane features the pipeline of Section 3.3 depends on, multi-table matching and kernel connection tracking, which a fixed-function hardware switch would not expose; OpenFlow 1.3 provides the multi-table pipeline, and Open vSwitch supplies the connection-tracking action used with it. The switches `ovs1` and `ovsgw` run together with the Snort sensor and the supporting services on one node, which also hosts the simulated supervisory and attacker endpoints in Linux network namespaces, while `ovs2` runs on a separate node. A Windows machine runs TIA Portal and Factory IO and acts as the engineering workstation. The two PLCs are Siemens S7-1200 units; PLC1 is a 1212C, model 6ES7 212-1BE40-0XB0 running firmware 4.2.3, confirmed by scanning it during evaluation. An operator HMI is present on each cell. The supervisory hosts, the SCADA or Modbus operator, the historian, the remediation agent and the Modbus unit, are realised as namespaces on the fabric-and-sensing node, each bound to a switch port, and the external attacker is a Kali Linux virtual machine on that node, attached to the OT segment and holding no registered role.

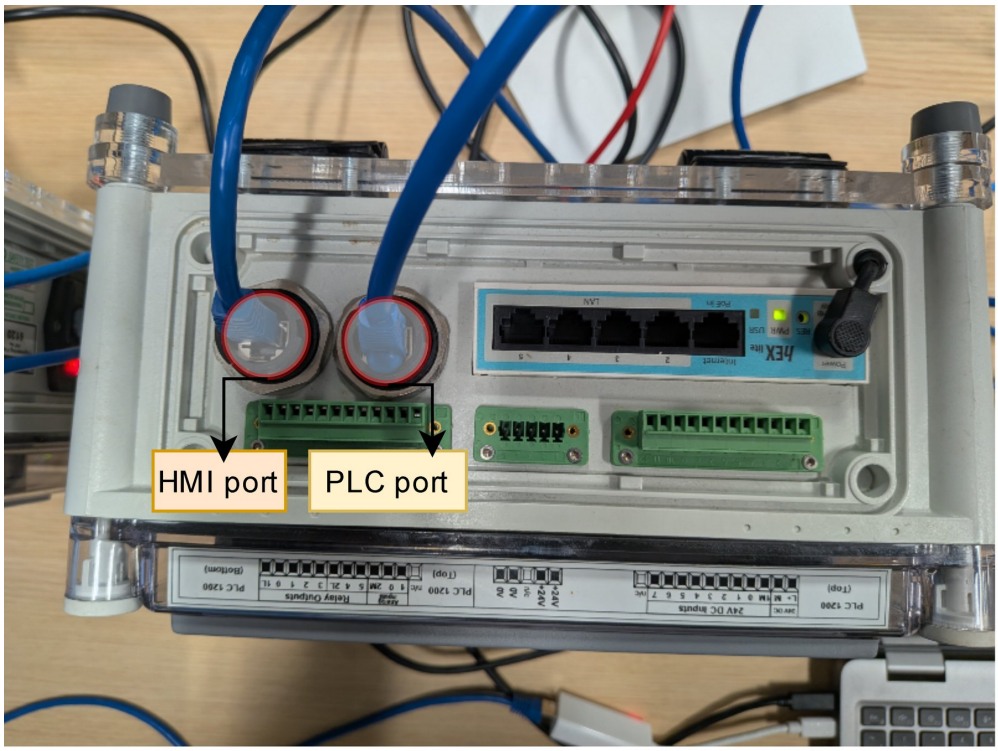


Figure 3.8: The physical process cell. The Siemens S7-1212C PLC and its operator HMI are wired into the cell through the labelled HMI and PLC network ports, which face the Open vSwitch fabric. The green terminal blocks carry the PLC's 24 V digital inputs and relay outputs. Photographed on the testbed.

### 3.7.2 The two cells

The fabric is divided into two process cells and a gateway, each an OpenFlow switch under the one controller. Two cells rather than one are used so that the same pipeline is shown installed identically on more than a single switch, the multi-switch case the portability and scale argument of Section 3.6 rests on, and so that cross-cell and gateway-transit conduits exist to exercise rather than being asserted; the second cell is held at a lower criticality than the first. Cell-1, on switch `ovs1`, holds PLC1, which controls the critical tank, and its operator HMI. Cell-2, on switch `ovs2`, holds PLC2 and its HMI, reached through a network-address translation that presents their internal addresses `192.168.2.10` and `192.168.2.9` as `192.168.3.10` and `192.168.3.9` to the rest of the fabric; the translation lets the second cell reuse the Cell-1 addressing plan while remaining separately reachable. The gateway switch, `ovsgw`, carries the supervisory seams, the engineering and Factory IO host, the Snort mirror, the honeypot decoy, and the attacker vantage. The two cells and the gateway are joined by a patch link and a translated transit, so that all traffic to a protected asset crosses the CARS fabric and is subject to its policy. Beyond the fabric sits a modelled perimeter, an enterprise IT zone and a DMZ that reach the process network only through an enterprise and an OT firewall; that perimeter is the north-to-south path exercised by the two-pivot analysis of Chapter 4, and on the OT segment itself CARS is the enforcing layer. The fabric read directly from the switches with `ovs-vsctl show` is given in Figure 3.9: each bridge holds its OpenFlow connection to the controller, runs in secure fail mode, and presents the PLC and HMI on separate ports.

### 3.7.3 The hardware-in-the-loop process

Cell-1 runs a live tank-level process, provided by Factory IO on the Windows machine and controlled by PLC1 over S7 across the fabric, as shown in Figure 3.10. Factory IO supplies the level from a sensor and the state of the Start, Reset and Stop buttons as PLC inputs, and renders the fill valve, the discharge valve and the indicator lamps from the PLC outputs. The control runs in a cyclic block, OB30, which scales the raw sensor to a level of zero to one hundred, latches a running state from the operator buttons, and holds the level in a band by opening the fill valve below thirty and closing it above seventy against a constant discharge. The level that the PLC computes, held in a data block, is the process value that CARS protects; driving it out of its band, or blinding the PLC to it, is the physical harm the evaluation


Figure 3.9: The fabric read from the switches with `ovs-vsctl show`: Cell-1 (`ovs1`, top) and Cell-2 (`ovs2`, bottom). Each bridge is connected to the controller at `tcp:10.10.10.1:6653` with `fail_mode: secure`, and carries its PLC and HMI on separate ports; `ovs1` also holds the patch to the gateway and `ovs2` the internal `cell2gw`. Captured live on 3 August 2026 (OVS 3.3.4); the arrows are annotations.

measures. The running scene and the operator HMI are shown in Figure 3.11, and the Factory IO driver that binds the scene to PLC1 in Figure 3.12.

## 3.8 Deployment

CARS is deployed across the machines of the testbed, in the arrangement shown by the topology of Figure 3.7. The arrangement keeps the two planes apart: the controller, which decides, is separate from the switches, which enforce, and the sensing sits with the fabric so that detection stays close to the traffic while the decision stays central.

The controller runs as an os-ken application, `cars_engine.py`, launched with link observation enabled.

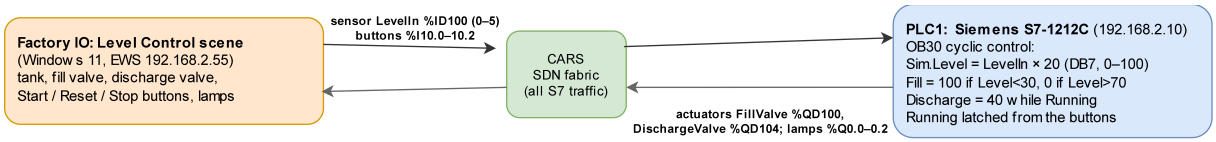


Figure 3.10: The hardware-in-the-loop. Factory IO supplies the level and button inputs and renders the valve and lamp outputs; PLC1 runs the control in OB30; every exchange crosses the CARS fabric. Addresses and mappings are from the deployed project.

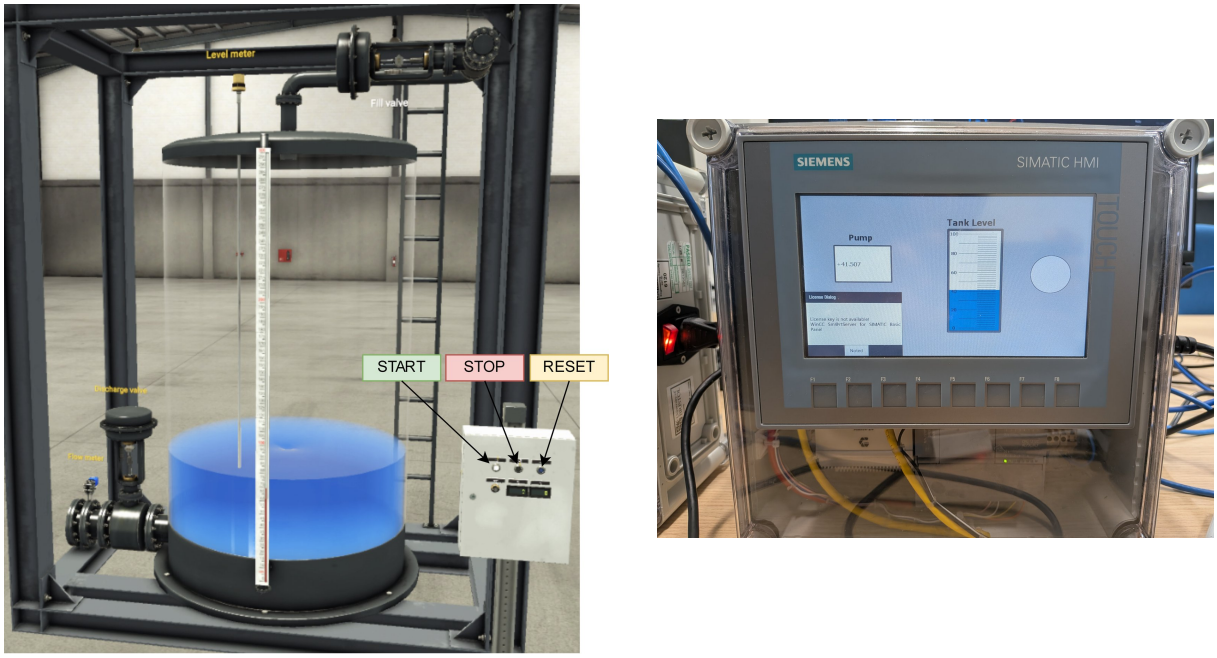


Figure 3.11: The live process. Left: the Factory IO tank scene, with the level meter, the fill and discharge valves, the flow meter, and the Start, Stop and Reset station. Right: the operator HMI reading the tank level, here about 40 per cent, and the pump value. The scene runs on the Windows machine and exchanges its inputs and outputs with PLC1 over S7 across the fabric.

It holds the whole decision logic described in Section 3.4, speaks OpenFlow 1.3 to the switches on port `6653`, and exposes the control interface used for responses and for the guarded operations on port `8080`. Nothing else of CARS runs on this node. It runs continuously and prints each decision as it is taken; a live extract is shown in Figure 3.14, and its status interface (Figure 3.13) reports the three switches connected and both guards enabled.

The fabric and the sensing run together on one node. The two Open vSwitch bridges, `ovs1` for Cell-1 and `ovsgw` for the gateway, carry the data plane. The Snort sensor reads a mirror of the gateway traffic and hands its findings to a small bridge, `cars-bridge`, which posts them to the controller at `/cars/respond`; this is the detection path of Section 3.5. The bridge de-duplicates, posting at most one event per source, conduit and operation every three seconds so a sustained attack cannot storm the controller with identical alerts, and it estimates each operation's arrival rate over that same three-second window for the volumetric overlay. Three supporting services run alongside: the flow-integrity checker, `cars-flowaudit`, which polls the tables every ten seconds against the trusted baseline; the remediation agent on `192.168.2.45`; and the decision dashboard. The operator, attacker and service endpoints are hosted in Linux network namespaces on this node, which is why a single machine can present the operator station, the Modbus device, the honeypot and the attacker vantage on distinct addresses.

Cell-2 runs on its own switch, `ovs2`, with the address-translation clone that lets it re-use the Cell-1 plan, as described in Section 3.7. The Windows machine runs TIA Portal and Factory IO; it is both the engineering workstation, at `192.168.2.55`, and the physical process of the hardware-in-the-loop. One boundary follows from this layout: the Snort mirror is on the gateway, so traffic that stays within Cell-2 on `ovs2` is enforced by the proactive policy but is not seen by the detector. The reactive layer therefore covers the paths that cross the gateway, which is where the supervisory and cross-cell conduits run.

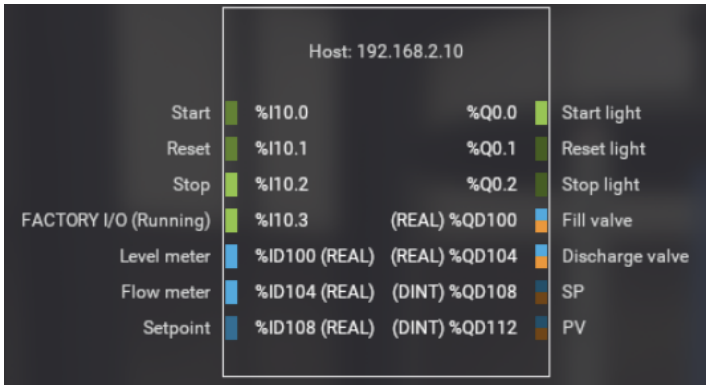


Figure 3.12: The Factory IO S7-1200 driver, bound to PLC1 at `192.168.2.10`. The inputs are the Start, Reset and Stop buttons at `%I10.0`–`%I10.2`, the running flag at `%I10.3`, and the level, flow and setpoint values at `%ID100`/`%ID104`/`%ID108`; the outputs are the lamps at `%Q0.0`–`%Q0.2`, the fill and discharge valves at `%QD100`/`%QD104`, and the setpoint and process value at `%QD108`/`%QD112`. Read from the deployed driver configuration; these match the OB30 tag table.

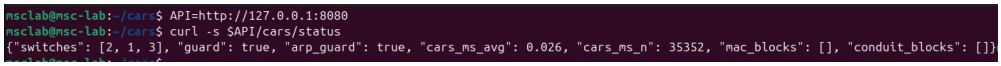


Figure 3.13: The controller status from `curl $API/cars/status`: the three switches connected, identity and ARP guards enabled, and the running mean decide time. Captured 3 August 2026.

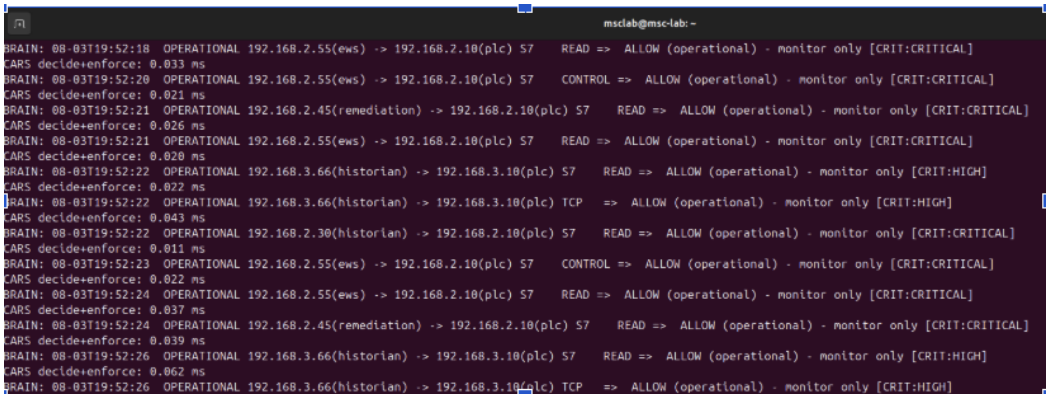


Figure 3.14: The deployed controller running live: each line is one decision, giving the source and destination, the protocol and operation, the verdict and response, the asset criticality, and the decide-and-enforce time. Captured from the controller console on 3 August 2026.

# Chapter 4

# Critical Evaluation

## 4.1 Method and metrics

The first question for a reactive defence in an operational network is not whether it can block an attack but whether it can be trusted not to block legitimate work. A false positive that cuts a control conduit is the deployment barrier. The decision layer is a deterministic function of the source role, the operation and the target criticality, so the property to establish is not a statistical accuracy but conformance: does that function restrain every unauthorised action and permit every legitimate one, across the decision space, with no gap. The evaluation checks this conformance over the whole space of source role, operation and target criticality on the testbed, and reports the false-positive rate as the headline figure. This chapter evaluates the deployed instance; that the same engine carries unchanged to another site is a property of the framework, established by the configuration-parity test of Section 3.6 rather than re-measured here.

Each case is a tuple of source, destination, operation and rate, carrying a ground-truth label assigned from security intent: legitimate, attack, grey (criticality-graded) or flood-exempt. The measured decision is the deployed engine's own verdict, obtained by posting the case to the controller's `/cars/respond` interface, which runs the exact production path: the first-match classification of (role, operation) to a tier, the criticality elevation of a SENSITIVE operation on a CRITICAL asset, and the graded response selection. The engine is not reimplemented for the test; the labelled set is driven through it. The pass runs with enforcement disarmed, so it measures pure decisions with no side effect on the live conduits, then re-arms.

The verdicts follow the intent. A legitimate case is a true negative if it is permitted, that is ALLOW, MONITOR, or REFUSE, the last being the safety loop that is mirrored and alerted but never cut, and a false positive if it is restricted. An attack case is a true positive if it is restricted, that is THROTTLE, DEFLECT, ISOLATE or BLOCK, and a false negative if it is permitted. Grey cases are shown to demonstrate grading rather than scored: they test how the response scales with criticality, not a binary legitimate-or-attack decision, so they carry no true or false label by construction. All three are nonetheless handled correctly, and including them would leave the result unchanged. The matrix was regenerated on the live system for this writeup and reproduces the stored baseline row for row; the decisions are corroborated at the packet level by the campaign of Section 4.4, so the verdicts are not decision-only artefacts. Because the decision is deterministic, a false positive or a false negative can arise only from a defect in the logic, an overlapping or shadowed rule, a mistaken elevation, or a connection-state error, so the pass is in substance a search for such defects rather than a measurement of predictive skill.

Several attacker identities recur across the evaluation and are named consistently. The unregistered on-segment attacker occupies the namespaces `192.168.2.66` and `192.168.2.67` and drives most of the campaign; the compromised but allowlisted supervisory host `192.168.2.31` launches the insider false-data injection; and a dual-homed Kali virtual machine provides the two-pivot trace of Section 4.7, seen on the OT segment as `192.168.2.77` and, when routed through the modelled perimeter, collapsed by network-address translation onto the single gateway identity `192.168.2.1`. Each figure is captioned with the identity of the run it shows.

## 4.2 Decision conformance and false positives

Table 4.1 gives the full case set as measured. Of the twenty-four scored cases every one conforms to the deployed policy: thirteen true positives, eleven true negatives, no false positives and no false negatives.

The figure that matters for deployment is the false-positive rate, which is zero. Table 4.2 summarises this per class: all eleven legitimate cases were permitted and all thirteen attack cases restricted, so the false-positive and false-negative rates are both zero. The full case set, with the inputs and the verdict for each case, is reproduced in Appendix A.5. The twenty-four is not an arbitrary sample but the closure of the decision space: one case per reachable branch of the classification, elevation and response-selection path, so that every rule the engine can apply is exercised at least once and none is duplicated. A twenty-fifth case on the same branches would add no coverage. This is exhaustive coverage of the policy as deployed on this testbed, not a claim that the same rulebook generalises unchanged to another plant's assets and operations: it establishes correctness of the decision logic here, not deployment-grade confidence for industrial traffic at large.

Coverage establishes conformance on every branch; a larger run establishes that no defect appears at scale. Beyond the twenty-four cases of the decision-space closure above, a larger corpus of 2,078 labelled cases, each the (source, destination, operation, rate) tuple with the ground-truth label defined in Section 4.1, was driven through the same decision-only path with the harness `vast_conformance.py`. It is composed of fifty-one unregistered attacker sources against an asset of every tier, across read, write, control and diagnostic operations at normal and flood rates, giving 2,040 attack cases, together with the legitimate supervisory conduits at sub-flood rates and the flood-exempt high-rate input, the thirty-eight cases that form the false-positive test. The engine restrained all 2,040 attack cases and permitted all 38 legitimate and exempt cases: a false-negative rate of zero over the attacks and a false-positive rate of zero over the legitimate work. The two sides are reported separately, and it is honest to keep them apart, because the corpus is deliberately attack-heavy and they do not carry the same evidential weight. The attack label is that the source is unregistered, and the engine's default-deny keys on the same fact, so the 2,040 restrained cases are not an independent prediction but a coverage-and-robustness result: they confirm that the deny and elevation paths fire without exception across a wide span of sources, operations and rates, with no case slipping through a rule gap or a connection-state error.

The result that is not circular is the zero false positive over the authorised set, including the flood-exempt control stream at fifty writes per second that a naive rate limiter would cut; that is the operator-trust result, and it rests on these cases together with the live campaign of Section 4.4, where the legitimate loop and telemetry were permitted throughout. Because the decision is deterministic, either error can arise only from a defect in the logic, so this run is in substance a defect search at scale rather than a measurement of statistical skill; the per-class outcome is summarised in Appendix Figure A.13. It remains a single-platform, single-policy validation on the same testbed: it strengthens confidence in the decision logic itself, but it is not evidence that the result transfers to a different fabric, asset mix or rulebook, and it should not be read as production-proven.

Every legitimate operation is permitted, including the two that most tempt a blunt filter. The high-rate control stream from the engineering and Factory IO host is flood-exempt and passes at fifty writes per second, because that host drives the hardware-in-the-loop and its rate is expected. The operator loop between HMI and PLC is tier CRITICAL and carries the REFUSE marker, so it is watched but never enforced against. The 0% false-positive rate on this traffic is the operator-trust result: the defence can be armed on the live process without cutting the work that keeps it running.

The three grey cases show the criticality grading on one operation class. A SCADA write to the CRITICAL PLC1 is elevated from SENSITIVE to FORBIDDEN and quarantined; the same role's write to the LOW Modbus unit stays SENSITIVE and is throttled; a historian write to the HIGH operator panel is throttled. The response scales with the target's criticality, not the operation alone.

The set also isolates the operation-awareness a five-tuple filter cannot reach. The identical conduit from SCADA to the Modbus unit draws three different verdicts by operation and rate alone: a read is allowed, a write is throttled as a SENSITIVE actuation, and a read driven to fifty per second is throttled as a volumetric abuse. The decision is made on the industrial operation, not the addresses and ports.

The matrix is exhaustive over the meaningful decision space of the testbed rather than a random sample: it covers every role, operation and criticality combination that arises here. The same verdicts are then corroborated on live traffic by the campaign of Section 4.4, where the deployed engine judged every phase and logged on the order of a thousand decisions per run, and the legitimate loop and telemetry were permitted throughout with no false positive observed (Appendix A.6). The zero-error result is therefore a statement about this system's decision logic, established over the decision space and confirmed on live traffic, not a statistical guarantee for an arbitrary production network; that scope is stated in Section 4.12.

To probe further, the engine was driven across its entire decision domain, every source, asset, operation and rate combination on the testbed, 868 cases in all. It returned a defined verdict for each, and on inspection every verdict is consistent with the deployed policy: the unregistered attacker is isolated on

| Src→Dst | Operation | Tier | Response | V | Case |
|---|---|---|---|---|---|
| *Legitimate benign (false-positive test)* | | | | | |
| 9→10 | READ | CRITICAL | REFUSE | TN | HMI→PLC1 read (loop) |
| 9→10 | WRITE | CRITICAL | REFUSE | TN | HMI→PLC1 setpoint (loop) |
| 10→9 | READ | CRITICAL | REFUSE | TN | PLC1→HMI reply (loop) |
| 55→10 | READ | OPERATIONAL | ALLOW | TN | EWS/Factory IO read (HIL) |
| 55→10 | CONTROL | OPERATIONAL | ALLOW | TN | EWS/Factory IO write (HIL) |
| 45→10 | CONTROL | OPERATIONAL | ALLOW | TN | Remediation restore |
| 30→10 | READ | OPERATIONAL | ALLOW | TN | Historian poll PLC1 |
| 30→20 | READ | OPERATIONAL | ALLOW | TN | Historian poll Modbus |
| 31→20 | READ | OPERATIONAL | ALLOW | TN | SCADA Modbus poll |
| 31→10 | READ | OPERATIONAL | ALLOW | TN | SCADA read PLC1 |
| *Grey, criticality-graded (shown, not scored)* | | | | | |
| 31→10 | WRITE | FORBIDDEN | ISOLATE | grey | SCADA write CRITICAL (elevated) |
| 31→20 | WRITE | SENSITIVE | THROTTLE | grey | SCADA write LOW Modbus |
| 30→9 | WRITE | SENSITIVE | THROTTLE | grey | Historian write HIGH HMI1 |
| *Attacks (true-positive / false-negative test)* | | | | | |
| 77→10 | TCP | FORBIDDEN | ISOLATE | TP | Kali outsider TCP |
| 77→10 | READ | FORBIDDEN | ISOLATE | TP | Kali outsider read |
| 66→20 | WRITE | FORBIDDEN | BLOCK | TP | Unknown write Modbus |
| 31→10 | CONTROL | FORBIDDEN | ISOLATE | TP | Compromised SCADA actuates |
| 31→10 | PROGRAM | FORBIDDEN | ISOLATE | TP | SCADA program download |
| 55→10 | PROGRAM | FORBIDDEN | ISOLATE | TP | EWS program download (no window) |
| 55→10 | DIAG | FORBIDDEN | ISOLATE | TP | EWS diagnostics |
| 55→10 | ILLEGAL | FORBIDDEN | ISOLATE | TP | Malformed S7 |
| 1→10 | READ | FORBIDDEN | ISOLATE | TP | Gateway reaches PLC (no conduit) |
| 30→10 | CONTROL | FORBIDDEN | ISOLATE | TP | Historian issues CONTROL |
| 66→10 | READ | FORBIDDEN | ISOLATE | TP | Unknown read PLC1 |
| *Flood (volumetric overlay)* | | | | | |
| 30→10 | READ (50) | OPERATIONAL | BLOCK | TP | Historian read flood |
| 55→10 | CONTROL (50) | OPERATIONAL | ALLOW | TN | EWS high-rate (flood-exempt) |
| 31→20 | READ (50) | OPERATIONAL | THROTTLE | TP | SCADA read flood on Modbus |

Table 4.1: The decision matrix, measured live from the deployed engine via `/cars/respond`. Hosts are `192.168.2.x` (last octet shown). Twenty-four scored cases: 13 TP, 11 TN, 0 FP, 0 FN; three grey cases show grading. Regenerated for this writeup; identical to the 3 August baseline. Full per-decision controller log in Appendix A.6.

| | Permitted | Restricted |
|---|---|---|
| Actually legitimate (11) | 11 (TN) | 0 (FP) |
| Actually attack (13) | 0 (FN) | 13 (TP) |

Table 4.2: Per-class conformance summary: false-positive rate 0/11, false-negative rate 0/13. Every scored case conforms to the deployed policy.

every operation against every asset, high-rate operations are restricted as floods, non-allowlisted conduits are default-denied, and the supervisory roles keep their reads. Building an independent oracle for a policy this nuanced is itself error-prone, so this exhaustive pass is a totality and consistency check rather than a second accuracy figure. The one behaviour it highlights is that the trusted remediation agent may issue even dangerous operations to the PLC unblocked, the trusted-role exposure of Section 4.12.

## 4.3 Criticality-graded response

The criticality model earns its place only if the target's importance changes the outcome. The sweep, rerun with `cars_criticality_proof.sh`, holds the actor and the operation fixed and varies only the target's tier, in three parts: the bounded elevation of the decision, the proportionality of the response, and the invariants that criticality must never break. The testbed instantiates the general model of Section 3.4 with the six assets of Table 4.3, read live from the controller (Figure 4.1); these span the four tiers the sweep drives.

Elevation is bounded. A SENSITIVE operation on a CRITICAL asset is raised to FORBIDDEN, as the grey cases of Table 4.1 show for a SCADA write to PLC1, whereas a read is never elevated: the same SCADA reading the same CRITICAL PLC1 stays OPERATIONAL and is allowed. The elevation is specific to the critical tier: the same actuating write against the HIGH PLC2, weight two, is not raised to FORBIDDEN but stays SENSITIVE and is rate-limited, so the two assets are handled differently

| Asset | Role | Tier | Weight $w$ |
|---|---|---|---|
| PLC1 | plc | CRITICAL | 3 |
| PLC2 | plc | HIGH | 2 |
| HMI1 | hmi | HIGH | 2 |
| HMI2 | hmi | MEDIUM | 1 |
| Historian | historian | MEDIUM | 1 |
| Modbus unit | plc | LOW | 0 |

Table 4.3: The testbed instantiation of the criticality model (Table 3.1): the six protected assets and the tier each is assigned by the consequence of its compromise. This mapping is an illustrative instantiation, chosen so that the testbed populates all four tiers; it is site-specific configuration rather than a prescribed or safety-certified assignment, and it is the graded response the tiers drive, not this particular allocation, that the framework contributes. The concrete address bindings are read live from the `/cars/criticality` interface (Figure 4.1) and listed in full in Appendix A.

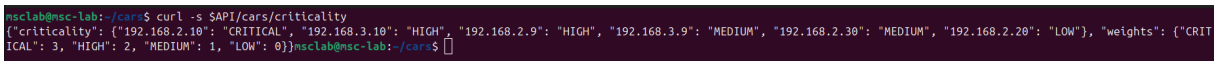


Figure 4.1: The criticality read live from the controller with `curl $API/cars/criticality`, backing Table 4.3: the six tiered assets and the weight of each tier. Captured on 3 August 2026.

by design, which the four-tier sweep and the two-hundred-and-forty-decision run below exercise directly. The model suspends this elevation within an authorised maintenance window (Section 3.4), so scheduled engineering is not blocked by the rule that protects the asset at other times.

Proportionality is measured on the response side. The same forbidden control, from the same harmless attacker, draws a reactive rule whose length scales with the target's weight: 75 seconds on the CRITICAL PLC1, 60 seconds on the HIGH PLC2, 45 seconds on the MEDIUM historian and 30 seconds on the LOW Modbus unit, each self-healing when it expires. The duration follows $30 + 15w$ seconds for tier weight $w$, the timeout ladder of Figure 3.6, measured live at all four tiers. The scaled quantity is the timeout; it is applied to whichever reactive rule the escalation state selects, a source ISOLATE once an attacker persists, or a single-conduit BLOCK on a first offence, as the MEDIUM case shows. One honest qualification bounds the LOW result: that tier is populated in the testbed only by the simulation Modbus unit, and at weight zero its response is by construction the base case, so the sweep measures the floor of the ladder at LOW rather than a distinct low-consequence device. The discriminating power of the criticality model is therefore carried by the separation of the MEDIUM, HIGH and CRITICAL tiers above that floor, which this run exercises directly; the LOW point confirms that the base holds, not that a low-value production asset was independently graded.

A larger run confirms the grading holds across operations and in combination, not one case at a time. Two hundred and forty decisions were driven through the deployed engine: an unregistered attacker and a compromised but trusted supervisory host, against all four tiers, across read, write, control and diagnostic operations, at normal and flood rates, and in both alternating and simultaneous bursts. The timeout tracks the target's tier throughout, 75, 60, 45 and 30 seconds, and the engine returned a correct verdict for every tier even when all four were attacked at once. The source is judged by what it does rather than by a blanket rule: the unregistered attacker is isolated on every operation, whereas the trusted host keeps its reads and is cut only on the operations its role must not perform. A flood withdraws the leniency a normal rate earns, so a permitted read on the critical asset is escalated from allowed to cut, and a source that keeps offending is treated more harshly as its offence count rises. The grading is stable across the full range of operation, rate, source and concurrency, not only the single-operation sweep of Figure 3.6; the response mix per operation and source is plotted in Appendix Figure A.14.

The invariants hold throughout. The operator control loop is never enforced against, even on the CRITICAL PLC1, where an HMI-to-PLC control returns REFUSE and is only mirrored and alerted, the safety cap of the model. Trusted monitoring is always allowed, even on the CRITICAL asset, where a SCADA read returns OPERATIONAL and passes. Criticality raises the response to a threat but never overrides the two rules that keep the process controllable.


Figure 4.2: Process devastation, disarmed. Left: the Factory IO tank driven full and over the top. Right: the operator panel at the same moment, the level cresting past the full mark at `+104.326` with the high-level alarm raised. The attacker holds the process out of its safe band and the operator cannot recover it.

## 4.4 Wire-level campaign: armed versus unprotected

The decision conformance of Section 4.2 is necessary but not sufficient; a decision matters only when it is enforced on the wire against a real attacker. This campaign exercises the defence at the packet level, comparing the armed system against the unprotected process. Unprotected has two honest meanings, kept separate: *disarmed*, where reactive enforcement is switched off but CARS still detects and decides and the proactive structure stands, used for the insider and process vectors; and a true *bypass* around the governed pipeline, used for the outsider vectors, since the proactive default-deny is always in force and merely disarming does not expose an unregistered host. Each vector is captured at three wire points, the PLC S7 port, the OpenFlow control channel and the Snort mirror. The legitimate Factory IO loop and the historian telemetry ran throughout and were permitted in every state, so the campaign carried zero false positives from end to end. The path a single attack packet takes in each state, armed and disarmed, was set out in the design (Figure 3.4).

### 4.4.1 Process devastation and the deception

The centrepiece is a false-data injection on the live tank, the attack that most directly threatens a physical process. A compromised supervisory host, `192.168.2.31`, writes the level sensor and the discharge valve in the process image over S7 at high rate, driving the tank past its safe band. Each write is a CONTROL operation to the CRITICAL PLC1. The disarmed and armed cases are compared on the same attacker, the same operation and the same target.

Disarmed, the injection runs unimpeded and the process is devastated. The tank is driven clean over the top: the Factory IO vessel fills completely and the operator panel's high-level alarm fires, the level cresting past the full mark at about 104% (Figure 4.2). A sensor-spoof variant of the same attack instead pins the reported level near zero, so the operator sees a safe, empty tank while the real one fills, the deception at the heart of a process attack (Appendix A.8). Either way, while the attacker holds the actuators the operator cannot regain control.

Armed, the same attack is severed before it can move the process. The injection's writes are CONTROL operations to the CRITICAL PLC1; CARS classifies them FORBIDDEN and isolates the source.

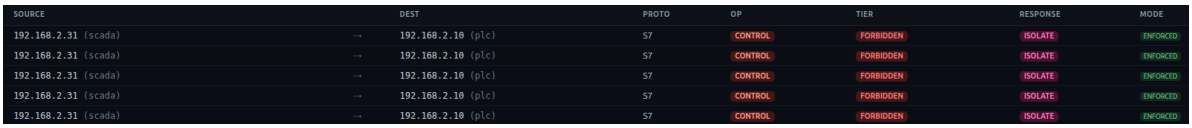


Figure 4.3: The same armed isolation seen at the decision layer on the CARS dashboard: the compromised supervisory host `192.168.2.31 (scada)` issuing S7 CONTROL to the CRITICAL PLC1 is classed FORBIDDEN and met with an ENFORCED ISOLATE. This is the cross-layer counterpart to the datapath rule of Figure 4.4: the same event as a policy decision rather than an installed flow.

```
== ovs1 ==
 cookie=0xca, duration=6.505s, table=1, n_packets=1, n_bytes=93, hard_timeout=75,
send_flow_rem priority=110,ip,nw_src=192.168.2.31 actions=drop
== ovsgw ==
 cookie=0xca, duration=6.526s, table=1, n_packets=11, n_bytes=906, hard_timeout=75
, send_flow_rem priority=110,ip,nw_src=192.168.2.31 actions=drop
```

Figure 4.4: Armed enforcement at the datapath. The reactive source-quarantine rule installed on `ovs1` and `ovsgw` in response to the injection: `cookie=0xca`, `priority=110`, `nw_src=192.168.2.31`, `actions=drop`, `hard_timeout=75`. The non-zero packet counters are the attacker's own traffic being dropped; the rule self-expires after 75 s.

The attacker's client cannot land a write that moves the process. In this run its S7 session is cut before any write commits, none landing against the 973 disarmed, and a second attempt cannot even open a connection, its traffic already dropped. The wire-level proof below (Figure 4.6) is exact about the boundary on a conduit that is already open: at most a single first-packet frame reaches the PLC port before the isolate installs, and the tank absorbs it without leaving its band, so no write reaches the process in a state that can move it. The enforcement is a real installed rule, not a log entry. On every switch the controller installs `cookie=0xca`, `table=1`, `priority=110`, `nw_src=192.168.2.31`, `actions=drop` with a 75-second self-healing timeout, the CRITICAL duration (Figure 4.4); the controller's own decision log shows the same event across the layer as a `FORBIDDEN ⇒ ISOLATE (ENFORCED)` verdict on `192.168.2.31` (Figure 4.3), and the rule self-expires after its 75-second window. With no write reaching the PLC the tank never leaves its band and the process stays under the PLC's control. This is block-and-maintain: the attack is severed at the datapath before it can move the process. The dashboard itself, how it discovers the topology from the controller and renders each live decision, is described in Appendix A.10.

The contrast is stark on both the network and the process (Figure 4.5). The identical FDI injection, sourced from the compromised supervisory host `192.168.2.31` and run over a matched forty-second window, landed 973 writes at the PLC disarmed against none armed; disarmed the tank crested past the full mark and overflowed, the process consequence of an unblocked injection shown in Figure 4.2, and armed it stayed within its safe band. The armed zero is not a near miss but a clean severance: the source is classified FORBIDDEN and isolated before its S7 session commits a single write.

### 4.4.2 The rest of the repertoire

The remaining vectors ran in the same armed session, captured at the three wire points. Against the control plane, an unauthenticated request to disarm the defence, `POST /cars/defense {on:false}`, was refused with `HTTP 401` and logged as `DENIED (bad/missing token)`; CARS stayed armed, so a compromised control-plane host cannot silently switch the defence off. A flow tamper that injected a bogus rule and deleted a real conduit was caught by the flow-integrity checker within one ten-second poll, its verdict moving to `ok:0, missing:1, extra:1`, and cleared back to `ok:1` on restore without re-baselining, which shows the restoration was byte-exact. An out-of-state attack, forged ACKs from `192.168.2.66` to the PLC, did not reach it: the stateful table admitted none. And the operation-aware case, a compromised SCADA writing a bogus level to the PLC, was isolated on the write while the remediation agent restored the value, the level holding near 50 and the loop still running, the same block-and-maintain as the devastation phase. Through all of it the legitimate Factory IO loop and the historian telemetry were permitted, so the campaign carried no false positives from end to end.

The two external vectors close the campaign, driven from the Kali attacker `192.168.2.77`, the unregistered outsider of the threat model. Reconnaissance learned nothing under CARS: every scanned port on

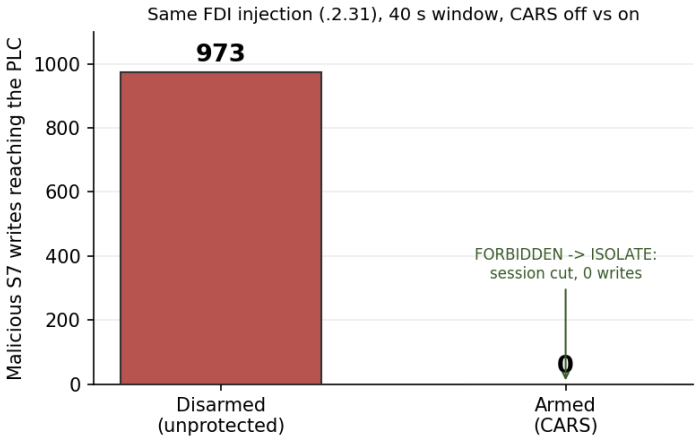


Figure 4.5: The same FDI injection from the compromised supervisory host `192.168.2.31`, run over a matched forty-second window with CARS disarmed then armed: malicious S7 writes reaching the PLC, 973 disarmed against 0 armed. Armed, the source is classified FORBIDDEN and isolated before its S7 session commits a write. Measured from the injection runs and the controller decision log.

| Vector | Unprotected | Armed (CARS) |
|---|---|---|
| Reconnaissance | PLC ports open; model and firmware fingerprinted | all ports `filtered`; no S7 identity leak; scanner isolated |
| Identity spoofing | forged frames accepted as the trusted seam | 5/5 spoofed ARP dropped at GUARD ingress; real seams unaffected |
| State manipulation | out-of-state frames serviced by the PLC | none admitted by the stateful table |
| Control-plane disarm | defence silently switched off | refused (`HTTP 401`); stays armed |
| Flow tamper | injected and deleted rules persist | flagged by the flow-integrity checker within one poll; self-clears on restore |
| Operation-aware injection | bogus writes actuate the process | isolated on the write; remediation restores (block-and-maintain) |
| Process devastation | tank overflows to ∼104%, alarm raised | isolated on the first write; datapath drop installed; tank held |

Table 4.4: The campaign, armed against unprotected. Every *armed* outcome was captured live on 5 August, the external vectors from the Kali attacker `192.168.2.77` and the insider vectors from the compromised supervisory host `192.168.2.31`, corroborated by the controller decision log, the nmap and GUARD dumps, and the installed flows (Appendix A.8); the process-devastation contrast was re-captured fresh on the live tank (Section 4.4.1). The two *unprotected* reconnaissance and spoofing entries are the known flat-network baseline, an unenforced switch trivially serves a port scan and accepts a forged frame, and are given as reference rather than as a fresh measurement of our system. The legitimate Factory IO loop and historian telemetry were permitted throughout, so the campaign held zero false positives.

the PLC, including the S7 port 102, returned `filtered`, the `s7-info` probe drew no model or firmware, and the scan was reactively isolated. The controller logged the scan as `ISOLATE source 192.168.2.77 75s ...ENFORCED [CRIT:CRITICAL]`, and the drop rule appeared on the switches as `cookie=0xca, priority=110, nw_src=192.168.2.77`. Identity spoofing was dropped at the door: forging ARP as the trusted seam `192.168.2.31`, every forged frame was dropped at GUARD ingress and logged as `REFUSED (identity-spoof) - 192.168.2.31 impersonation dropped at GUARD ingress (dpid3)`, while the real seam kept flowing (Appendix A.8). The whole campaign is summarised in Table 4.4.

## 4.5 Cross-layer validation

A decision log can be doubted; independent agreement across every layer is harder to dismiss. A deep end-to-end trial drove the full response spectrum through the real autonomous chain and recorded, for each scenario, five independent layers: the packet on the Snort mirror, the Snort alert, the controller's decision, the installed OpenFlow rule, and the outcome at the client. Table 4.5 gives the result; at every rung the layers tell the same story.

| Response (trigger) | Snort | CARS tier | OVS enforcement | Outcome |
|---|---|---|---|---|
| REFUSE (HMI loop) | n/a | CRITICAL | none installed | loop runs |
| ALLOW (READ FC3) | +283 | OPERATIONAL | none | read returns `[101..104]` |
| THROTTLE (WRITE FC6) | +285 | SENSITIVE | meter, p100 | write rate-limited |
| BLOCK (attacker WRITE) | +420 | FORBIDDEN | drop, p100 | connect fails |
| ISOLATE (persistence) | alert | FORBIDDEN | `0xca` src-drop, p110 | source cut |
| DEFLECT (policy) | alert | FORBIDDEN | `0xca` redirect, p105 | sent to honeypot |
| default-deny (bypass) | none | silent | A2 pre-drop | connect fails, no window |

Table 4.5: Cross-layer proof from the deep end-to-end trial: each response, with the Snort alert delta (the count of new lines appended to `/var/log/snort/alert` at that step, so a non-zero value confirms detection fired and `alert` marks a fired alert not separately counted), the controller tier, the installed OpenFlow rule, and the client outcome. The last row is the proactive default-deny with the detection bridge off, showing a drop with no Snort or controller involvement and therefore no reaction window.

| S7 Write-Var frames (`.31`→`.10`) | at Snort mirror | reached PLC |
|---|---|---|
| Armed | 8 | 1 |
| Disarmed | 8 | 7 |

Table 4.6: The drop shown on the wire. Attacker S7 Write-Var frames counted from the pcaps at the two capture points. Armed, eight are seen by the detector but seven are dropped in the fabric and only one leaks through in the reaction window; disarmed, seven of eight reach the PLC.

The DEFLECT row was probed further to see whether the deception actually engages. Forcing DEFLECT on the attacker's conduit installs the redirect, a priority-105 rule rewriting the destination to the honeypot, and a fresh capture at the decoy confirmed the diversion on the wire: the attacker's packets to the CRITICAL PLC arrived instead at the honeypot `192.168.3.99`, which began to answer, while the real PLC saw nothing. The full interactive round-trip, the decoy sustaining a two-way session so the attacker receives spoofed replies, did not complete in testing, because the minimal decoy could not resolve the attacker's address across the deception boundary; a production interactive honeypot is future work. The value demonstrated is the one that matters for protection: DEFLECT moves the attacker off the real asset and onto the decoy (Appendix A.11).

The sharpest single proof is at the packet level. In the S7 write attack on PLC1, the attacker's Write-Var frames were counted at two capture points, the Snort mirror (what the detector sees) and the PLC's own port (what actually arrives). Armed, the mirror saw eight write attempts but only one reached the PLC, the single reaction-window leak, and the other seven were dropped in the fabric; disarmed, seven of the eight reached the PLC (Table 4.6). The drop is shown on the wire, not inferred from a log. The size of this leak depends on where the reaction window falls against the S7 handshake: in the matched forty-second injection of Section 4.4.1 the isolate landed during session setup, so no write committed at all, whereas here it fell one write later. In both the leak is at most a single write, and in neither did the process move.

The same two captures make the drop visible frame by frame (Figure 4.6). At the mirror the attacker's writes appear at frame 1348 (`t+6.380 s`) and 1415 (`t+6.691 s`); at the PLC port only the first, frame 1374 (`t+6.380 s`), arrives. The isolate installed in the intervening third of a second, so every write from 6.691 s on is seen by the detector but never reaches the PLC. The operation CARS acts on is read straight from the frame (Figure 4.7): a Write Var to the output area, an actuation.

### 4.5.1 Operation-awareness at the wire

CARS decides on the industrial operation recovered from the packet, not the five-tuple. Firing the operation spectrum from one trusted operator (`192.168.2.31`) at the Modbus unit, the same conduit draws a different verdict per function code (Table 4.7): a read passes, a write is throttled, and the dangerous operations are forbidden. The wire confirms the discriminator directly, the identical conduit carried an FC 03 (read) and an FC 06 (write), the function code sitting at offset seven of the Modbus payload, and CARS allowed the first and throttled the second on that one byte.

The detection column records an honest limitation, and it is the one qualification on the operation-awareness claim. The classification is the policy, and it is correct for every operation. The sensor that recovers the operation from the wire, however, is not equally reliable across all of them. The common operations (read, write, control) are recovered reliably, and a sustained dangerous operation is caught and

s7comm && ip.src == 192.168.2.31

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 449 | 2.079622 | 192.168.2.31 | 192.168.2.10 | S7COMM | 79 | ROSCTR:[Job ] Function:[Setup communication] |
| 454 | 2.135668 | 192.168.2.31 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 1342 | 6.323773 | 192.168.2.31 | 192.168.2.10 | S7COMM | 79 | ROSCTR:[Job ] Function:[Setup communication] |
| 1348 | 6.380042 | 192.168.2.31 | 192.168.2.10 | S7COMM | 90 | ROSCTR:[Job ] Function:[Write Var] |
| 1415 | 6.691245 | 192.168.2.31 | 192.168.2.10 | S7COMM | 90 | ROSCTR:[Job ] Function:[Write Var] |

s7comm && ip.src == 192.168.2.31

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 454 | 2.079629 | 192.168.2.31 | 192.168.2.10 | S7COMM | 79 | ROSCTR:[Job ] Function:[Setup communication] |
| 459 | 2.135670 | 192.168.2.31 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 1368 | 6.323780 | 192.168.2.31 | 192.168.2.10 | S7COMM | 79 | ROSCTR:[Job ] Function:[Setup communication] |
| 1374 | 6.380041 | 192.168.2.31 | 192.168.2.10 | S7COMM | 90 | ROSCTR:[Job ] Function:[Write Var] |

Figure 4.6: The drop on the wire, armed, the S7 conduit filtered to the attacker (`s7comm && ip.src==192.168.2.31`). Top, at the Snort mirror: the write frames are seen (1348 at `6.380 s`, 1415 at `6.691 s`). Bottom, at the PLC port: only the first write (1374 at `6.380 s`) arrives; every later write is dropped in the fabric.

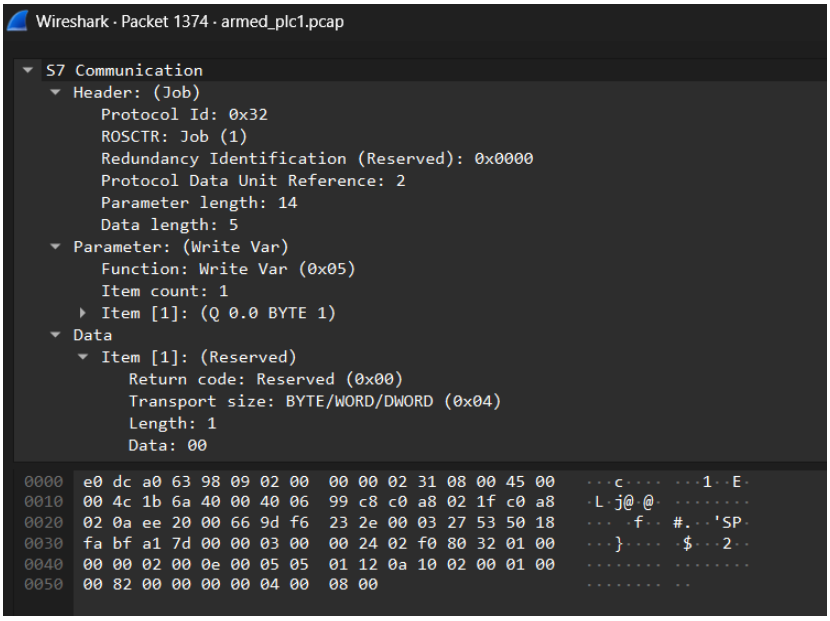


Figure 4.7: The malicious operation on the wire (frame 1348). CARS recovers the operation from the S7 parameter, `Function: Write Var (0x05)` to `Area: Outputs (Q) (0x82)`, an output actuation, and classifies it as CONTROL, forbidden on the CRITICAL PLC.

escalated (a repeated DIAG was blocked and then isolated). But a single crafted dangerous frame can evade the single-packet inspection, and the rarer Modbus function codes were detected only intermittently, the MEI program request slipped through a sustained run and the illegal-code family was caught only once. A dangerous operation the sensor fails to recognise is not classified, so it falls to the source's base tier and is permitted. Two distinct weaknesses sit behind this and they are treated differently. Intermittent recovery of the rarer function codes is a sensor-tuning limit that remains, carried into Section 4.12.

The sharper one, fragmentation, was closed. Because the deployed rules matched the operation byte at a fixed packet offset with no stream reassembly, an attacker on an already-permitted conduit could split an S7 write across two TCP segments so the function byte fell in the second, and the operation was never recovered. A fresh test from the allowlisted `192.168.2.31` confirmed it: a Write-Var fragmented at byte fourteen drew only `TCP => ALLOW`, no rule was installed and the write reached the PLC, whereas the identical write sent whole drew `FORBIDDEN => ISOLATE`. The fix has two parts, because reassembly alone was not enough: TCP stream reassembly was enabled for the S7 and Modbus ports, and the operation-class rules were re-anchored to the protocol header: the operation byte, previously matched at a fixed `offset:17`, is now matched PDU-relative as `distance:8; within:9` after the `32 01` S7 job header, so a reassembled PDU is recognised wherever it sits in the stream. Re-run against the same fragmented attack

| Operation | Modbus FC | CARS tier | Response | Detection |
|---|---|---|---|---|
| READ | `0x03` | OPERATIONAL | ALLOW | reliable |
| WRITE | `0x06` | SENSITIVE | THROTTLE | reliable |
| CONTROL | `0x05` | FORBIDDEN | BLOCK | reliable |
| DIAG | `0x08` | FORBIDDEN | BLOCK, then ISOLATE | sustained only |
| PROGRAM | `0x2B` | FORBIDDEN | BLOCK | unreliable |
| ILLEGAL | `0x64` | FORBIDDEN | BLOCK | intermittent |

Table 4.7: Operation-awareness on the same conduit. The tier and response are the policy, correct for every operation, as the decision matrix of Section 4.2 establishes by driving each operation through the engine directly. The detection column records how reliably the Snort sensor recovers that operation from the wire.

at two split points, the write is now recovered, classified FORBIDDEN and the source isolated, with the legitimate high-rate loop still permitted and no new false positive (Appendix A.9; the before-and-after capture and the full rule diff, Listing A.38, are in Appendix A.11). This raises the bar rather than settling the evasion question: overlapping-segment and timing evasions remain a general IDS problem, and single-frame recognition of the rarest codes is still imperfect. The annotated frame breakdowns at the three wire points, the S7 and Modbus function bytes read off the wire and the armed-versus-unprotected captures are collected in Appendix A.7.

## 4.6 Reaction window and latency

The decision itself is close to free. The controller reports a mean decide-and-enforce time of 0.026 ms over the campaign (Figure 3.13). That figure is the controller-internal cost alone: the time, measured on the controller's own clock, to classify the received alert against the first-match rulebook, elevate it by criticality, select the graded response and hand the resulting flow-modification to the switch. It excludes the detection and transport that deliver the alert to `/cars/respond` and the receipt of that request, and it excludes the switch's own installation of the rule once the message is sent. It is a few tens of microseconds across the clean runs, and rises to about 0.50 ms only under the concurrent load of the fourteen-hour night campaign (Section 4.10). So classifying an operation and selecting its response adds nothing measurable to the path; the cost that matters for a reactive defence lies elsewhere, in the reaction window: the interval between an attack reaching the wire and the reactive rule landing in the switch, during which a few packets can still reach the target.

The armed process-devastation run bounds the window at the process level: the attacker's client could not commit a single write, the isolate cutting its S7 session before the first write landed, so zero writes reached the PLC against the 973 it managed disarmed, and a second connection moments later could not open at all. The source was quarantined for 75 seconds and the tank never left its band. The window itself is measured directly, not inferred from this run, by the harness below.

A dedicated harness times the window directly, measuring attack-frame-to-enforcement on a single clock. Over one hundred autonomous trials the median mitigation was 7.6 ms, with a 95th percentile of 38.4 ms and a 99th of 67.9 ms; the fastest of the hundred was 5.5 ms and the slowest 72.9 ms, and every trial ended in an ISOLATE (Figure 4.9). The distribution is a tight mode of a few milliseconds with a small upper tail, not a broad spread, and its shape has a mechanical explanation rather than being random jitter. A fresh sixty-trial run reproduced the same shape, a median of 7.7 ms with ninety-two per cent of trials within ten milliseconds and a tail of five trials between twelve and thirty-five milliseconds, so the mode and the tail are both stable properties of the loop, not artefacts of one run.

The window is the detect-then-respond path taken in series: Snort recovering the operation from the mirrored frame, the bridge reading the alert, the post to the controller, the decision, and the flow-modification message reaching the switch, as broken down in Figure 4.8. Of these stages only the decision is separately instrumented, at a mean of 0.026 ms (Figure 3.13), which is under half a per cent of the window, so the seven-millisecond median is the detection and transport plumbing around a decision that is effectively free, not the decision itself.

Because that plumbing is near-constant when the controller thread is otherwise idle, the window clusters into a tight mode. The window is timed from the attacking frame on the mirror to the installed rule, with the install time recovered from the switch's own flow-duration counter rather than from a polling loop, so it is a true wire-to-enforcement interval on a single clock. The small upper tail is not a fixed polling boundary; the seven-millisecond median itself rules out a fifty-millisecond sampling cycle

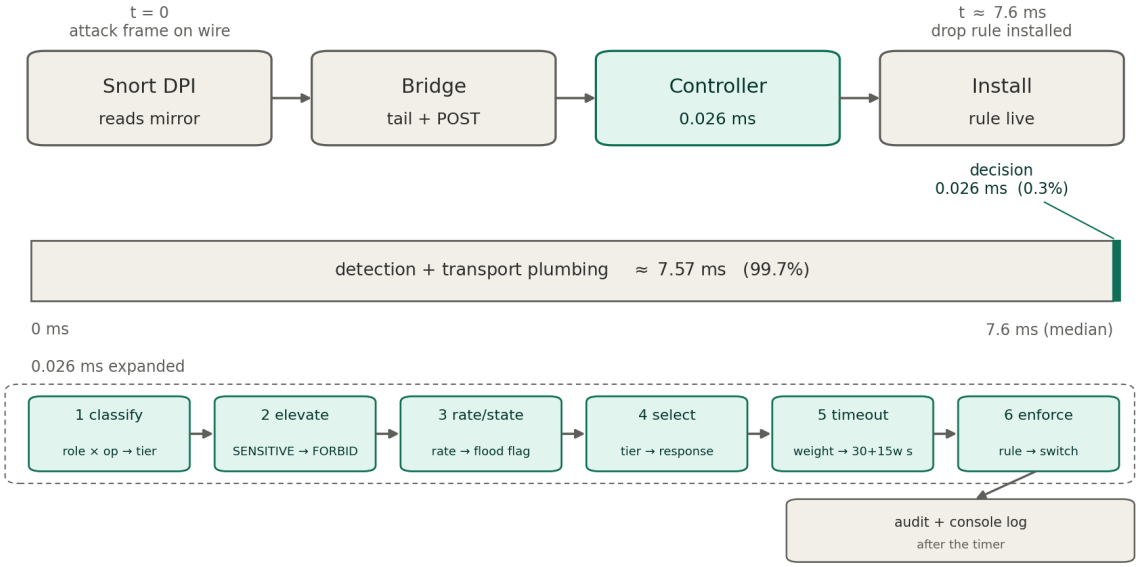


Figure 4.8: Anatomy of the reaction window. The wire-to-enforcement path runs in series: Snort inspecting the mirrored frame, the file-tailing bridge and its `POST`, the controller's decide-and-enforce, and the flow-modification reaching the switch. Only two points are separately measured, the decision (0.026 ms) and the total window (median 7.6 ms); the remaining ≈7.57 ms is aggregate detection-and-transport plumbing, not timed per stage. The decision is about 0.3 per cent of the window, expanded below.

sitting in the path. The slower trials are those whose alert arrived while the single-threaded bridge and controller were still clearing a heavier background alert load, so the new request queued behind that work, the head-of-line queueing that the load correlation below quantifies, with occasional scheduler and connection-setup jitter on top. It is this single-threaded serialisation, not any per-packet cost, that produces the tail: most trials meet an idle pipeline and clear in about seven milliseconds, while the few that coincide with other in-flight work wait their turn.

The window also tracks the load on the detection path. Across the hundred trials the reaction correlated with the volume of background alerts the bridge was processing, at a Pearson coefficient of 0.43, and the tail trials carried a heavier background than those in the mode. That background is not small: the hardware-in-the-loop input alone drives about thirty-two industrial operations a second on this testbed, so the reactive path already competes with a steady alert stream (the normal-traffic baseline is characterised in Appendix Figure A.12, and the window against background load in Appendix Figure A.17). A heavier multi-source event would lengthen the queue at the file-tailing, HTTP-posting bridge and widen the window further; that bridge is the acknowledged bottleneck, and its remedy, in-memory inter-process communication, is carried in Section 4.12 and the future work. An attempt to load the detector from a single source did not widen it far, because the volumetric overlay throttles a flood before it can build, which is a useful bound in its own right.

To characterise that load ceiling directly, the deployed decision path was measured two ways. Driving the engine's own `classify` and `select_response` code as a bench, the per-decision compute is a few microseconds, and the decision together with its audit-log write sustains on the order of sixteen to eighteen thousand alerts a second on a single core before the single-threaded stage saturates and latency grows. On the live testbed, sweeping a benign background decision load against a timed probe, the reaction window held at roughly eight to fifteen milliseconds up to about a thousand background operations a second, rising to about seventy milliseconds only under thirty-two simultaneous in-flight requests, which is head-of-line queueing at the single-threaded controller rather than throughput saturation, and not the serial path a rate-limited detection bridge actually presents. Two properties keep an attacker well below the ceiling: identical alerts are collapsed by the three-second cooldown to at most one event per source, conduit and operation, and a diverse flood is still bounded by the same throttle that governs the reactive path. The window therefore stays in the tens of milliseconds across any rate a real, de-duplicated attacker can offer, and lengthens only as the single-threaded controller nears saturation, which is exactly what the in-memory inter-process bridge of the future work is meant to raise.

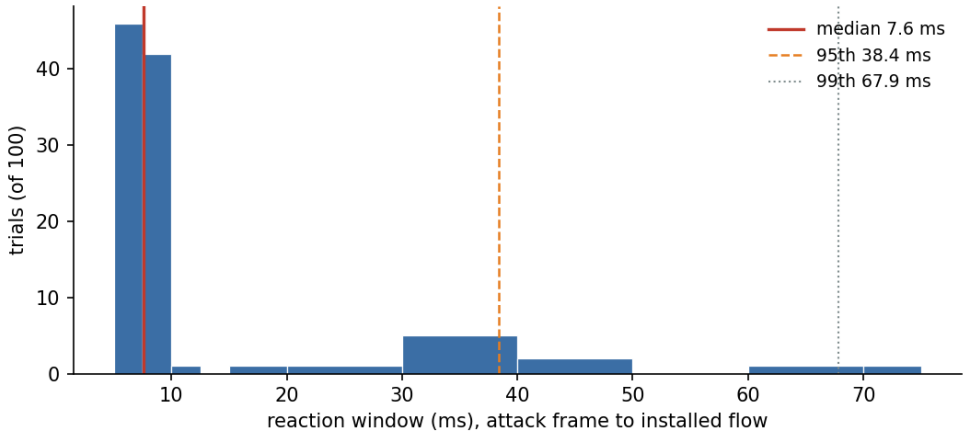


Figure 4.9: Reaction window over one hundred autonomous trials, attack frame to enforcement on a single clock. Median 7.6 ms, 95th percentile 38.4 ms, 99th 67.9 ms, maximum 72.9 ms; every trial ended in an ISOLATE. The window is the detect-then-respond plumbing (Snort alert to installed flow); the upper tail reflects queueing at the single-threaded bridge under a heavier background alert load, not a fixed poll boundary.

These figures are for a single injection, the case that matters for a false-data attack that needs only one write to land. A sustained flood behaves differently: fired continuously from the Kali attacker virtual machine rather than a co-located namespace, an ICMP flood on the CRITICAL PLC was isolated within a median of about one second, leaking of the order of forty frames first. Those frames are inert layer-three probes that never reach the S7 process, and the tank cycled without disturbance throughout, so a single injection is cut in milliseconds and a sustained real-attacker flood in about a second, with what leaks in either case unable to move the process.

This window is a property of any detect-then-respond defence, a finite detection, decision and installation latency, not a defect to be removed: each of those steps takes a small but non-zero time. The defence's value is that the window is short, the leaked packets are inert, and the enforcement then holds and self-heals. Pre-empting every first packet is the job of a proactive layer, which is exactly what the identity binding, allowlist and default-deny of the pipeline provide for the conduits they cover; the reactive graded response governs what those static layers pass on to inspection.

## 4.7 Anatomy of a defended attack: two pivots end to end

The preceding sections measured CARS one property at a time. This section binds them into a single causal account: one attacker machine, driven from two network positions, with every stage of the defence followed on one clock from the launched packet to the installed flow and the operator's log. The value of the trace is that nothing is asserted in isolation: each claim about a rule, a decision or an enforcement is the next link in a chain that begins with a real frame captured on the wire.

The attacker is a single Kali virtual machine, dual-homed by design (Figure 4.10). Its `eth2` interface (`10.0.40.66`) sits in the enterprise IT zone and reaches the process network only by routing north-to-south through the modelled perimeter: an Enterprise firewall, a DMZ, and an OT firewall that source-NATs every crossing onto the single gateway address `192.168.2.1`. Its `eth1` interface (`192.168.2.77`) sits directly on the OT segment as an unregistered host. The same tool, the same target and the same operation are launched from each; only the attacker's position changes. That isolates the one variable that matters for an identity-bound defence: whether the fabric can see who is really attacking.

Every packet that enters the fabric meets the same three-table pipeline of Section 3.3 (Figure 3.3). What matters for the anatomy is where each attack class terminates in it, set out in Table 4.8: a spoofed identity dies at Table 0; a non-conduit source is default-denied and, on any attempt against the PLC, reactively isolated; and an allowlisted conduit that turns malicious is delivered one packet, then caught by the deep-packet-inspection loop that recovers the S7 operation and installs the `0x00ca` isolate. The two pivots exercise the middle class; the false-data injection of Section 4.4 exercised the third.

**Pivot A, the IT attacker.** The Kali opens an S7 session to `192.168.2.10:102` over `eth2`. The

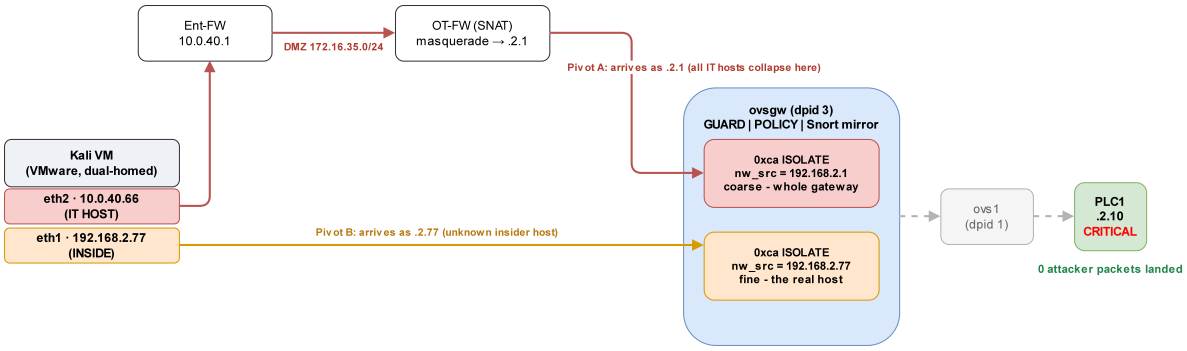


Figure 4.10: The two pivots from one Kali. `eth2` routes north and source-NATs to the gateway identity `192.168.2.1`; `eth1` sits on the OT segment as `192.168.2.77`. Both are quarantined at the gateway switch before a single packet reaches PLC1, but on different `nw_src` identities.

| Attack class | Terminates at | Outcome |
|---|---|---|
| Spoofed protected identity (protected address, wrong port) | Table 0, GUARD | dropped at ingress, never reaches policy |
| Unregistered / non-conduit source (both pivots) | Table 1, default-deny | the attempt is FORBIDDEN; the source is reactively ISOLATEd (cookie `0x00ca`) |
| Out-of-state packet (forged ACK) | Table 1, conntrack | refused, no established state |
| Dangerous operation on an allowlisted conduit (the insider FDI) | reactive DPI loop | the first packet is delivered, then Snort recovers `op=CONTROL`, the engine classifies FORBIDDEN and installs the ISOLATE |
| Authorised, benign operation | Table 2, SWITCH | forwarded to the asset |

Table 4.8: Where each attack class terminates in the pipeline of Figure 3.3, rather than re-drawing the pipeline itself. The two pivots are the unregistered-source class; the false-data injection of Section 4.4 is the allowlisted-conduit class.

frame is routed through the perimeter and arrives at the gateway switch source-NATed: the mirror records `192.168.2.1.33440 > 192.168.2.10.102` with the SYN flag, preceded by an ARP for the PLC, so every northern host has collapsed onto the one gateway identity. The controller resolves `192.168.2.1` to the `gateway` role, finds no allowlisted conduit to the CRITICAL PLC1, and classifies the attempt FORBIDDEN; because the destination is CRITICAL the response is an immediate source isolation. The audit line reads `FORBIDDEN 192.168.2.1(gateway) -> 192.168.2.10(plc) TCP => ISOLATE source 192.168.2.1 75s [CRIT:CRITICAL]`, and a matching flow appears on both `ovs1` and `ovsgw`: `cookie=0xca, priority=110, nw_src=192.168.2.1, actions=drop, hard_timeout=75`. The consequence for the attacker is a TCP connect timeout with zero writes landed; the gateway rule absorbed the SYN retransmits (`n_packets`=4 at `ovsgw`) and the PLC-side capture shows nothing reaching the process.

The containment is total but, as first captured, *coarse*: the quarantine was a source isolation on `192.168.2.1`, so it stopped this attacker and every other northern host alike, the NAT collapse leaving CARS no finer handle on identity. That coarse cut is itself a cost, a denial of service on the shared gateway conduit, and it prompted a refinement to the response selection: a shared, NAT-collapsed identity is now cut at the conduit rather than the source. Verified on the deployed engine, the same attempt from `192.168.2.1` now installs `cookie=0xca, priority=100, nw_src=192.168.2.1, nw_dst=192.168.2.10, actions=drop`, dropping only the PLC-bound flow while the gateway's other traffic is untouched by construction, whereas a true single host such as Pivot B still draws the finer source isolation (Appendix A.11). The residual limit is stated honestly: across a NAT, CARS cannot separate the rogue IT host from its legitimate neighbours on that one conduit, so full per-host attribution still needs the boundary firewall to carry the real source.

**Pivot B, the on-segment insider.** The identical S7 connection is launched from `eth1`, and now the

| Stage | Pivot A: IT attacker | Pivot B: insider |
|---|---|---|
| Launch | S7 connect from `eth2` (`10.0.40.66`) | S7 connect from `eth1` (`192.168.2.77`) |
| Travel path | Ent-FW → DMZ → OT-FW (SNAT) → ovsgw | OT segment → vmnet2 → ovsgw ofport 12 |
| Seen on the wire as | `192.168.2.1` (gateway, collapsed) | `192.168.2.77` (true host) |
| Rule matched | no conduit; default-deny | no conduit; default-deny |
| Decision | `gateway`→`plc` FORBIDDEN | `unknown`→`plc` FORBIDDEN |
| Response | ISOLATE 75 s [CRIT:CRITICAL] | ISOLATE 75 s [CRIT:CRITICAL] |
| Flow installed | `0xca nw_src=192.168.2.1` (ovs1+ovsgw) | `0xca nw_src=192.168.2.77` (ovs1+ovsgw) |
| Wire effect | connect timeout, 0 landed | connect timeout, 0 landed |
| Observability | audit + dashboard ISOLATE row | audit + dashboard ISOLATE row |

Table 4.9: The same attack, two positions, followed through nine stages. Enforcement is identical; the granularity of the quarantine is not.

mirror records the attacker's *true* address: `192.168.2.77.42712 > 192.168.2.10.102` with the SYN flag. The controller resolves `192.168.2.77` to `unknown` (it is unregistered), again finds no conduit, and issues the same verdict, `FORBIDDEN 192.168.2.77(unknown) -> 192.168.2.10(plc) TCP => ISOLATE source 192.168.2.77 75s [CRIT:CRITICAL]`, but the installed flow now reads `nw_src=192.168.2.77`. The attacker again gets a connect timeout and zero landed, but the quarantine is *fine-grained*: it names the actual host, leaving every other OT identity untouched. Both pivots were severed at the TCP-connect layer: because neither is an allowlisted conduit, the connection to the PLC's control port is itself forbidden, so the handshake never completes and no S7 payload is ever emitted.

Table 4.9 lays the two traces side by side. The enforcement is identical in kind, tier and duration; the single difference is the identity CARS can act on, and that difference is the argument for pushing enforcement onto the segment. A perimeter defence sees only `192.168.2.1` and must treat the whole enterprise as one suspect; CARS on the OT fabric names `192.168.2.77` and quarantines exactly it. Neither attacker moved the process, but only the on-segment case could be attributed.

These two pivots are stopped before they can speak S7 at all. The third termination class, an *allowlisted* conduit that turns malicious, is where the operation-aware machinery is exercised, and it was captured in the false-data injection of Section 4.4: the compromised supervisory host holds a legitimate conduit, so its first write reaches the PLC (first-packet reality), the DPI loop recovers the S7 operation as a CONTROL write, and only then does the controller classify it FORBIDDEN and isolate the source while the remediation agent restores the process. Read together, the three cases are the whole of CARS: identity and conduit gating for who may connect, deep inspection for what an authorised party then does, and one graded, self-healing response mechanism behind both.

Placing the timing on this story closes it (Figure 4.11). In each pivot the first SYN was seen once and the isolate was installed within the reaction window characterised in Section 4.6: a median of 7.6 ms, 67.9 ms at the 99th percentile. The attacker's own TCP stack did not retransmit until roughly one second later, by which time the drop was already in place: the defence closed roughly two orders of magnitude faster than the attacker could send its next packet, which is why the wire shows one SYN seen and the rest absorbed. The reaction window is real, but on this evidence it is far shorter than the interval over which even an automated attacker operates. Equally, the decision at the heart of each trace is not a one-off: the `classify()` verdict that both pivots received is the same rule evaluated, without error, across the scored matrix and the exhaustive decision domain of Section 4.2, so each trace is one observed instance of a rule whose correctness is quantified with a confidence interval rather than asserted from a single run.

## 4.8 Does the defence disturb the process?

A defence for a live plant is deployable only if arming it does not perturb the process it protects. Two measurements answer this: the process under the armed pipeline with no attack, and the process while the defence actually fires.

Under normal operation the armed and disarmed fabric are indistinguishable at the process. No reactive rule is installed while nothing is detected, so the datapath a legitimate packet crosses is identical with enforcement on or off, and any difference would have to come from the pipeline itself. Interleaving

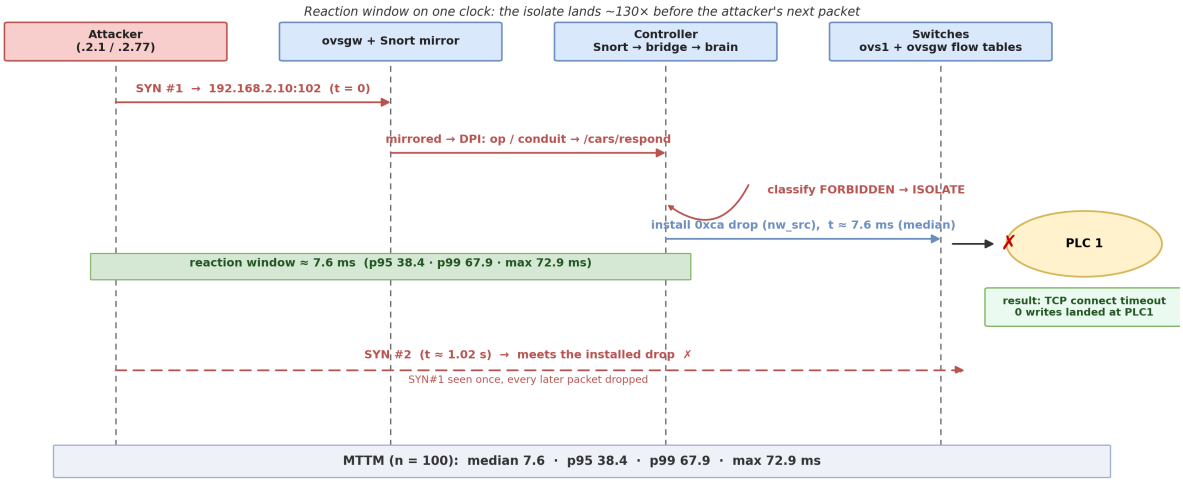


Figure 4.11: The reaction window placed against the attacker's packet cadence. The `0x00ca` isolate lands within the measured mean-time-to-mitigate (median 7.6 ms), while the attacker's TCP retransmits arrive ∼1 s apart, so the first SYN is seen once and every later packet meets the installed drop.

the two states in short blocks over two hours under matched load, so that any drift in the hardware-in-the-loop simulation falls on both equally, the tank held the same band and spread in each: a mean level of 51.5% disarmed against 50.7% armed, standard deviations of 12.8 and 13.2, and no excursion beyond the control band in either, across seven thousand per-second samples, with legitimate throughput unchanged (Appendix A.12; the two distributions are plotted in Appendix Figure A.16). Arming the defence has no measurable effect on the process. This result is established for the testbed and its hardware-in-the-loop tank, not for industrial processes in general: it shows that arming the pipeline did not perturb this live process, not that arming is harmless on every plant, since a faster or more tightly coupled loop could respond differently to the same enforcement machinery. A first, naive comparison of one armed hour against one disarmed hour was discarded when the simulation host stalled part way through and skewed the armed hour; the interleaved, load-matched method was adopted precisely to control for that, and it is why the result can be stated cleanly rather than as a difference confounded by drift.

When the defence does fire, it holds the process rather than harming it. A compromised but allowlisted supervisory host launched a false-data injection at the CRITICAL PLC under the armed defence: CARS classified the write FORBIDDEN and isolated the source after nine packets, the reactive drop carrying the criticality-scaled seventy-five-second timeout. For the whole of that block the tank stayed within its band with no excursion, the legitimate conduits kept flowing without interruption, and the restoration agent was never called, because the cut came fast enough that no physical anomaly developed. The leaked packets do not move the process because the boundary is a race the control loop wins here: the legitimate loop reasserts the actuator every cycle and reads it back, counting any readback-versus-intent mismatch as interference, so a flip that lands inside the reaction window is overwritten on the next cycle and the tank's inertia absorbs the few frames that precede the cut.

This is a window, not a guarantee: on a faster actuator, or a process without that inertia, a write leaked inside the window could actuate before the loop corrects it, which is precisely why the proactive layer, not the reactive window, is what pre-empts the first packet on the conduits it covers. The rule then self-healed: it expired at its timeout, the source returned to service with no operator action, and the flow table returned to its baseline, the install-to-withdrawal spanning seventy-six seconds. This is the block-and-maintain property measured at the process rather than the wire: the attacker is severed, the plant is held, and the network heals itself without a human in the loop. The nine-packet cut is specific to this attack and this load, not a general guarantee; how the same bound behaves under mixed background traffic, bursts and TCP retransmissions, or a heavier controller load was not measured here and is a question for a larger trial.

## 4.9 Process-layer defence in depth

The evaluation so far concerns the network: who may reach an asset, and with which operation. A separate question is what happens when the attacker is a genuinely trusted, allowlisted party acting within its permitted operations, the compromised engineering station or the corrupted agent behind classic ICS incidents. The network layer, by design, must not cut such a party on the critical path, so the natural worry is that a trusted insider drives the process to harm unchecked. Probing this on the live tank narrowed the concern before it justified a new layer.

The high-rate route is already covered. Driving the tank over its limit means spoofing the level sensor faster than the hardware-in-the-loop refreshes it, so the injection is necessarily high rate; when the trusted remediation identity `192.168.2.45` was used to run exactly that overflow, the volumetric overlay classified it a flood and cut the conduit after seventy-seven writes, the tank peaking in the mid-seventies rather than overflowing. Reading the PLC program confirmed there is no slow path to the same end: the fill thresholds are constants in the control block, the `Setpoint` input word is not referenced, and the level is recomputed from the sensor every scan, so a single low-rate write cannot re-aim the process. The rate overlay therefore closes the overflow that the network gating leaves to it.

Two residuals remain, both narrower and both real. The first is a low-rate authorised action: a trusted role issuing one legitimate command that is individually valid yet operationally harmful. The second is a blind spot inside the existing process agent, whose tamper test (Appendix A.9) checks only the low side, a reading below a floor or a sudden fall, so it restores a sensor spoofed downward but ignores a reading driven high. To cover these a small process guardian was added as a defence-in-depth layer. It is a standalone, read-only Python monitor, separate from the SDN controller and its engine, that reads the tank level from the remediation agent's status feed and alarms on a high-side envelope breach, an impossible rate of change, or a stall in the level while the process should be cycling. Detecting attacks by checking the physical process against its expected invariants, rather than inspecting the network, is an established direction in control-system security [33]; the guardian applies that idea as a narrow, alarm-only backstop to the response framework. It never writes the PLC and never touches the enforcement or the remediation agent, so it cannot disturb the running system; it reports, and the network layer keeps its hands off the safety loop.

Both residuals were then exercised live. A trusted `192.168.2.45` issuing a single authorised HMI stop was permitted by CARS, correctly for an authorised control, and the process halted with the tank frozen at 40.4%; the guardian raised a stall alarm twelve seconds later, catching a denial the network layer allows by design. Separately, with the reported level pinned high at 90% (the reported-full deception, the real tank draining beneath it), the existing agent stayed silent, its low-side test blind to the excursion, while the guardian raised a high-level and a rate alarm on the same reading. The guardian catches what the network layer must not act on and what the low-side agent misses, without changing either. These are narrow, live demonstrations that the guardian catches cases the lower layers miss, evidence that the layer has a role, not proof that it is sufficient against a sophisticated covert manipulation. The guardian alarm feed, and the flood block that cut the runaway trusted identity, are collected in Appendix A.11.

This does not claim to solve the insider problem. A manipulation that stays inside the physical envelope and moves the process slowly and plausibly evades an invariant monitor as it evades the agent; because the agent judges tampering by a rate of change the control law cannot produce, a drift small enough to pass that check is taken as healthy and advanced into the agent's own last-good baseline, so the low-side agent not only fails to catch a slow drift but adopts it and can no longer restore against it. This is why the read-only guardian, not the agent, is the backstop here, and even so the guardian is detection, an alarm to the operator, not prevention, because automating prevention on the safety loop is the risk this whole approach avoids. That the guardian only alarms is a deliberate boundary rather than a shortfall: it is the same operator-trust logic that runs through the whole design, since the reason the network layer must not cut the safety loop is the reason a process monitor must not act on it either, and a backstop an operator can leave armed is worth more than an automated action on the process that the operator would switch off. Moving from an alarm to a bounded, non-safety-loop action, for instance triggering the existing remediation agent's safe-value restore on a clear envelope breach, is a defensible next step and is carried as future work. What the layer buys is a clean division of responsibility: the network stops who and what should never connect, the rate overlay stops the high-rate abuse, and a process-invariant monitor makes the residual physical anomalies visible. The remaining stealthy-in-envelope case is carried to Section 4.12 and the conclusion as future work.

## 4.10 Long-run and stress-test campaign

The sections above measured CARS one property at a time, each from a short dedicated run. To test whether the same behaviour holds under sustained attack, and to probe two boundaries the earlier sections left open, whether noisy but legitimate traffic can force a false positive and what repeated isolation does to the PLC itself, the defence was left under continuous attack across four autonomous runs on the same testbed. This is a long-run and stress test on one bench, not a production trial: the process, the fabric and the two PLCs are the same single testbed throughout, so the runs extend the evidence in duration and repetition, not in network size.

The runs measured 440 attacks in total, and every one ended in enforcement (440 of 440). The reaction window kept its earlier shape: pooled over 356 source-cut attacks the median mitigation was 8.5 ms, close to the 7.6 ms of the controlled single-clock measurement in Section 4.6, with the false-data injections from the compromised trusted host forming a separate, slower mode near 100 ms because that path installs a conduit block after the first packets rather than an immediate source cut (Appendix Figures A.19 and A.20). The graded response reproduced exactly at repetition: 120 tier probes installed the criticality-scaled timeout with no deviation, 75 s for CRITICAL, 60 s for HIGH, 45 s for MEDIUM and 30 s for LOW, at a median engine decide-time of 0.50 ms, measured under the concurrent load of the fourteen-hour night campaign and so above the 0.026 ms of the dedicated clean run of Section 4.6 (Appendix Figure A.21), and the response ladder, the GUARD anti-spoof drop (a spoofed protected identity dropped on every one of 29 passes), the refused control API (`HTTP 401` throughout, the defence never disarmed) and the segmentation check all repeated their earlier verdicts. These runs cycle a fixed set of attack vectors rather than sampling a large space of unique source, operation and rate combinations, so their value is sustained repetition on the live process, not decision breadth, which the labelled corpus of Section 4.2 covers separately.

The earlier evaluation noted that no adversarial-benign stress had been run to try to force a false positive. That test was added here as a wire-level campaign, separate from the decision-level conformance corpus of Section 4.2: alongside the attacks, the run generated noisy but legitimate traffic, broadcast and multicast storms together with varied-rate legitimate reads from two allowlisted supervisory sources, and watched the fabric for any reactive rule landing on a legitimate source. Across fifteen sampled windows no legitimate operation was cut. This is an observed result under the tested stress profile, not a statistical guarantee, and the benign denominator is honestly small: the same zero-false-positive outcome holds across the thirty-eight legitimate or exempt cases of the labelled corpus (thirty ordinary legitimate, of which five are the mirrored safety-loop refusals, and eight flood-exempt), which by an exact binomial bound still admits a two-sided 95% upper limit near 9%. The result is therefore no observed false positive on this rulebook and this traffic, not a proven-low rate for arbitrary industrial networks. The process guardian's last-good restore was exercised repeatedly and behaved honestly, sometimes invoked to restore a nudged level and sometimes pre-empted by the reactive cut before drift developed.

The stress test also produced the campaign's clearest negative result, and it concerns the PLC rather than the defence. Section 4.12 noted analytically that an isolate installs a silent drop, which leaves a PLC holding a half-open S7 session until its own TCP timeout, and that on an S7-1200, whose connection pool is small, repeatedly isolating an attacker mid-session could exhaust that pool. The long run confirmed this on the device. The CPU 1212C exposes only six dynamic connection resources, a figure read directly from the CPU's connection-resources view in the engineering tool rather than inferred, and under the heaviest stress density, about five S7 isolations packed into each short cycle, those slots filled faster than the CPU reaped them and the legitimate Factory IO client was evicted, identified by the tank level freezing while the controller still reported the process online. The eviction is the endpoint's limit, not a defect in the defence, and this is verifiable on the switch. An isolate installs a single source-scoped drop that matches only the attacker's address (`priority=110, ip, nw_src=`*`attacker`*`, actions=drop`) and never the legitimate conduit; a live capture taken while an isolation was in force confirmed it, the reactive rule matching `nw_src=192.168.2.77` alone while the `192.168.2.55` → `192.168.2.10` conduit stayed a committed allow with its counter advancing, its only drop the dormant identity guard at zero packets (Listing A.48). The fabric therefore kept delivering the client's traffic to the PLC; the client was refused at the PLC, where the six slots were full, not at the switch. Pacing the attacks to one S7 isolation per cycle, so the slots had time to reap, changed the outcome: the process then ran for about ninety minutes under continuous attack. That the freeze depends on the isolation rate, and disappears when the same isolations are spaced out, is itself the discriminator: a rule error would not be sensitive to timing, whereas a connection pool being reaped is. The eviction counts are small and come from single runs, so they bound the effect rather than estimate a stable rate: the packed-stress run recorded two evictions

in roughly thirty minutes before it self-halted, whereas the two paced runs recorded three evictions over eighty-nine minutes and two over ninety-one minutes (Appendix Figure A.22).

The finding is that reactive isolation and a resource-constrained OT endpoint are in direct tension: the same drop that protects the process consumes a scarce connection slot on it, so on small CPUs the response must either rate-limit its isolations or emit a TCP reset toward the PLC on quarantine so the socket closes at once instead of lingering. That reset was not implemented or tested here; what this run establishes is the hazard under the current silent-drop isolation, on this CPU and this process, and the reset stays a plausible but unvalidated remedy carried in the future work.

Underneath these tests the process stayed live within the limits of the sampling. Across the four runs the controller and its remediation agent reported the process online in every one-minute snapshot, the tank level held within its control band, and the flow-integrity auditor reported no drift in steady state (Appendix Figure A.23). That online flag is the remediation agent's own link to the PLC, sampled once a minute, which is not the same as continuous Factory IO client availability: the paced run's two evictions were sub-minute events caught by the liveness guard and then recovered, so what is shown is controller-side continuity and an in-band process under the tested pacing, not a proof of uninterrupted client sessions. The two transport-layer stressors the campaign also fired, a spoofed-source SYN flood and a half-open connection pressure test against the PLC, are the same class of transport exhaustion already bounded in Section 4.12; under that load the controller's own decision time held at about 0.17 ms while a probe attack was still cut in roughly 100 ms, so the detection-and-decision path did not degrade even as the flood pressed on the PLC's network stack.

### 4.10.1 Framework limits: flow-table capacity

The connection pool is the endpoint's limit; the controller has its own, in the flow table it fills. Why the table grows at all is the granularity of the isolate response: an ISOLATE matches a single source address, so the controller installs one reactive drop rule, on every switch, for each distinct attacking source it quarantines. A flood mounted from many distinct source addresses therefore makes the controller install one new flow per source, and the reactive table grows in step with the number of distinct sources seen; a flood from a single source, by contrast, installs just one rule however many packets it sends. That per-source growth is the limit this section bounds. To reach it the enforcement layer was driven directly by the stress harness `flowtable_stress.py`: it posts a stream of *distinct* unregistered sources, drawn from the `100.64.0.0/10` carrier-grade-NAT range so that none is a real host, to `/cars/respond` as FORBIDDEN `CONTROL` requests against the critical PLC, and it is that distinctness, a new source address on every post, that forces the engine to classify each one FORBIDDEN and install a fresh source-scoped isolate per switch, so the reactive table fills one flow at a time. Concurrent worker threads raise the post rate to the saturation point in its `ceiling` mode, while a `sustain` mode holds the flood past twice the timeout to show the table plateaus rather than growing without bound; the gateway's OpenFlow table was sampled as it filled (Figure 4.12). The harness logic, the distinct-source flood and the sampling of the filling table, is stated as pseudocode in Listing 4.1.

```
globals: next_ip <- 100.64.0.0            // carrier-grade-NAT range, no real host

function NEXT_SOURCE():                    // a fresh, distinct source every call
    s <- next_ip; next_ip <- next_ip + 1   // this distinctness forces a new flow
    return s

function POST(src):                        // one attack event to the controller
    send {src, dst = CRITICAL_PLC, op = CONTROL, proto = S7} to /cars/respond
    // engine: unregistered src + forbidden CONTROL -> ISOLATE(src)
    //         -> installs one 0x00ca drop rule per switch

function WORKER(cap):                      // many run at once to raise the rate
    while not stopped:
        if reactive_flows() >= cap:  wait; continue     // safety ceiling
        POST(NEXT_SOURCE())

function RUN(threads, cap):
    start ‘threads‘ WORKERs
    every 5 s:
        sample reactive_flows, install_latency, decide_ms, vswitchd_memory
        if decide_ms spikes or install_latency > 25 ms or flows >= cap:
            record the ceiling; break          // first sign of saturation
    stop all WORKERs
    watch reactive_flows drain to zero as the hard_timeouts expire   // self-heal
```

Listing 4.1: The flow-table saturation harness (`flowtable_stress.py`): a flood of distinct sources posted to the controller, each forcing one source-scoped isolate, sampled as the reactive table fills and then self-heals on timeout.

The switches carry no configured flow cap and the kernel connection-tracker holds 262,144 entries, so the ceiling is set by the controller and the switch software rather than a fixed limit.

Under a single injector at about four hundred isolations a second the fabric was untroubled. Reactive flows tracked the sources one for one to twenty thousand, the per-install time stayed flat at two milliseconds and the decide time under a tenth of a millisecond, and the process ran undisturbed. Once the flood stopped the criticality-scaled hard timeout drained the whole table to zero within its window, so a sustained flood cannot grow the table without bound: it plateaus near the install rate times the timeout and self-heals from there.

Pushed harder the framework does reach a limit. Twenty-four concurrent injectors sustaining about two thousand isolations a second piled the gateway table to ninety-four thousand reactive flows. As it filled the per-install time bent from ten to twenty-five milliseconds and the switch daemon's resident memory grew from two hundred and eighty to three hundred and ninety-three megabytes at the peak, lingering near four hundred and twenty-five as the daemon released it only slowly, and near the peak the flow-modification load saturated the switch control plane: the OpenFlow channel to all three switches dropped, they fell back to `fail_mode=secure`, and the synchronous S7 session between the simulator and the PLC stalled until the driver was reconnected. The table then self-healed to zero within twenty-three seconds.

Two points matter. The disruption is not the isolate rules cutting legitimate traffic, which they cannot do by construction, since each matches only the attacker's source (Listing A.48); it is the control-plane cost of installing rules too fast, the same class of load the controller-reconnection test exposed in Section 4.12. And the fail-secure switches kept forwarding the process throughout the disconnect, so the plant survived the controller losing its fabric, which is the behaviour a centralised design must have. A secondary effect appeared in the controller's book-keeping: the burst of flow-removed notifications from ninety thousand near-simultaneous expiries outran the single-threaded controller and left stale entries in its block-tracking set until the switches reconnected. The reaper is correct under normal load but lags under a self-heal storm. That installation slows and the channel saturates as the table fills is consistent with a known limit of the software OpenFlow datapath rather than a fault specific to CARS: Open vSwitch has been shown not to scale well to large OpenFlow rule-sets, its tuple-space-search megaflow classifier [34] coming to dominate datapath processing, on the order of 86 to 97 per cent of CPU time once rules reach the hundred-thousands, with each rule change at that scale forcing megaflow invalidation and control-path upcalls that depress throughput [35]. A reactive scheme that installs one drop per attacking source inherits that ceiling, which is what the right-hand panel of Figure 4.12 records as the per-install time bends and the fabric drops to fail-secure near ninety-four thousand flows.

This limit is real but sits well above the modelled threat. A genuine attacker's isolations arrive through the detection path, whose single-threaded bridge caps the achievable install rate far below two thousand a second, while its three-second per-source cooldown means any one source contributes at most a single flow however hard it attacks; so filling the table needs both many distinct sources and a rate that the bridge cannot deliver, and it stays in the graceful regime with a wide margin. The ninety-four-thousand-flow figure was only reachable by driving the internal decision interface directly, bypassing that bridge, on the isolated management plane. The engineering response is to rate-limit or batch reactive-rule installation so a burst cannot saturate the control channel, and to sweep the block-tracking set on a timer rather than relying on flow-removed events alone. Both are carried in the future work.

## 4.11 Comparison with related work

CARS sits within the body of work on software-defined response for industrial networks, and its contribution is clearest against that work's known limits. Surveys of reactive SDN defence for ICS observe that most systems respond uniformly, dropping or redirecting a flagged flow regardless of what the flow was doing or how important its target is [2]. Concrete systems bear this out: the two-level P4 whitelist-and-DPI IDS of Kabasele Ndonda and Sadre [13], the cloud-hosted ICS detection-and-prevention framework of Brugman et al. [15] and the DNP3 intrusion detection and prevention system DIDEROT [16] all detect and then drop or block, without scaling that response to the asset's criticality. CARS decides instead on two axes, the industrial operation recovered from the packet and the criticality of the asset, so the same

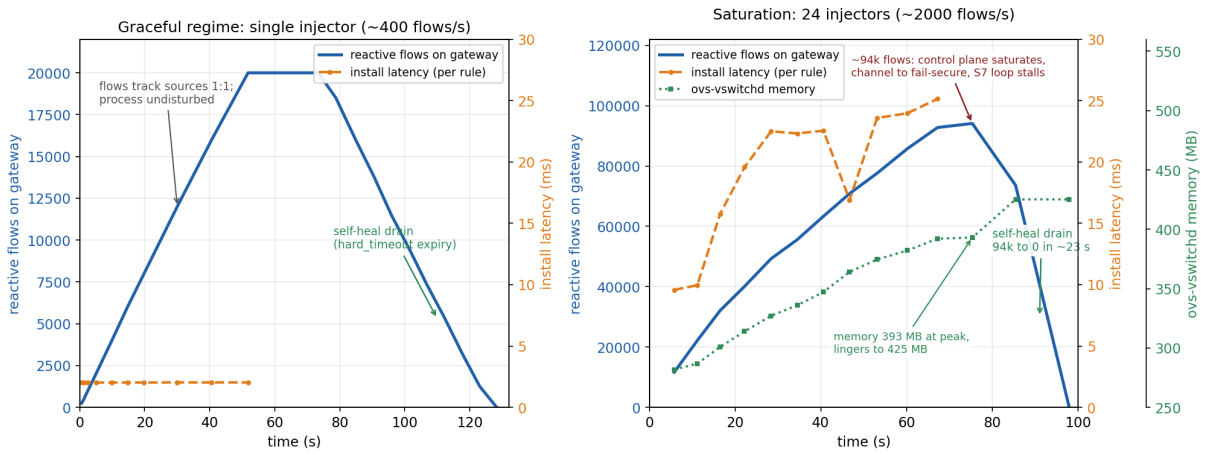


Figure 4.12: Framework flow-table behaviour under a distinct-source isolate flood, driving the enforcement interface directly. Left: a single injector at about four hundred a second accumulates reactive flows one for one to twenty thousand, with the per-install time flat at 2 ms and the process undisturbed, and the criticality-scaled `hard_timeout` drains the table to zero once the flood stops. Right: twenty-four injectors at about two thousand a second reach ninety-four thousand flows, at which the per-install time bends to 25 ms, `ovs-vswitchd` memory reaches 393 MB at that peak and lingers to 425 MB through the drain as the daemon releases it only slowly, and the flow-modification load drops the controller-switch channel to fail-secure and stalls the S7 loop; the table then self-heals to zero in about twenty-three seconds. Each series is drawn against its own axis. Source: `flowtable_ramp_20k.csv` and `flowtable_ceiling_94k.csv`, plotted by `fig_flowtable.py`.

forbidden control draws a 75-second quarantine on the CRITICAL PLC and a 30-second block on the LOW-value unit, and a read from a source is allowed where a write from it is refused. The evaluation shows this grading changes the measured response tier by tier (Section 4.3), rather than being cosmetic.

The self-checking layer relates to the policy-verification work of Melis et al. [20], which checks the consistency of installed flow rules; the CARS flow-integrity checker performs the same class of check against a trusted baseline and, in the campaign, caught an injected rule and a deleted conduit within one poll. The threat it defends, a control-plane host that rewrites the fabric, is the one identified by Gardiner et al. [4]; CARS adds an authenticated control interface so the defence cannot be silently disarmed, which the campaign confirmed with the refused `HTTP 401`. The stateful shield in the pipeline follows the data-plane connection-tracking idea of AVANT-GUARD [21], and the replica-and-deception direction of Etxezarreta et al. [3] is an alternative to the CARS block-and-maintain, trading a decoy for a graded cut. The response mechanism itself has a precedent in the virtualised, SDN-based incident-response functions of Murillo Piedrahita et al. [17]; CARS differs in binding the response to the operation recovered from the packet and the criticality of the asset, rather than acting on the flow alone.

What separates CARS from the SDN-response systems it builds on, including the intrusion-response design of Samanis [25], is the deployment barrier it targets. The reactive defences in the literature are attractive but largely undeployed on live OT, because an over-eager block can cut the process; the central result here is a 0% false-positive rate on the live process traffic with a bounded, reversible, evidence-generating response. That property, more than raw detection, is what an operator needs before a reactive defence can be armed on a running plant.

## 4.12 Threats to validity

### 4.12.1 Scope of the validation

The results are honest within limits that should be stated plainly. This is a functional-prototype validation on a single testbed with one process, not a product trial. The conformance is exact over the twenty-four scored cases and is corroborated by the campaign's live decisions, but twenty-four scored permutations on one bench cannot guarantee a zero false-positive rate on a large, noisy production network; that would need a far larger and more varied traffic sample. An adversarial-benign stress was subsequently added in the long-run campaign (Section 4.10): broadcast and multicast storms with varied-rate legitimate reads

from allowlisted sources, over fifteen sampled windows, cut no legitimate operation. That strengthens the result beyond the scored decision space, but it stays a result on this rulebook and this traffic, not a deliberate attempt with every malformed-but-valid variant. The result should be read as: on this system and its rulebook, across the decision space and the live campaign, no legitimate operation was wrongly cut, and not as a statistical guarantee for arbitrary industrial traffic. The physical process is a hardware-in-the-loop: the PLC, the S7 traffic, the control logic and the enforcement are all real, but the water is simulated in Factory IO, so the overflow is a faithful process consequence rather than a spill of real fluid.

### 4.12.2 The reaction window and first-packet residual

The reaction window deserves a sharper caveat than a general remark. Over a hundred trials the median mitigation was 7.6 ms and the 99th percentile 67.9 ms for a single injection, while a sustained flood fired continuously from the real attacker virtual machine was cut in about one second. On this slow tank that second is harmless; on a fast process, high-speed manufacturing or an electrical protection loop, one second is ample time for physical damage, and the reactive layer alone would not suffice there. The mitigation is architectural: the proactive layer, identity binding, allowlist and default-deny, drops a non-conduit or spoofed packet with no reaction window at all, so the residual exposure is a novel operation on an already-permitted conduit. This distinction bounds the pipelined-burst worry: an unregistered attacker never gets a window, because the default-deny stops it at the TCP handshake before any payload lands (the two pivots of Section 4.7 landed zero writes), so a burst of malicious writes inside the window is only possible from a compromised but allowlisted host, where the first packets reach the PLC by design and the injection of Section 4.8 was isolated after nine packets, at most one of which reached the PLC.

The leak is sharper for a discrete state change than for the analogue process measured here: process inertia and the cyclic scan absorb a leaked write to a continuous level, but a single diagnostic command such as an S7 `STOP` (`0x29`) changes the controller's state on receipt, so on a compromised conduit the reaction window cannot prevent an atomic operation that its first packet completes (Figure 4.13), and reaction latency, however small, is not the relevant defence for that class. A short pre-isolation burst on such a conduit is therefore the true residual, narrowed by pushing more coverage into the proactive layer and closed at the endpoint only by a TCP reset on quarantine, both carried in the future work. Narrowing it further is a matter of pushing more coverage into the proactive layer or shortening the detection poll, and a fast process would demand both.

The window measured here also reflects this attacker's cadence and the testbed's single-controller path: the injection client sends well below line rate and the control path carries no competing controller load, so it is a partial proxy for a production plant under concurrent traffic, scheduling jitter or multi-controller pressure, where queueing on the detection path would widen it, as the load correlation of Section 4.6 already begins to show.

### 4.12.3 Detection-sensor coverage

A further limit is in the detection sensor, not the policy. The reactive layer depends on Snort to recover the operation from the packet. An earlier configuration inspected single packets at fixed offsets, which let a fragmented operation on an already-permitted conduit evade recognition; that specific gap was closed by enabling stream reassembly and re-anchoring the rules to the protocol header (Section 4.5.1), and a fragmented write is now recovered and blocked at more than one split point. What remains is narrower: the rarest Modbus function codes are still recognised only intermittently, and a sufficiently crafted overlapping-segment or timing evasion is a general IDS problem this work does not claim to settle. The rule anchoring also assumes the usual header layout: the S7 rules key on the `32 01` job header behind a standard three-byte COTP, and the Modbus rules read the function code at the standard MBAP offset, so a non-standard COTP length or a Modbus-in-tunnel encapsulation would shift the field and drop recovery back to the stateless five-tuple layer, the same sensor-coverage class as the residual above. The decision logic classifies every operation correctly once it is known, so this is a sensor-coverage limit rather than a flaw in the response, and sharpening the sensor's protocol parsing further is future work. Enabling stream reassembly to close the fragmentation gap does add state to the sensor, which a reassembly-targeted session flood could try to exhaust; that state is bounded on this deployment (`max_tcp 8192` sessions, sixty-second timeout), and because the sensor runs on a passive mirror rather than inline, exhausting it degrades detection rather than stalling forwarding, since the proactive rules live in the switches independently of the sensor. The consequence of a sensor overload is therefore a loss of

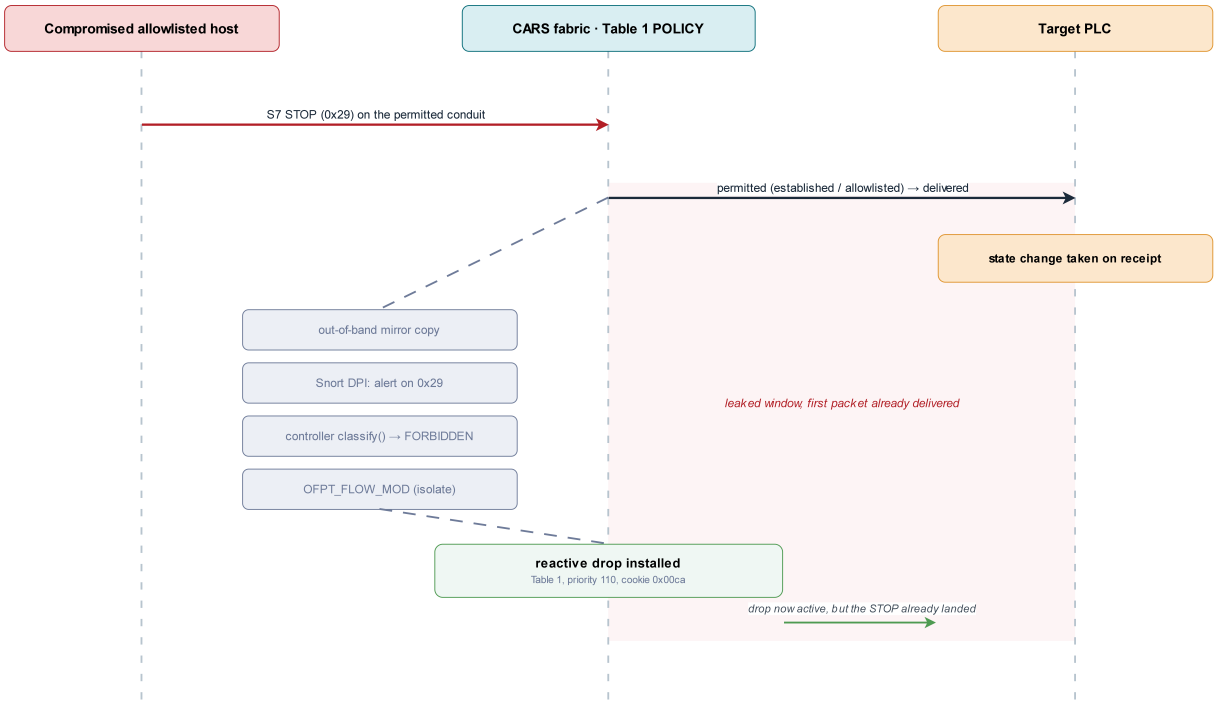


Figure 4.13: The first-packet residual on a compromised allowlisted conduit. The permitted conduit's first packet, here an S7 `STOP` (`0x29`), is delivered before the out-of-band detect-then-isolate path installs the reactive drop (Table 1, priority 110, cookie `0x00ca`), so for an atomic operation reaction latency does not bound the outcome. An unregistered source never reaches this point, as the default-deny stops it at the handshake; the residual is the same compromised-conduit case measured in Section 4.4, where the first packets landed by design. The figure illustrates that timing analytically; it is not a measured process-stop.

operation-awareness on allowlisted conduits, not a halted process, and per-source metering on the mirror is the further mitigation.

The sensor also has a spatial reach limited to what the gateway mirror carries: an attack that stayed within a single cell and never crossed the gateway, such as one launched from a foothold inside the second cell against its local PLC, would not reach the mirror and would draw no operation-aware response (illustrated in Appendix Figure A.1), though the proactive identity binding and default-deny still apply on that switch independently of the sensor, so an unregistered intra-cell source is still refused. Recovering full operation-awareness inside every cell would need per-cell sensing, a distributed Snort or a data-plane classifier, and is future work.

A deeper limit sits underneath this: the operation-awareness depends on a readable payload. The classic S7comm and Modbus/TCP of this testbed carry the function in clear, but modern industrial protocols that encrypt or integrity-protect the payload, such as S7CommPlus on newer Siemens firmware, OPC UA with security enabled, or CIP Security, would deny the sensor the operation byte altogether. Under such a protocol the operation-aware reactive tier degrades to what can be judged without the payload, while the proactive stateful layer, the identity binding, the role and conduit allowlist and the default-deny, and the process guardian still hold; recovering operation-awareness there would need endpoint or host-based operation telemetry, or a protocol-aware proxy that terminates the secure session, both beyond this work.

### 4.12.4 Architectural limits

Several further limits are architectural rather than defects in the decision logic, and they mark where a lab prototype stops short of a hardened deployment. The OpenFlow control channel runs in plaintext (`tcp:6653`), so an attacker who reached the control plane could intercept or forge flow rules; that plane is a separate wired management network (`10.10.10.0/24`) which none of the evaluated attackers can reach, so the exposure is bounded on this testbed. Relying on segmentation alone is nonetheless a weak posture for a centralised controller: a misconfiguration, a VLAN-hopping technique, or an engineering station

dual-homed onto both the operational and management planes would defeat it, leaving the channel open to the control-plane interception and rule-forgery set out in Section 2.4 [4]. The proper mitigation is to run OpenFlow over TLS; this prototype does not implement it, and does not measure its latency overhead.

The flow-integrity checker polls every ten seconds, which bounds but does not close its window. A fresh thirty-trial test of each case quantified this: a persistent injected rule was caught on every trial, thirty of thirty, at a median of 8.5 seconds, whereas a rule injected and withdrawn inside a two-second window evaded the auditor on twenty-five of thirty trials, caught only on the five occasions its life happened to straddle a poll boundary (Appendix A.11). Polling has an inherent blind spot below its interval, so an event-driven flow-monitor was built to close it: subscribing to the switch's Nicira flow-monitor, it checks every change to the policy tables against the trusted baseline the instant the change is installed, rather than sampling every ten seconds. Re-running the same thirty-trial transient test with the monitor active, the two-second injection was caught on every trial, thirty of thirty, at a median of 0.27 seconds, against five of thirty for the poll, closing the sub-poll gap on the deployed OpenFlow stack (Appendix A.11; detection rate and latency are plotted in Appendix Figure A.15).

The fabric was tested against volumetric floods from known endpoints but not against a spoofed-source state-exhaustion flood that could fill the connection-tracking table; GUARD drops spoofed protected identities before they reach conntrack, but random unregistered spoofed sources would still consume state, and conntrack limits or per-source metering are the mitigation. The reactive flow table itself was stress-tested to saturation (Section 4.10.1): the criticality-scaled timeout bounds and drains it, and at a very high injection rate the flow-modification load, not the table size, is what degrades the fabric. The specific vector left untested is the connection-tracking state a spoofed-source TCP flood would consume, which the isolate drops themselves do not exercise. The stateful stage tracks each connection by its five-tuple and the kernel connection-tracker's TCP state, and GUARD blocks a source that spoofs a protected identity at the port and MAC, so an out-of-state packet is refused, as the forged-ACK case confirmed; a finer attack, an on-segment host injecting a correctly-sequenced segment into an already-established flow, was not specifically tested, and its outcome depends on the connection-tracker's TCP-window-tracking configuration rather than on the policy.

The detection bridge reads Snort's alert file and posts over HTTP; with a follow-on-write tail and a per-conduit cooldown this is adequate at the loads measured, but a heavy multi-source flood would bottleneck it, and a production system would use in-memory inter-process communication. That reporting interface is also unauthenticated by design: the shared-token gate that protects the state-changing endpoints deliberately excludes `/cars/respond` so the sensor feed stays lightweight, which means a host that could reach the reporting interface could forge an alert and drive a reactive isolation of its choosing. The exposure is bounded by the same management-plane isolation as the control channel, and no evaluated attacker could reach it, but a routable deployment would need an authenticated or HMAC-signed feed on `/cars/respond`, carried in the future work.

The enforcement also carries a consequence worth stating plainly: the isolate and block responses install a silent `drop`, and for a session already established to a PLC that leaves the PLC holding a half-open socket until its own TCP timeout expires, because its outbound segments go unacknowledged rather than being reset. On an S7-1200, whose concurrent-connection pool is small, repeatedly isolating an attacker mid-session could exhaust that pool and degrade the very device the defence protects. The long-run campaign confirmed this on the device (Section 4.10): the CPU 1212C exposes six dynamic connection resources, and under a packed isolation density the legitimate Factory IO client was evicted at about 4.0 times an hour and the run could not be sustained, whereas pacing the isolations held the process for about ninety minutes at 1.3 to 2.0 evictions an hour. Emitting a TCP reset toward the PLC on quarantine, so its socket closes at once rather than lingering as a half-open session, is the remedy, carried in the future work and now backed by measurement rather than analysis alone.

The controller is also a central dependency, noted for compromise in Section 2.4 but with an availability face as well, and a two-part reconnection test drew the line between the two planes. Restarting the Snort sensor while the process ran was transparent: the inter-frame timing on the live conduit was indistinguishable from baseline (mean 4.90 against 4.91 ms, worst gap 33 against 42 ms) because the sensor is passive and out of band, so re-attaching it never touches forwarding. Forcing a controller reconnection was not: tearing the controller off and back onto the switches opened a roughly twenty-second window with multi-second forwarding stalls, during which the synchronous S7 session between the process simulator and the PLC dropped and did not auto-recover, freezing the tank until the driver was reconnected by hand (Appendix Figure A.18). That figure must be read with a caveat, however. The switches already run with `fail_mode=secure`, the correct hardening, so a genuine controller crash that

leaves the installed flow rules intact should keep the data plane forwarding; the disruption observed here largely reflects the heavy delete-and-re-add reconnection and the controller's pipeline re-sync rather than a faithful crash, and the precise crash-transparency figure was left for careful future work rather than re-frozen out of the live process (Appendix A.12). Operational availability under a genuine controller failure is therefore not proven here, only bounded: the fail-secure switches should keep the data plane forwarding on a clean crash, but a faithful crash test at production depth remains outstanding, so this is a limit of scope rather than a claim of crash resilience. Nor was the controller's headroom on a fabric far larger than three switches and two PLCs measured, where many more nodes and multicast-heavy protocols such as PROFINET could add processing jitter. Each of these is a known engineering step, identified and bounded here, not a flaw in the response model.

### 4.12.5 Trust boundaries carried forward

Four further boundaries are carried through honestly. Layer-two host discovery by ARP broadcast is a network function and is not policy-gated, so an attacker on the segment learns the host inventory even under CARS; only the services and the operations stay hidden. The operation-aware policy is likewise scoped to IP-based industrial protocols: S7comm and Modbus/TCP are gated, but real-time protocols carried directly over Ethernet, such as PROFINET RT and the IEC 61850 GOOSE and Sampled-Values traffic of substation automation, are not IP-routed, fall to layer-two forwarding beneath the IP-scoped default-deny, and are outside the operation-aware scope of this prototype; gating them would need layer-two match fields in the pipeline and is future work. The system trusts its role assignment: GUARD stops an attacker impersonating a role at the port and MAC level, but a genuinely compromised host acting within the operations its role is permitted is trusted by design and only monitored.

The starkest case is a genuinely trusted party acting within its permitted operations, the compromised HMI or corrupted agent behind classic ICS incidents; this design bounds that case on three sides without eliminating it. The rate overlay already cuts the high-rate injection an overflow requires, shown when a trusted identity's flood was blocked and the tank held (Section 4.9), and the PLC program offers no low-rate path to the same effect. A process guardian, added as a read-only defence-in-depth layer, alarms on the high-side envelope and liveness anomalies that the network must not act on and that the low-side remediation agent misses, catching both a trusted stop and a reported-full sensor spoof in testing. What is left is genuinely hard: a manipulation that stays within the physical envelope and moves the process slowly and plausibly evades an invariant monitor as it evades the agent, and the remediation agent itself, trusted to issue any operation so it can restore the process, is trusted by design, though a high-rate abuse of that role is still cut by the flood overlay (a runaway agent driven at flood rate against the PLC was blocked in testing), so only a low-rate, in-role abuse of it slips through. The guardian is detection, not prevention, since automating prevention on the safety loop is the risk this whole approach avoids. Finally, a fully subverted controller that knows the defence's own internal markings is outside what a self-checking control plane can catch. These are boundaries of the approach, deliberate or fundamental, not defects of the implementation, and they bound the claims made here.

## 4.13 Ethics

All offensive work was conducted on an isolated, self-contained testbed with no connection to any production or external network, using hardware and a simulated process owned by the research group. No live industrial infrastructure was touched, and no data beyond the group's own was involved. The attacks, including the process-devastation injection, were run to measure a defence and were reversed after each run, with the process returned to its safe state. The intent throughout is defensive: to establish whether a reactive network defence can be trusted on a live process, so that operators have evidence before arming one. The tooling and the findings are reported at the level needed to substantiate the results and to be reproduced by other researchers, without providing a turnkey attack against any specific deployed system.

# Chapter 5

# Conclusion

Automated intrusion response is still rarely enabled on operational technology, and the reason is trust rather than detection: an operator will not arm a defence that might cut the process it is meant to protect (Chapter 1). This project set out to lower that barrier with CARS, an SDN intrusion-response framework whose action is conditioned on the criticality of the asset and the industrial operation being attempted, and whose responses are bounded in time, reversible, and evidence-generating. It was built on a hardware testbed with two real Siemens S7-1212C PLCs and a live tank process, and evaluated at the level of individual packets. The central result is that the armed defence refused every illegitimate operation while cutting none of the legitimate process traffic: a false-positive rate of zero on the live process, with a response that scales by criticality and enforcement shown on the wire rather than inferred from a log. That property, more than raw detection, is what an operator needs before arming a reactive defence on a running plant.

## 5.1 Contributions

The project met the five objectives set out in Chapter 1, summarised against their outcomes in Table 5.1. A representative three-node SDN-ICS testbed was built with real PLCs, an operator panel, an engineering workstation and a hardware-in-the-loop tank, so that every response could be judged against a physical consequence rather than a simulated one. On that fabric, CARS was realised as a three-table OpenFlow pipeline, identity binding against spoofing, a stateful policy stage carrying the criticality- and operation-aware rulebook, and an L2 switch, behind which sit a graded seven-level response ladder, a flow-integrity self-check and an authenticated control interface. The decision logic was measured across the range of source role, operation class and asset criticality and returned a correct verdict for every case, with no legitimate operation wrongly cut, and the same verdicts were corroborated by the live campaign. Each capability was then validated at the packet level, protected against unprotected, from reconnaissance through identity spoofing, out-of-state traffic, control-plane and flow tampering, operation-aware commands, and a false-data injection that overflowed the tank unprotected yet, when the defence was armed, was severed at its first write, a single frame reaching the PLC before the cut and no write moving the process. The reaction window was characterised directly, with a median mitigation of 7.6 ms, and the whole account was drawn together in an end-to-end trace of two attacker positions from one machine, which showed the same enforcement applied at coarse gateway granularity for a NAT-collapsed external attacker and at fine per-host granularity for an on-segment insider.

## 5.2 What the evaluation establishes, and what it does not

The result is best read as a layered account of responsibility rather than a single defence. The proactive layer, identity binding and a stateful default-deny allowlist, stops a source that should never connect with no reaction window at all, which the two-pivot trace confirmed for both the external and the on-segment attacker. The reactive, operation-aware layer governs what an already-permitted party may then do, refusing a dangerous operation on a critical asset while permitting a read; the volumetric overlay cuts high-rate abuse even from a trusted role; and, added as defence in depth, a read-only process guardian raises an alarm on the physical anomalies the network layer must not act on. Together these draw a clean division: the network decides who and what may connect, and a process-invariant monitor watches the

| Objective | Outcome |
|---|---|
| Design a graded, bounded, reversible, evidence-generating response model | Seven-rung ladder (ALLOW to REFUSE) with criticality-scaled self-healing durations (75/60/45/30 s) and an audit record per decision (Chapters 3, 4) |
| Implement it as an OpenFlow controller with a layered pipeline and integrity checks | Three-table pipeline (GUARD, stateful POLICY, SWITCH), flow-integrity checker and token-authenticated control API (Chapters 3, 4) |
| Keep the design portable, a core separated from site-specific configuration, applicable to any plant | Engine decoupled from a site file of assets, roles, conduits and tiers, verified to reproduce the deployed policy, and run unchanged in a hardware-free emulation (Chapter 3) |
| Build a hardware testbed with real PLCs, an operator panel and a live process, and instantiate CARS on it for real-consequence testing | Three-node fabric, two S7-1212C PLCs, KTP700 panel, EWS and a Factory IO tank on the live PLC, judged by overflow versus safe band (Chapters 3, 4) |
| Evaluate decision conformance and false positives; validate each capability at the packet level, protected versus unprotected | zero false positives and zero false negatives over the decision space and a 2,078-case corpus; wire-level campaign across every capability; reaction window characterised (median 7.6 ms) (Chapter 4) |

Table 5.1: Aims and objectives against outcomes.

consequence.

The evaluation is honest about where a lab prototype stops. It is a functional-prototype validation on a single testbed with one hardware-in-the-loop process, so the zero-false-positive figure is a statement about this system and its rulebook across the decision space and the live campaign, not a statistical guarantee for arbitrary industrial traffic. The reaction window is a few milliseconds here but is a property of any detect-then-respond defence, and a faster process would demand more of the proactive layer. The residual adversary the design cannot fully answer is a trusted party manipulating the process slowly and within its physical envelope; the guardian narrows this case but does not close it, because prevention on the safety loop is the very thing that must not be automated. Several further limits are architectural rather than flaws in the response model, and are carried openly in Chapter 4: the OpenFlow control channel runs in plaintext on an isolated management plane, the flow-integrity poll left a sub-interval blind spot that an event-driven monitor was added to close, the fabric was not tested against a spoofed-source state-exhaustion flood, and the file-based detection bridge would bottleneck under a heavy distributed load.

## 5.3 Future work

The limitations above map directly onto the next steps.

- **Toward deployment.** Run OpenFlow over TLS and measure the added control-path latency; test and harden the fabric against spoofed-source state-exhaustion flooding with connection-tracking limits and per-source metering; and move the detection bridge from file-and-HTTP to in-memory inter-process communication to hold the reaction window under heavy load. The transient-injection blind spot in the flow-integrity poll has already been closed with the event-driven flow-monitor added in Chapter 4, and the coarse quarantine of a NAT-collapsed identity has been refined to a conduit-level cut; both are folded into the evaluated system rather than left as proposals.
- **Deepening the process layer.** Generalise the process guardian into sensor-aware remediation on the live process, with setpoint- and invariant-level anomaly detection, so that a trusted insider driving the process slowly within its envelope can be flagged rather than only a gross excursion.
- **Extending the response repertoire.** Give the DEFLECT path a fully interactive decoy that sustains a session with the attacker, and add a TCP reset on quarantine so a recoverable endpoint fails fast instead of hanging on a silent drop.
- **Broader validation.** Exercise the decision logic against a larger and noisier traffic sample, more assets and alternative rulebooks, and a faster process, to move the conformance result from a functional-prototype demonstration toward a statistical one; and extend enforcement across the full multi-cell fabric.

- **Advanced deception.** Explore replica-based moving-target defence as an alternative to block-and-maintain, trading a decoy and a shifting target for a graded cut [3].

CARS does not claim to solve intrusion response for industrial control systems. What it shows is narrower and, for the deployment question, more useful: that a reactive SDN defence can be made criticality- and operation-aware, bounded and reversible enough to stop real attacks on a live process at the packet level without, by policy, ever cutting the process itself. On the evidence of this testbed, the trust barrier that keeps such defences switched off is not fundamental, and an operator given a defence that refuses an unsafe action before it will interrupt the plant has reason to arm it.

# Bibliography


[1] National Institute of Standards and Technology, "Guide to operational technology (OT) security," NIST Special Publication 800-82 Rev. 3, National Institute of Standards and Technology, Sept. 2023.

[2] X. Etxezarreta, I. Garitano, M. Iturbe, and U. Zurutuza, "Software-defined networking approaches for intrusion response in industrial control systems: A survey," *International Journal of Critical Infrastructure Protection*, vol. 42, p. 100615, 2023.

[3] X. Etxezarreta, F. Turrin, I. Garitano, M. Iturbe, U. Zurutuza, and M. Conti, "Replica-based moving target defense against injection attacks in software-defined industrial control systems," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 3, pp. 5163–5180, 2026.

[4] J. Gardiner, N. Race, A. Eiffert, S. Nagaraja, P. Garraghan, and A. Rashid, "Controller-in-the-middle: Attacks on software defined networks in industrial control systems," in *Proceedings of the 2nd Workshop on CPS & IoT Security and Privacy (CPSIoTSec '21)*, (Virtual Event, Republic of Korea), pp. 63–68, ACM, Nov. 2021.

[5] Idaho National Laboratory, "Consequence-driven cyber-informed engineering (CCE): Mission support center concept paper," tech. rep., Idaho National Laboratory, Mission Support Center, National and Homeland Security Directorate, Oct. 2016.

[6] Cybersecurity and Infrastructure Security Agency, "Foundations for OT cybersecurity: Asset inventory guidance for owners and operators," tech. rep., U.S. Cybersecurity and Infrastructure Security Agency (CISA), Aug. 2025.

[7] Open Networking Foundation, "OpenFlow switch specification, version 1.3.0 (wire protocol 0x04)," ONF Technical Specification TS-006, Open Networking Foundation, June 2012.

[8] A. Lara, A. Kolasani, and B. Ramamurthy, "Network innovation using OpenFlow: A survey," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 493–512, 2014.

[9] A. H. Abdi, L. Audah, A. Salh, M. A. Alhartomi, H. Rasheed, S. Ahmed, and A. Tahir, "Security control and data planes of SDN: A comprehensive review of traditional, AI, and MTD approaches to security solutions," *IEEE Access*, vol. 12, pp. 69941–69980, 2024.

[10] S. Rivera, S. Lagraa, C. Nita-Rotaru, S. Becker, and R. State, "ROS-defender: SDN-based security policy enforcement for robotic applications," in *2019 IEEE Security and Privacy Workshops (SPW)*, pp. 114–119, IEEE, 2019.

[11] D. Jin, Z. Li, C. Hannon, C. Chen, J. Wang, M. Shahidehpour, and C. W. Lee, "Toward a cyber resilient and secure microgrid using software-defined networking," *IEEE Transactions on Smart Grid*, vol. 8, no. 5, pp. 2494–2504, 2017.

[12] S. Lakshminarayana, Y. Chen, C. Konstantinou, D. Mashima, and A. K. Srivastava, "Survey of moving target defense in power grids: Design principles, tradeoffs, and future directions," *arXiv preprint arXiv:2409.18317*, 2024.

[13] G. Kabasele Ndonda and R. Sadre, "A two-level intrusion detection system for industrial control system networks using P4," in *Proceedings of the 5th International Symposium for ICS & SCADA Cyber Security Research (ICS-CSR)*, BCS Learning & Development, 2018.

[14] M. Sainz, I. Garitano, M. Iturbe, and U. Zurutuza, "Deep packet inspection for intelligent intrusion detection in software-defined industrial networks: A proof of concept," *Logic Journal of the IGPL*, vol. 28, no. 4, pp. 461–472, 2020.

[15] J. Brugman, M. Khan, S. Kasera, and M. Parvania, "Cloud based intrusion detection and prevention system for industrial control systems using software defined networking," in *2019 Resilience Week (RWS)*, IEEE, 2019.

[16] P. Radoglou-Grammatikis, P. Sarigiannidis, G. Efstathopoulos, P.-A. Karypidis, and A. Sarigiannidis, "DIDEROT: An intrusion detection and prevention system for DNP3-based SCADA systems," in *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES '20)*, (Virtual Event, Ireland), pp. 1–8, ACM, Aug. 2020.

[17] A. F. Murillo Piedrahita, V. Gaur, J. Giraldo, Á. A. Cárdenas, and S. J. Rueda, "Virtual incident response functions in control systems," *Computer Networks*, vol. 135, pp. 147–159, 2018.

[18] S. Oyucu, O. Polat, M. Türkoğlu, H. Polat, A. Aksöz, and M. T. Ağdaş, "Ensemble learning framework for DDoS detection in SDN-based SCADA systems," *Sensors*, vol. 24, no. 1, p. 155, 2024.

[19] P. Manso, J. Moura, and C. Serrão, "SDN-based intrusion detection system for early detection and mitigation of DDoS attacks," *Information*, vol. 10, no. 3, p. 106, 2019.

[20] A. Melis, D. Berardi, C. Contoli, F. Callegati, F. Esposito, and M. Prandini, "A policy checker approach for secure industrial SDN," in *2018 2nd Cyber Security in Networking Conference (CSNet)*, IEEE, 2018.

[21] S. Shin, V. Yegneswaran, P. Porras, and G. Gu, "AVANT-GUARD: Scalable and vigilant switch flow management in software-defined networks," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13)*, (Berlin, Germany), pp. 413–424, ACM, Nov. 2013.

[22] F. Holik, M. M. Cook, X. Li, A. A. Shah, and D. Pezaros, "Programmable data planes for increased digital resilience in OT networks," *IEEE Communications Magazine*, vol. 63, no. 7, pp. 162–169, 2025.

[23] N. Goldenberg and A. Wool, "Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems," *International Journal of Critical Infrastructure Protection*, vol. 6, pp. 63–75, 2013.

[24] A. Kpoze, J. Degila, and A. Ahouandjinou, "Leveraging network reconfiguration to mitigate stealthy FDI attacks in smart grid SCADA systems by exploiting attacker uncertainty," in *Towards New e-Infrastructure and e-Services for Developing Countries (AFRICOMM 2024)*, vol. 652 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pp. 293–314, Springer, 2025.

[25] E. Samanis, *Protecting Against Reconnaissance Attacks on Industrial Control Systems: A Moving Target Defense Approach*. PhD thesis, University of Bristol, Aug. 2025.

[26] E. Samanis, J. Gardiner, and A. Rashid, "SoK: A taxonomy for contrasting industrial control systems asset discovery tools," in *Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES '22)*, (Vienna, Austria), pp. 1–12, ACM, 2022.

[27] D. Antonioli and N. O. Tippenhauer, "MiniCPS: A toolkit for security research on CPS networks," in *Proceedings of the First ACM Workshop on Cyber-Physical Systems-Security and/or PrivaCy (CPS-SPC '15)*, (Denver, Colorado, USA), pp. 91–100, ACM, Oct. 2015.

[28] J. Gardiner, B. Craggs, B. Green, and A. Rashid, "Oops I Did It Again: Further adventures in the land of ICS security testbeds," in *Proceedings of the ACM Workshop on Cyber-Physical Systems Security and PrivaCy (CPS-SPC '19)*, (London, UK), pp. 75–86, ACM, Nov. 2019.

[29] International Electrotechnical Commission, "Functional safety of electrical/electronic/programmable electronic safety-related systems," International Standard IEC 61508:2010, International Electrotechnical Commission (IEC), 2010.

[30] Forum of Incident Response and Security Teams, "Common vulnerability scoring system version 3.1: Specification document," tech. rep., Forum of Incident Response and Security Teams (FIRST), 2019.

[31] International Electrotechnical Commission, "Industrial communication networks – network and system security – part 3-3: System security requirements and security levels," International Standard IEC 62443-3-3:2013, International Electrotechnical Commission (IEC), 2013.

[32] A. Tsuchiya, F. Fraile, I. Koshijima, Á. Ortiz, and R. Poler, "Software defined networking firewall for Industry 4.0 manufacturing systems," *Journal of Industrial Engineering and Management*, vol. 11, no. 2, pp. 318–333, 2018.

[33] Á. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS '11)*, (Hong Kong, China), pp. 355–366, ACM, Mar. 2011.

[34] B. Pfaff, J. Pettit, T. Koponen, E. J. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado, "The design and implementation of Open vSwitch," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pp. 117–130, USENIX Association, 2015.

[35] A. Rashelbach, O. Rottenstreich, and M. Silberstein, "Scaling Open vSwitch with a computational cache," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pp. 1359–1374, USENIX Association, 2022.

# Appendix A

# Supporting Material

This appendix reproduces the full captures that back the diagrams, tables and claims of the Design and Implementation chapter (Chapter 3) and the Critical Evaluation chapter (Chapter 4). The configuration and pipeline captures of Sections A.1 to A.4 are from the stable baseline of 3 August 2026; the evaluation artefacts of Sections A.5 onward are from the dedicated fresh runs of 5 to 7 August 2026 that produced Chapter 4. All were taken from the running testbed with the defence armed except where a disarmed contrast is stated. The short outputs appear inline in the chapters; the long ones are collected here in full.

**Code and data availability.** The complete system, every evaluation harness and the raw result files are published at the project GitHub repository. The excerpts reproduced in this appendix are the load-bearing lines of larger files; the full versions, and the captured data behind every chart and table, are in the repository at the paths given in Table A.1. Each path below is a live link to its file in the repository.

| Repository path | Role in this dissertation |
|---|---|
| 06_Build/cars_engine.py | The controller: `classify()`, criticality elevation, `select_response()`, GUARD binding, reactive install (cookie `0x00ca`, `30+15w`), authenticated API |
| 06_Build/snort_bridge.py | Snort alert to `/cars/respond` detection bridge |
| 06_Build/cars_flow_audit.py | Flow-integrity poll against the trusted baseline |
| 06_Build/cars_remediation.py | Process last-good restore agent |
| 06_Build/cars_process.py | Process / control-law reference model |
| 07_Evaluation/overnight/ | Conformance-at-scale, criticality behaviour, flow-integrity, jitter and the Gap 1/2/4 harnesses, with raw data under `results/` |
| gap4_flowmonitor/cars_flowmonitor.py | Event-driven flow-integrity monitor (Section 4.12) |
| report/ | This dissertation, its figures and its `appendix_listings/` data files |

Table A.1: Map from the report to the published repository (the project GitHub repository).

## A.1 Full OpenFlow tables

The complete three-table pipeline of Section 3.3, dumped from each switch with `ovs-ofctl -O OpenFlow13 dump-flows`. Table 0 is GUARD (identity binding and anti-spoof), Table 1 is POLICY (the stateful conntrack layer with the allowlist under cookie `0xa2` and the default-deny), and Table 2 is the L2 learning switch. These are the source for the pipeline in Figure 3.3.

```
### ovs1 table=0
 cookie=0x0, duration=20378.869s, table=0, n_packets=27920, n_bytes=3464030, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=20378.874s, table=0, n_packets=1598927, n_bytes=128822228, priority=200,ip,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,nw_src=192.168.2.10
       actions=goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=46876, n_bytes=5356173, priority=200,ip,in_port=enx9c69d331aef0,dl_src=e0:dc:a0:62:b7:4c,nw_src=192.168.2.9 actions=
       goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=673, n_bytes=40380, priority=200,arp,in_port=enx9c69d331d874,arp_spa=192.168.2.10,arp_sha=e0:dc:a0:63:98:09 actions=
       goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=2354, n_bytes=141240, priority=200,arp,in_port=enx9c69d331aef0,arp_spa=192.168.2.9,arp_sha=e0:dc:a0:62:b7:4c actions
       =goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=2795418, n_bytes=215929736, priority=150,in_port="patch-ovs1-gw" actions=goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.10 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.20 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.30 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.31 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.45 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.55 actions=drop
```

```
 cookie=0x0, duration=20378.873s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.9 actions=drop
 cookie=0x0, duration=20378.873s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.10 actions=drop
 cookie=0x0, duration=20378.873s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.20 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.30 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.31 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.45 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.55 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.9 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=50,ip actions=goto_table:1
 cookie=0x0, duration=20378.872s, table=0, n_packets=1, n_bytes=124, priority=0 actions=goto_table:1
### ovs1 table=1
 cookie=0xa2, duration=20378.892s, table=1, n_packets=4416490, n_bytes=345191112, priority=90,ct_state=-trk,ip actions=ct(table=1)
 cookie=0xa2, duration=20378.892s, table=1, n_packets=4394166, n_bytes=339589437, priority=85,ct_state=+est+trk,ip actions=goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=1, n_bytes=74, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.9,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.3.66,nw_dst=192.168.3.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=2, n_bytes=132, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.45,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.9,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.20 actions=drop
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.10 actions=drop
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.9 actions=drop
 cookie=0xa2, duration=20378.889s, table=1, n_packets=22311, n_bytes=4332105, priority=10,ct_state=+new+trk,ip actions=ct(commit),goto_table:2
 cookie=0x0, duration=20378.894s, table=1, n_packets=27759, n_bytes=5098769, priority=0 actions=goto_table:2
### ovs1 table=2
 cookie=0x0, duration=20378.914s, table=2, n_packets=2735, n_bytes=169833, priority=2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=20378.895s, table=2, n_packets=47206, n_bytes=5223109, priority=1,in_port=enx9c69d331aef0,dl_src=e0:dc:a0:62:b7:4c,dl_dst=e0:dc:a0:63:98:09 actions
       =output:enx9c69d331d874
 cookie=0x0, duration=20378.891s, table=2, n_packets=30961, n_bytes=3466370, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=e0:dc:a0:62:b7:4c actions
       =output:enx9c69d331aef0
 cookie=0x0, duration=20377.892s, table=2, n_packets=128863, n_bytes=8961760, priority=1,in_port="patch-ovs1-gw",dl_src=92:b7:80:63:54:56,dl_dst=e0:dc:a0:63:98:09
       actions=output:enx9c69d331d874
 cookie=0x0, duration=20377.883s, table=2, n_packets=71278, n_bytes=5712750, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=92:b7:80:63:54:56 actions
       =output:"patch-ovs1-gw"
 cookie=0x0, duration=20376.539s, table=2, n_packets=19569, n_bytes=1465943, priority=1,in_port="patch-ovs1-gw",dl_src=de:5a:28:ae:96:03,dl_dst=e0:dc:a0:63:98:09 actions
       =output:enx9c69d331d874
 cookie=0x0, duration=20376.534s, table=2, n_packets=10762, n_bytes=917479, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=de:5a:28:ae:96:03 actions=
       output:"patch-ovs1-gw"
 cookie=0x0, duration=20269.803s, table=2, n_packets=2022, n_bytes=121384, priority=1,in_port=enx9c69d331aef0,dl_src=e0:dc:a0:62:b7:4c,dl_dst=b4:e9:b8:a4:ce:46 actions=
       output:"patch-ovs1-gw"
 cookie=0x0, duration=20269.755s, table=2, n_packets=0, n_bytes=0, priority=1,in_port="patch-ovs1-gw",dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:62:b7:4c actions=output:
       enx9c69d331aef0
 cookie=0x0, duration=20268.710s, table=2, n_packets=1485928, n_bytes=117609981, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=b4:e9:b8:a4:ce:46
       actions=output:"patch-ovs1-gw"
 cookie=0x0, duration=20268.703s, table=2, n_packets=2600279, n_bytes=196266885, priority=1,in_port="patch-ovs1-gw",dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:63:98:09
       actions=output:enx9c69d331d874
 cookie=0x0, duration=20378.914s, table=2, n_packets=44646, n_bytes=9105763, priority=0 actions=CONTROLLER:65535
```

Listing A.1: Cell-1 switch `ovs1`, tables 0 to 2.

```
### ovsgw table=0
 cookie=0x0, duration=20414.028s, table=0, n_packets=19281, n_bytes=1455423, priority=200,ip,in_port=sup0,dl_src=de:5a:28:ae:96:03,nw_src=192.168.2.30 actions=goto_table
       :1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,ip,in_port=opr,dl_src=02:00:00:00:02:31,nw_src=192.168.2.31 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=129262, n_bytes=8990206, priority=200,ip,in_port=rem0ovs,dl_src=92:b7:80:63:54:56,nw_src=192.168.2.45 actions=
       goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,ip,in_port=mbplc,dl_src=02:00:00:00:02:20,nw_src=192.168.2.20 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=2627106, n_bytes=200945835, priority=200,ip,in_port=enx00e04c680018,dl_src=b4:e9:b8:a4:ce:46,nw_src=192.168.2.55
       actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=334, n_bytes=14028, priority=200,arp,in_port=sup0,arp_spa=192.168.2.30,arp_sha=de:5a:28:ae:96:03 actions=goto_table
       :1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,arp,in_port=opr,arp_spa=192.168.2.31,arp_sha=02:00:00:00:02:31 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=7, n_bytes=294, priority=200,arp,in_port=rem0ovs,arp_spa=192.168.2.45,arp_sha=92:b7:80:63:54:56 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,arp,in_port=mbplc,arp_spa=192.168.2.20,arp_sha=02:00:00:00:02:20 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=2027, n_bytes=121620, priority=200,arp,in_port=enx00e04c680018,arp_spa=192.168.2.55,arp_sha=b4:e9:b8:a4:ce:46
       actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=1593195, n_bytes=125806203, priority=150,in_port="patch-gw-ovs1" actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.10 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.20 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.30 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.31 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.45 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.55 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.9 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.10 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.20 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.30 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.31 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.45 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.55 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.9 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=29522, n_bytes=2349478, priority=50,ip actions=goto_table:1
 cookie=0x0, duration=20414.026s, table=0, n_packets=26072, n_bytes=5289125, priority=0 actions=goto_table:1
### ovsgw table=1
 cookie=0xa2, duration=20414.042s, table=1, n_packets=4337104, n_bytes=335271698, priority=90,ct_state=-trk,ip actions=ct(table=1)
 cookie=0xa2, duration=20414.042s, table=1, n_packets=4353311, n_bytes=333837310, priority=85,ct_state=+est+trk,ip actions=goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.9,nw_dst=192.168.2.10,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.3.66,nw_dst=192.168.3.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=2, n_bytes=132, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.45,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
```

```
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.9,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.20 actions=drop
 cookie=0xa2, duration=20414.041s, table=1, n_packets=22347, n_bytes=4339197, priority=10,ct_state=+new+trk,ip actions=ct(commit),goto_table:2
 cookie=0x0, duration=20414.044s, table=1, n_packets=50929, n_bytes=6773958, priority=0 actions=goto_table:2
### ovsgw table=2
 cookie=0x0, duration=20414.064s, table=2, n_packets=2741, n_bytes=170193, priority=2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=20413.048s, table=2, n_packets=129083, n_bytes=8977050, priority=1,in_port=rem0ovs,dl_src=92:b7:80:63:54:56,dl_dst=e0:dc:a0:63:98:09 actions=output
       :"patch-gw-ovs1"
 cookie=0x0, duration=20413.035s, table=2, n_packets=71400, n_bytes=5722528, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:63:98:09,dl_dst=92:b7:80:63:54:56 actions
       =output:rem0ovs
 cookie=0x0, duration=20411.696s, table=2, n_packets=19816, n_bytes=1477456, priority=1,in_port=ins2,dl_src=ba:f2:0f:34:89:a4,dl_dst=b4:e9:b8:a4:ce:5f actions=output:
       eth0
 cookie=0x0, duration=20411.694s, table=2, n_packets=19604, n_bytes=1468552, priority=1,in_port=sup0,dl_src=de:5a:28:ae:96:03,dl_dst=e0:dc:a0:63:98:09 actions=output:"
       patch-gw-ovs1"
 cookie=0x0, duration=20411.687s, table=2, n_packets=10780, n_bytes=919046, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:63:98:09,dl_dst=de:5a:28:ae:96:03 actions=
       output:sup0
 cookie=0x0, duration=20411.680s, table=2, n_packets=10768, n_bytes=921668, priority=1,in_port=eth0,dl_src=b4:e9:b8:a4:ce:5f,dl_dst=ba:f2:0f:34:89:a4 actions=output:ins2
 cookie=0x0, duration=20304.955s, table=2, n_packets=2026, n_bytes=121624, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:62:b7:4c,dl_dst=b4:e9:b8:a4:ce:46 actions=
       output:enx00e04c680018
 cookie=0x0, duration=20304.911s, table=2, n_packets=0, n_bytes=0, priority=1,in_port=enx00e04c680018,dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:62:b7:4c actions=output:"
       patch-gw-ovs1"
 cookie=0x0, duration=20303.862s, table=2, n_packets=1488511, n_bytes=117814329, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:63:98:09,dl_dst=b4:e9:b8:a4:ce:46
       actions=output:enx00e04c680018
 cookie=0x0, duration=20303.859s, table=2, n_packets=2604793, n_bytes=196607589, priority=1,in_port=enx00e04c680018,dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:63:98:09
       actions=output:"patch-gw-ovs1"
 cookie=0x0, duration=20414.064s, table=2, n_packets=67082, n_bytes=10751672, priority=0 actions=CONTROLLER:65535
```

Listing A.2: Gateway switch `ovsgw`, tables 0 to 2.

```
### ovs2 table=0
 cookie=0x0, duration=20884.122s, table=0, n_packets=8353, n_bytes=2330534, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=20884.124s, table=0, n_packets=37677, n_bytes=6254142, priority=200,ip,in_port=enx9c69d3413f16,dl_src=e0:dc:a0:46:ff:ce,nw_src=192.168.2.10 actions
       =goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=48038, n_bytes=7138971, priority=200,ip,in_port=enx9c69d3283cf9,dl_src=e0:dc:a0:5c:60:44,nw_src=192.168.2.9 actions=
       goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=947, n_bytes=56820, priority=200,arp,in_port=enx9c69d3413f16,arp_spa=192.168.2.10,arp_sha=e0:dc:a0:46:ff:ce actions=
       goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=341, n_bytes=20460, priority=200,arp,in_port=enx9c69d3283cf9,arp_spa=192.168.2.9,arp_sha=e0:dc:a0:5c:60:44 actions=
       goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.10 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.20 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.30 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.31 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.45 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.55 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.9 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.10 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.20 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.30 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.31 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.45 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.55 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.9 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=19718, n_bytes=1488405, priority=50,ip actions=goto_table:1
 cookie=0x0, duration=20884.123s, table=0, n_packets=622, n_bytes=26868, priority=0 actions=goto_table:1
### ovs2 table=1
 cookie=0xa2, duration=20884.141s, table=1, n_packets=85715, n_bytes=13393113, priority=90,ct_state=-trk,ip actions=ct(table=1)
 cookie=0xa2, duration=20884.141s, table=1, n_packets=105419, n_bytes=14713428, priority=85,ct_state=+est+trk,ip actions=goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=1, n_bytes=74, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.9,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.3.66,nw_dst=192.168.3.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.45,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.9,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.20 actions=drop
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.9 actions=drop
 cookie=0xa2, duration=20884.140s, table=1, n_packets=13, n_bytes=1066, priority=10,ct_state=+new+trk,ip actions=ct(commit),goto_table:2
 cookie=0x0, duration=20884.142s, table=1, n_packets=1910, n_bytes=104148, priority=0 actions=goto_table:2
### ovs2 table=2
 cookie=0x0, duration=20884.160s, table=2, n_packets=342, n_bytes=20502, priority=2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=20883.742s, table=2, n_packets=48377, n_bytes=7002655, priority=1,in_port=enx9c69d3283cf9,dl_src=e0:dc:a0:5c:60:44,dl_dst=e0:dc:a0:46:ff:ce actions
       =output:enx9c69d3413f16
 cookie=0x0, duration=20883.735s, table=2, n_packets=27217, n_bytes=5334395, priority=1,in_port=enx9c69d3413f16,dl_src=e0:dc:a0:46:ff:ce,dl_dst=e0:dc:a0:5c:60:44 actions
       =output:enx9c69d3283cf9
 cookie=0x0, duration=20879.689s, table=2, n_packets=20314, n_bytes=1513045, priority=1,in_port=cell2gw,dl_src=9e:e4:8a:2b:46:5d,dl_dst=e0:dc:a0:46:ff:ce actions=output:
       enx9c69d3413f16
 cookie=0x0, duration=20879.684s, table=2, n_packets=11063, n_bytes=945623, priority=1,in_port=enx9c69d3413f16,dl_src=e0:dc:a0:46:ff:ce,dl_dst=9e:e4:8a:2b:46:5d actions=
       output:cell2gw
 cookie=0x0, duration=20884.160s, table=2, n_packets=30, n_bytes=2496, priority=0 actions=CONTROLLER:65535
```

Listing A.3: Cell-2 switch `ovs2`, tables 0 to 2.

## A.2 Deployed decision logic

The rulebook as it is defined in the running `cars_engine.py`, backing Table 3.2. The ordering is significant: the engineering and remediation rows precede the dangerous-operations block, so their pre-authorised control of the PLC classifies as OPERATIONAL. This is the current deployed rulebook, in which `ews->plc` is OPERATIONAL.

```
98:RULEBOOK = [
99-    ("hmi",         "plc", "any",   "CRITICAL"),
100-    ("plc",         "hmi", "any",   "CRITICAL"),
101-    # F2: authorise the CARS remediation agent (.2.45) BEFORE the dangerous-ops block (first-match-wins),
102-    # so its last-good CONTROL restores classify OPERATIONAL, not FORBIDDEN. Mirrors runtime rulebook.json.
103-    ("remediation", "plc", "CONTROL", "OPERATIONAL"),
104-    ("remediation", "plc", "any",     "OPERATIONAL"),
105-    # CC-98: EWS/Factory-IO (.2.55) trusted process-I/O source (Approach A).
106-    ("ews", "plc", "READ",    "OPERATIONAL"),
107-    ("ews", "plc", "WRITE",   "OPERATIONAL"),
108-    ("ews", "plc", "CONTROL", "OPERATIONAL"),
109-    # CC-51: DANGEROUS operations FORBIDDEN regardless of source
110-    ("any", "plc", "CONTROL", "FORBIDDEN"),
111-    ("any", "hmi", "CONTROL", "FORBIDDEN"),
112-    ("any", "plc", "DIAG",    "FORBIDDEN"),
113-    ("any", "hmi", "DIAG",    "FORBIDDEN"),
114-    ("any", "plc", "PROGRAM", "FORBIDDEN"),
115-    ("any", "hmi", "PROGRAM", "FORBIDDEN"),
116-    ("any", "plc", "ILLEGAL", "FORBIDDEN"),
117-    ("any", "hmi", "ILLEGAL", "FORBIDDEN"),
118-    ("ews", "plc", "any", "OPERATIONAL"),
119-    ("ews",         "hmi", "any",   "SENSITIVE"),
120-    ("supervisory", "plc", "WRITE", "SENSITIVE"),
121-    ("historian",   "plc", "WRITE", "SENSITIVE"),
122-    ("scada",       "plc", "WRITE", "SENSITIVE"),
123-    ("supervisory", "hmi", "WRITE", "SENSITIVE"),
124-    ("historian",   "hmi", "WRITE", "SENSITIVE"),
125-    ("scada",       "hmi", "WRITE", "SENSITIVE"),
126-    ("supervisory", "plc", "any",   "OPERATIONAL"),
127-    ("historian",   "plc", "any",   "OPERATIONAL"),
128-    ("scada",       "plc", "any",   "OPERATIONAL"),
129-    ("supervisory", "hmi", "any",   "OPERATIONAL"),
130-    ("historian",   "hmi", "any",   "OPERATIONAL"),
131-    ("scada",       "hmi", "any",   "OPERATIONAL"),
132-    ("any",         "plc", "any",   "FORBIDDEN"),
133-    ("any",         "hmi", "any",   "FORBIDDEN"),
134-    ("any",         "any", "any",   "FORBIDDEN"),
135-]
```

Listing A.4: The deployed `RULEBOOK` (first match wins).

The proactive A2 policy read from `a2_policy.json` on the controller: the nine allowlisted conduits and the default-deny set that back the allowlist and default-deny rows of Table 1.

```
{
    "allowlist": [
        [
            "192.168.2.31",
            "192.168.2.20",
            6,
            502
        ],
        [
            "192.168.2.9",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.2.31",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.3.66",
            "192.168.3.10",
            6,
            102
        ],
        [
            "192.168.2.55",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.2.45",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.2.30",
            "192.168.2.10",
            6,
```

```
            102
        ],
        [
            "192.168.2.30",
            "192.168.2.20",
            6,
            502
        ],
        [
            "192.168.2.55",
            "192.168.2.9",
            6,
            102
        ]
    ],
    "default_deny": [
        [
            null,
            "192.168.2.20"
        ],
        [
            1,
            "192.168.2.10"
        ],
        [
            1,
            "192.168.2.9"
        ],
        [
            2,
            "192.168.2.9"
        ]
    ]
}
```

Listing A.5: The A2 allowlist and default-deny (`a2_policy.json`).

## A.3 Detection rules

The Snort rules that recover the industrial operation by deep packet inspection, backing Figures 3.5 and 3.3. Each rule maps an S7 or Modbus function to one of the operation classes CARS reasons about. The S7 rules are shown in their hardened form: after the fragmentation finding of Section 4.5.1, the operation match was re-anchored to the S7 job header with `distance`/`within` rather than a fixed packet offset, and TCP stream reassembly was enabled on the ICS ports (Listing A.32), so a PDU split across TCP segments is still recovered. The anchoring assumes the usual three-byte COTP between the `03 00` and `32 01` headers; a session negotiated with a non-standard COTP length would shift the match, a sensor-coverage residual carried in Section 4.12.

```
# --- A3 Modbus DPI: function-code detection (payload offset 7; proto-id anchor offset 2) ---
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coil";  content:"|00 00|"; offset:2; depth:2; content
     :"|05|"; offset:7; depth:1; sid:1000020; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-reg";   content:"|00 00|"; offset:2; depth:2; content
     :"|06|"; offset:7; depth:1; sid:1000021; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coils"; content:"|00 00|"; offset:2; depth:2; content:"|0F
     |"; offset:7; depth:1; sid:1000022; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-regs";  content:"|00 00|"; offset:2; depth:2; content
     :"|10|"; offset:7; depth:1; sid:1000023; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-READ-holding"; content:"|00 00|"; offset:2; depth:2; content
     :"|03|"; offset:7; depth:1; sid:1000024; rev:3;)
# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102). PDU-relative anchoring (Gap-2 reassembly fix,
     2026-08-07):
#     matched relative to the TPKT / S7-job header so a reassembled PDU is recognised wherever it sits in the stream.
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; content:"|72|"; distance:5;
     within:6; sid:1000040; rev:2;)
# CC-51 broadened ICS intelligence: dangerous Modbus ops
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-DIAG-diagnostics"; content:"|00 00|"; offset:2; depth:2; content
     :"|08|"; offset:7; depth:1; sid:1000025; rev:1;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-PROGRAM-mei"; content:"|00 00|"; offset:2; depth:2; content:"|2B
     |"; offset:7; depth:1; sid:1000026; rev:1;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-ILLEGAL-fc"; content:"|00 00|"; offset:2; depth:2; byte_test
     :1,>,43,7; sid:1000027; rev:1;)
# PD-3: classic S7comm Write-Var on real PLC1 (PDU-relative: match relative to the S7 job header, not a fixed offset)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|05|"; distance:8; within:9; sid:1000041; rev:2;)
# PD-7: classic S7comm PLC-control on real PLC1 -- STOP (0x29) / Control-start (0x28) => DIAG (dangerous)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|29|"; distance:8; within:9; sid:1000042; rev:2;)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|28|"; distance:8; within:9; sid:1000043; rev:2;)
# showcase: S7 read (func 0x04) => READ => OPERATIONAL (contrast vs write/stop)
```

```
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7;
      content:"|04|"; distance:8; within:9; sid:1000044; rev:2;)
# PLC2-2: Cell-2 target .3.10 S7 operation DPI (PDU-relative)
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|05|"; distance:8; within:9; sid:1000045; rev:2;)
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|29|"; distance:8; within:9; sid:1000046; rev:2;)
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|04|"; distance:8; within:9; sid:1000047; rev:2;)
# N2 (audit 07-24): Cell-2 symmetry -- S7 control-start (0x28) DPI
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|28|"; distance:8; within:9; sid:1000048; rev:2;)
```

Listing A.6: S7 and Modbus operation-class rules from `cars.rules`.

### A.3.1 Sensor coverage boundary

The operation-aware tier depends on what the Snort mirror carries, and that mirror taps the gateway switch alone. Figure A.1 illustrates the spatial consequence noted in Section 4.12: traffic that crosses `ovsgw` is inspected, but an attack that stays within a single cell, such as one from a foothold inside Cell-2 against `PLC2` on `ovs2`, never reaches the mirror and draws no operation-aware response. The proactive identity binding and default-deny run on every switch independently of the sensor, so an unregistered intra-cell source is still refused; the residual is the operation-aware tier for a compromised, already-allowlisted conduit. No intra-cell attacker was present in the evaluated set, so this is an architectural illustration rather than a measured attack, and per-cell sensing is the mitigation carried in the future work.

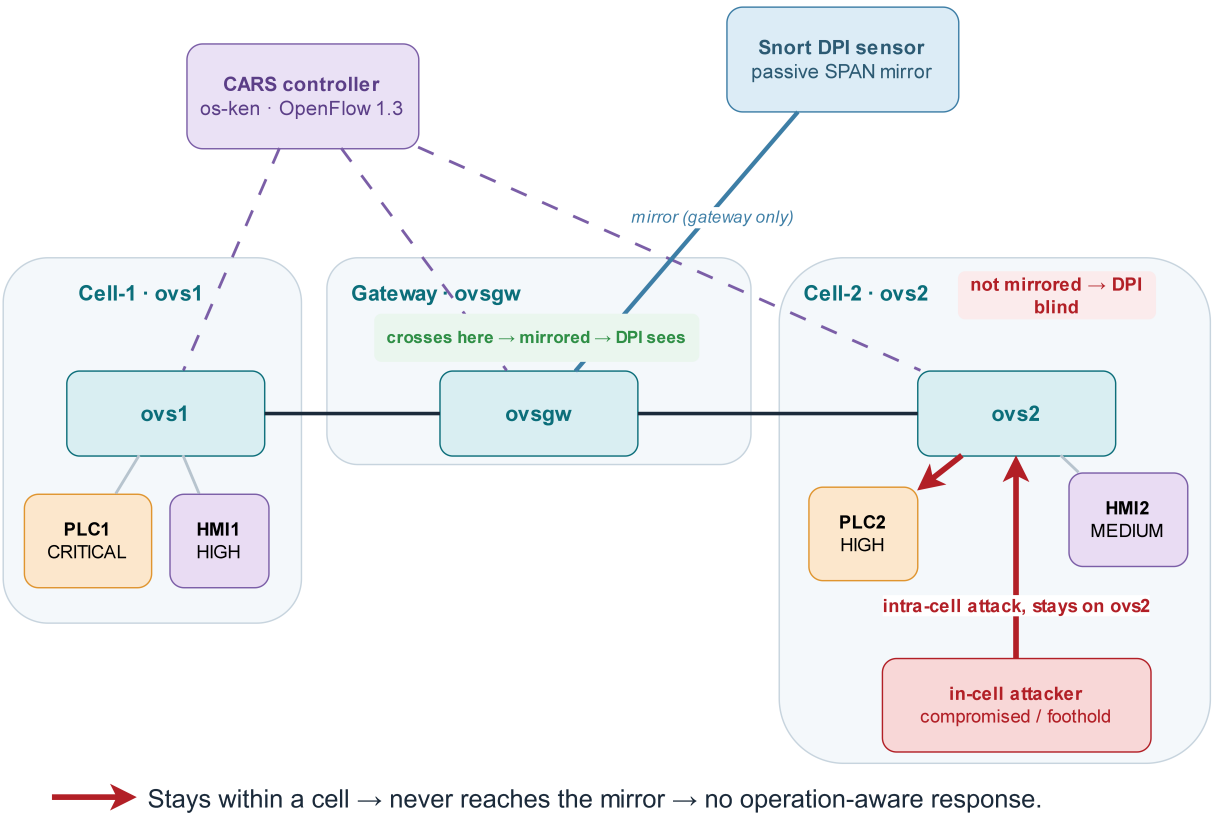


Figure A.1: The spatial coverage of the operation-aware tier. The Snort mirror taps `ovsgw` only: gateway-crossing traffic is inspected (green), while a purely intra-cell path (red), an in-cell foothold reaching `PLC2` over `ovs2`, never reaches the mirror and is not operation-aware. The proactive layer still applies on every switch, so an unregistered intra-cell source is refused regardless. This is an architectural illustration of a coverage boundary, not a tested attack.

## A.4 Runtime state

Two further live checks referenced from Section 3.5. Figure A.2 is the GUARD anti-spoof drop counter for every binding on every switch, all at zero under normal operation. Figure A.3 is the rate-limiting meter that backs the THROTTLE response.

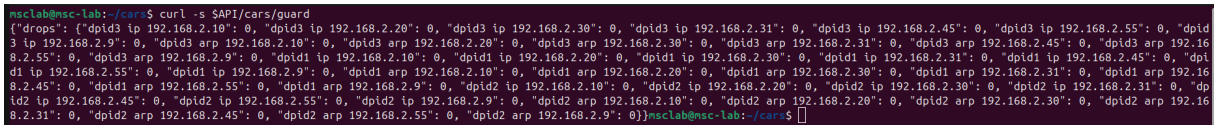


Figure A.2: GUARD anti-spoof drop counters from `curl $API/cars/guard`, per binding and per switch (all zero, armed and quiescent).

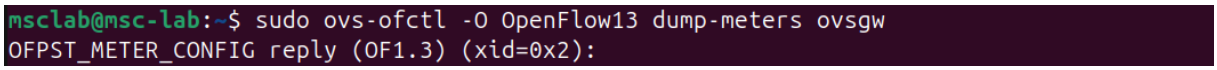


Figure A.3: The OpenFlow meter behind the THROTTLE response, from `ovs-ofctl dump-meters ovsgw`.

## A.5 Complete evaluation matrix

The full decision matrix of Section 4.2, all twenty-seven cases with the columns the body table abbreviates (source, destination, operation, rate, ground-truth label, tier, response, verdict, note), regenerated from the deployed engine via `/cars/respond`.

```
src,dst,op,rate,label,tier,response,verdict,note
192.168.2.9,192.168.2.10,READ,0,legit,CRITICAL,REFUSE,TN,HMI->PLC1 read (operator loop)
192.168.2.9,192.168.2.10,WRITE,0,legit,CRITICAL,REFUSE,TN,HMI->PLC1 setpoint (loop / safety-invariant)
192.168.2.10,192.168.2.9,READ,0,legit,CRITICAL,REFUSE,TN,PLC1->HMI reply (loop)
192.168.2.55,192.168.2.10,READ,0,legit,OPERATIONAL,ALLOW,TN,EWS/FactoryIO output read (HIL)
192.168.2.55,192.168.2.10,CONTROL,0,legit,OPERATIONAL,ALLOW,TN,EWS/FactoryIO input-image write (HIL)
192.168.2.45,192.168.2.10,CONTROL,0,legit,OPERATIONAL,ALLOW,TN,Remediation last-good restore
192.168.2.30,192.168.2.10,READ,0,legit,OPERATIONAL,ALLOW,TN,Historian telemetry poll PLC1
192.168.2.30,192.168.2.20,READ,0,legit,OPERATIONAL,ALLOW,TN,Historian telemetry poll Modbus
192.168.2.31,192.168.2.20,READ,0,legit,OPERATIONAL,ALLOW,TN,SCADA Modbus poll
192.168.2.31,192.168.2.10,READ,0,legit,OPERATIONAL,ALLOW,TN,SCADA read PLC1 (reads never elevated)
192.168.2.31,192.168.2.10,WRITE,0,grey,FORBIDDEN,ISOLATE,grey,SCADA WRITE CRITICAL PLC1 -> elevate SENSITIVE->FORBIDDEN
192.168.2.31,192.168.2.20,WRITE,0,grey,SENSITIVE,THROTTLE,grey,SCADA WRITE LOW Modbus -> SENSITIVE (graded)
192.168.2.30,192.168.2.9,WRITE,0,grey,SENSITIVE,THROTTLE,grey,Historian WRITE HIGH HMI1 -> SENSITIVE (graded)
192.168.2.77,192.168.2.10,TCP,0,attack,FORBIDDEN,ISOLATE,TP,Kali outsider TCP->PLC1 (unregistered)
192.168.2.77,192.168.2.10,READ,0,attack,FORBIDDEN,ISOLATE,TP,Kali outsider read PLC1
192.168.2.66,192.168.2.20,WRITE,0,attack,FORBIDDEN,BLOCK,TP,Unknown attacker write Modbus
192.168.2.31,192.168.2.10,CONTROL,0,attack,FORBIDDEN,ISOLATE,TP,Compromised SCADA forces PLC1 actuator
192.168.2.31,192.168.2.10,PROGRAM,0,attack,FORBIDDEN,ISOLATE,TP,SCADA unauthorized program download
192.168.2.55,192.168.2.10,PROGRAM,0,attack,FORBIDDEN,ISOLATE,TP,EWS unauthorized program download (no window)
192.168.2.55,192.168.2.10,DIAG,0,attack,FORBIDDEN,ISOLATE,TP,EWS unauthorized diagnostics
192.168.2.55,192.168.2.10,ILLEGAL,0,attack,FORBIDDEN,ISOLATE,TP,Malformed/illegal S7 to PLC1
192.168.2.1,192.168.2.10,READ,0,attack,FORBIDDEN,ISOLATE,TP,Gateway/DMZ reaches CRITICAL PLC (no conduit)
192.168.2.30,192.168.2.10,CONTROL,0,attack,FORBIDDEN,ISOLATE,TP,Historian (read-only) issues CONTROL
192.168.2.66,192.168.2.10,READ,0,attack,FORBIDDEN,ISOLATE,TP,Unknown attacker read CRITICAL PLC1
192.168.2.30,192.168.2.10,READ,50,attack,OPERATIONAL,BLOCK,TP,"Historian READ flood (volumetric DoS, CRITICAL)"
192.168.2.55,192.168.2.10,CONTROL,50,flood_exempt,OPERATIONAL,ALLOW,TN,EWS/FactoryIO high-rate CONTROL (FLOOD_EXEMPT)
192.168.2.31,192.168.2.20,READ,50,attack,OPERATIONAL,THROTTLE,TP,SCADA READ flood on Modbus
```

Listing A.7: The evaluation matrix, `cars_eval_matrix.csv`.

## A.6 Decision logs and events

The controller logs every decision it takes, so the conformance result rests on thousands of live judgements, not only the scored matrix. Listing A.8 is a representative extract spanning the campaign: the reconnaissance isolate, the identity-spoof refusal, the mean-time-to-mitigate isolate, the operation-aware isolate, the flow-integrity drift, and two legitimate operations permitted. The complete per-run logs are the exported CSV artefacts (each run logs on the order of a thousand decisions).

```
time,src,srole,dst,drole,proto,op,tier,resp,mode,action
08-05T21:01:49,192.168.2.77,unknown,192.168.2.10,plc,TCP,,FORBIDDEN,ISOLATE,ENFORCED,"ISOLATE source 192.168.2.77 75s (
     quarantine all conduits, self-healing) [CRIT:CRITICAL]"
08-05T21:03:14,192.168.2.31,SPOOFED,0.0.0.0,guard,ARP,SPOOF,FORBIDDEN,REFUSE,ENFORCED,"REFUSED (identity-spoof) -
     192.168.2.31 impersonation dropped at GUARD ingress (dpid3) [x2, total 10]"
08-05T21:05:02,192.168.2.66,unknown,192.168.2.10,plc,ICMP,,FORBIDDEN,ISOLATE,ENFORCED,"ISOLATE source 192.168.2.66 75s (
     quarantine all conduits, self-healing) [CRIT:CRITICAL]"
08-05T21:05:11,192.168.2.55,ews,192.168.2.10,plc,S7,CONTROL,OPERATIONAL,ALLOW,ENFORCED,ALLOW (operational) - monitor only
     [CRIT:CRITICAL]
08-05T21:04:59,192.168.2.30,historian,192.168.2.10,plc,S7,READ,OPERATIONAL,ALLOW,ENFORCED,ALLOW (operational) - monitor
     only [CRIT:CRITICAL]
08-05T19:15:38,192.168.2.31,scada,192.168.2.10,plc,S7,CONTROL,FORBIDDEN,ISOLATE,ENFORCED,"ISOLATE source 192.168.2.31 75s
     (quarantine all conduits, self-healing) [CRIT:CRITICAL]"
08-05T18:21:49,0.0.0.0,flowaudit,0.0.0.0,policy,OF,DRIFT,FORBIDDEN,REFUSE,ENFORCED,"REFUSED (flow-integrity:policy-removed
     ) - ovs1|t1|p80|c0xa2|priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502"
```

Listing A.8: Representative controller decisions across the campaign (extract of the decision log; times `08-05`).

## A.7 Packet-level and wire-level analysis

The wire evidence is dissected directly from the captured pcaps. On the S7 side, the attacker's Write-Var frames to PLC1 were counted at the two synchronised capture points, the Snort mirror and the PLC's physical port, armed and disarmed, giving the drop-on-the-wire result of Table 4.6: eight write attempts seen by the detector, one reaching the PLC when armed against seven when disarmed.

On the Modbus side, the operation-awareness discriminator is visible in the frame bytes. The same conduit `192.168.2.31` → `192.168.2.20:502` carried a read and a write, distinguished only by the function code at offset seven of the payload:

```
READ  (-> ALLOW)     MBAP+PDU: 00 01 00 00 00 06 00 03 00 00 00 04
                     tid=0001 proto=0000 len=0006 unit=00 FC=03  (read holding)
WRITE (-> THROTTLE)  MBAP+PDU: 00 01 00 00 00 06 00 06 00 04 00 07
                     tid=0001 proto=0000 len=0006 unit=00 FC=06  (write register)
```

Listing A.9: Modbus read and write on the same conduit; the function code sits at offset 7.

CARS allowed the FC 03 read and throttled the FC 06 write on that single byte. The dangerous function codes (`0x05` control, `0x08` diagnostic, `0x2B` program, `0x64` illegal) and the reliability with which the sensor detects each are given in Table 4.7. Figure A.4 shows the two Modbus function-code fields as Wireshark dissects them.

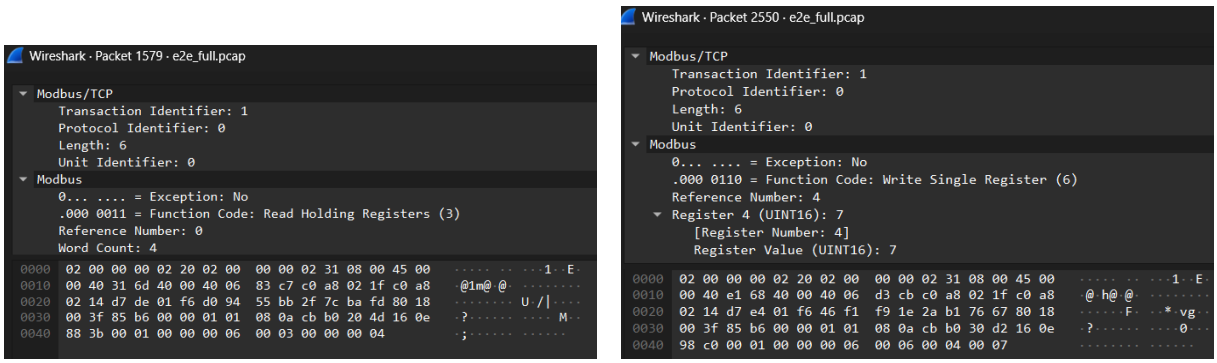


Figure A.4: The Modbus operation on the wire, same conduit `.31→.20`. Left: `Function Code: Read Holding Registers (3)`, permitted. Right: `Function Code: Write Single Register (6)` setting register 4 to 7, throttled. The verdict turns on this field.

Two further S7 views complete the picture. Figure A.5 shows the attacker's write reaching the PLC with CARS disarmed (contrast the single armed write of Figure 4.6). Figure A.6 shows that lone attacker frame amongst the legitimate Factory IO loop traffic on the PLC port, the attack is a needle in the live process stream.

The enforcement itself is visible on a third wire, the OpenFlow control channel (TCP 6653), captured in `of_control.pcap`. Listing A.10 is the `OFPT_FLOW_MOD` that installs the reactive isolate, decoded from that capture: an ADD of the drop rule stamped with the CARS reactive cookie `0x00ca`, at priority 110 in the POLICY table, matching the attacker's source with a 75-second self-healing timeout. Two identical

messages are sent, one to `ovs1` and one to `ovsgw`, so the quarantine is installed fabric-wide from a single decision. This is the control-plane counterpart to the datapath dump of Figure 4.4: the rule seen being installed, not just seen in place.

```
OFPT_FLOW_MOD              (version 0x04, type 14, control channel tcp/6653)
  command      = ADD
  cookie       = 0x00000000000000ca     (CARS reactive marker; 0xa2 = static A2 policy)
  table_id     = 1                        (POLICY)
  priority     = 110                       (above the allowlist and default-deny)
  hard_timeout = 75 s                      (CRITICAL asset: 30 + 15*3, then self-heals)
  match        = eth_type=0x0800, ipv4_src=192.168.2.31
  instructions = APPLY_ACTIONS []          (no output action = drop)
-> sent to ovs1 and ovsgw from one controller decision (fabric-wide quarantine)
```

Listing A.10: The reactive isolate as an `OFPT_FLOW_MOD`, decoded from `of_control.pcap` (OpenFlow 1.3, tcp/6653).

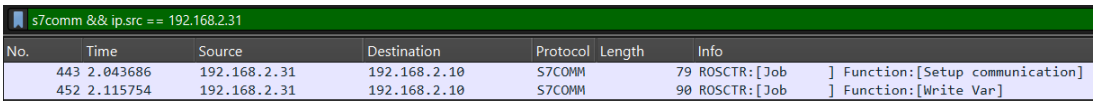


Figure A.5: Disarmed, the attacker's S7 Write Var (frame 452) reaches the PLC. With CARS armed the same write is dropped in the fabric (Figure 4.6).

| | | | | | | |
|---|---|---|---|---|---|---|
| 436 | 2.013042 | 192.168.2.55 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 437 | 2.026484 | 192.168.2.10 | 192.168.2.55 | S7COMM | 95 | ROSCTR:[Ack_Data] Function:[Read Var] |
| 441 | 2.044185 | 192.168.2.55 | 192.168.2.10 | S7COMM | 90 | ROSCTR:[Job ] Function:[Write Var] |
| 442 | 2.055486 | 192.168.2.10 | 192.168.2.55 | S7COMM | 76 | ROSCTR:[Ack_Data] Function:[Write Var] |
| 444 | 2.056223 | 192.168.2.55 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 449 | 2.071397 | 192.168.2.10 | 192.168.2.55 | S7COMM | 80 | ROSCTR:[Ack_Data] Function:[Read Var] |
| 451 | 2.072077 | 192.168.2.55 | 192.168.2.10 | S7COMM | 101 | ROSCTR:[Job ] Function:[Write Var] |
| 454 | 2.079629 | 192.168.2.31 | 192.168.2.10 | S7COMM | 79 | ROSCTR:[Job ] Function:[Setup communication] |
| 455 | 2.132453 | 192.168.2.10 | 192.168.2.55 | S7COMM | 76 | ROSCTR:[Ack_Data] Function:[Write Var] |
| 457 | 2.133267 | 192.168.2.55 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 458 | 2.135428 | 192.168.2.10 | 192.168.2.31 | S7COMM | 81 | ROSCTR:[Ack_Data] Function:[Setup communication] |
| 459 | 2.135670 | 192.168.2.31 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 462 | 2.156451 | 192.168.2.10 | 192.168.2.31 | S7COMM | 80 | ROSCTR:[Ack_Data] Function:[Read Var] |
| 465 | 2.161403 | 192.168.2.10 | 192.168.2.55 | S7COMM | 95 | ROSCTR:[Ack_Data] Function:[Read Var] |
| 469 | 2.178200 | 192.168.2.55 | 192.168.2.10 | S7COMM | 90 | ROSCTR:[Job ] Function:[Write Var] |
| 470 | 2.194432 | 192.168.2.10 | 192.168.2.55 | S7COMM | 76 | ROSCTR:[Ack_Data] Function:[Write Var] |
| 472 | 2.195052 | 192.168.2.55 | 192.168.2.10 | S7COMM | 85 | ROSCTR:[Job ] Function:[Read Var] |
| 473 | 2.207409 | 192.168.2.10 | 192.168.2.55 | S7COMM | 80 | ROSCTR:[Ack_Data] Function:[Read Var] |

Figure A.6: The attacker's Setup frame (`.31`→`.10`, highlighted) amongst the legitimate `.55` Factory IO loop traffic on the PLC port. CARS must pick the malicious operation out of a continuous stream of legitimate S7 reads and writes.

## A.8 Scenario testing in full

The process devastation of Section 4.4.1 has a second, quieter variant: instead of driving the tank visibly over the top, the attacker pins the reported level near zero, so the operator panel shows a safe, empty tank while the real one fills. Figure A.7 captures this deception from a disarmed run: the real Factory IO tank stands filled while the HMI reads a level of zero and a pump of `+0.000`.

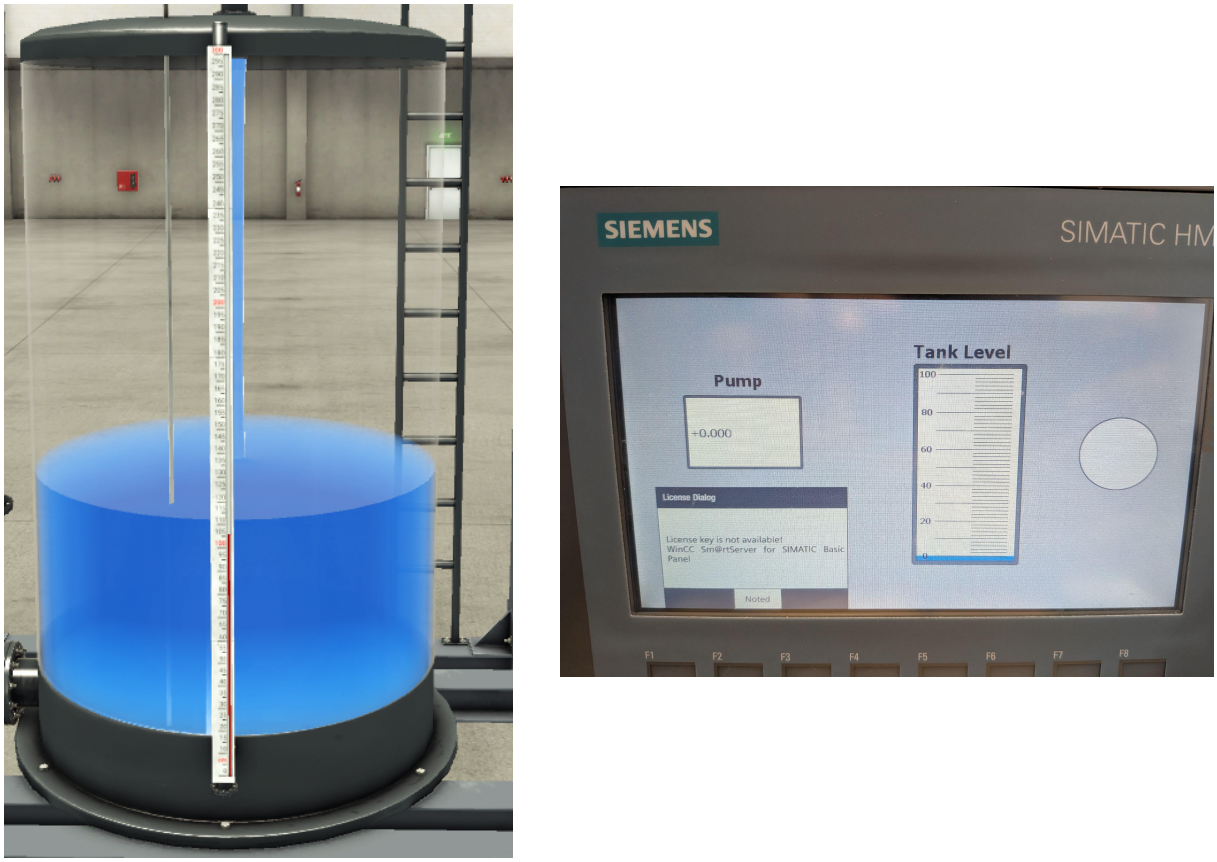


Figure A.7: The sensor-spoof deception, disarmed. Left: the real Factory IO tank, filled. Right: the operator HMI at the same moment, reading zero. The reported process has been divorced from the real one.

The external vectors were driven from the Kali attacker `192.168.2.77`. Reconnaissance under CARS is shown in Figure A.8: an armed port scan of the PLC returns every port `filtered`, and the scan is reactively isolated with a criticality-scaled quarantine. Identity spoofing is shown in Figure A.9: the GUARD anti-spoof drop counters register the forged ARP frames dropped at ingress. The reaction window over one hundred autonomous trials is in Figure A.10.

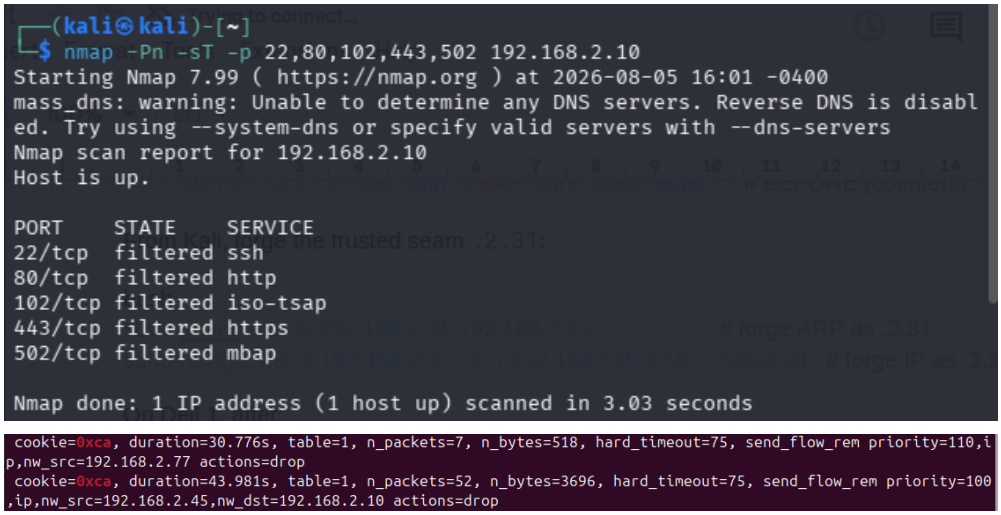


Figure A.8: Reconnaissance from Kali, armed. Top: the port scan of PLC1, every port `filtered`. Bottom: the reactive rule installed against the scanner, `cookie=0xca`, `priority=110`, `nw_src=192.168.2.77`, `actions=drop`, `hard_timeout=75`.

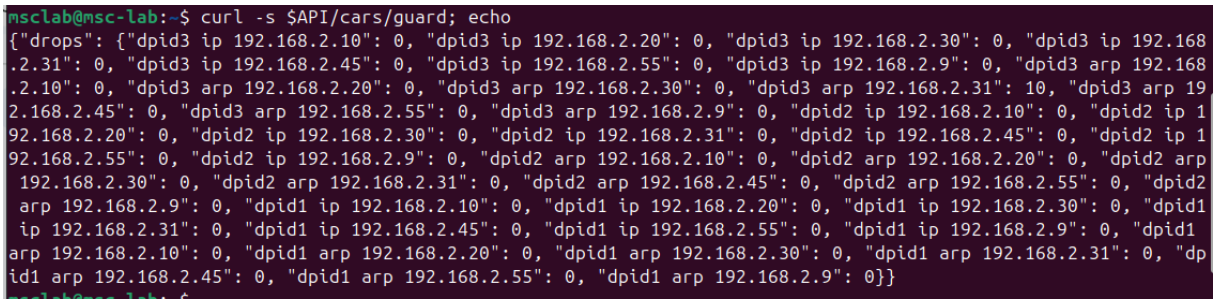


Figure A.9: Identity spoofing at GUARD. After forging ARP as the trusted seam `192.168.2.31` from Kali, the anti-spoof counter `dpid3 arp 192.168.2.31` registers the dropped frames (running total 10), every forged frame dropped at ingress with all other bindings untouched. The controller logged each as `REFUSED (identity-spoof)`.

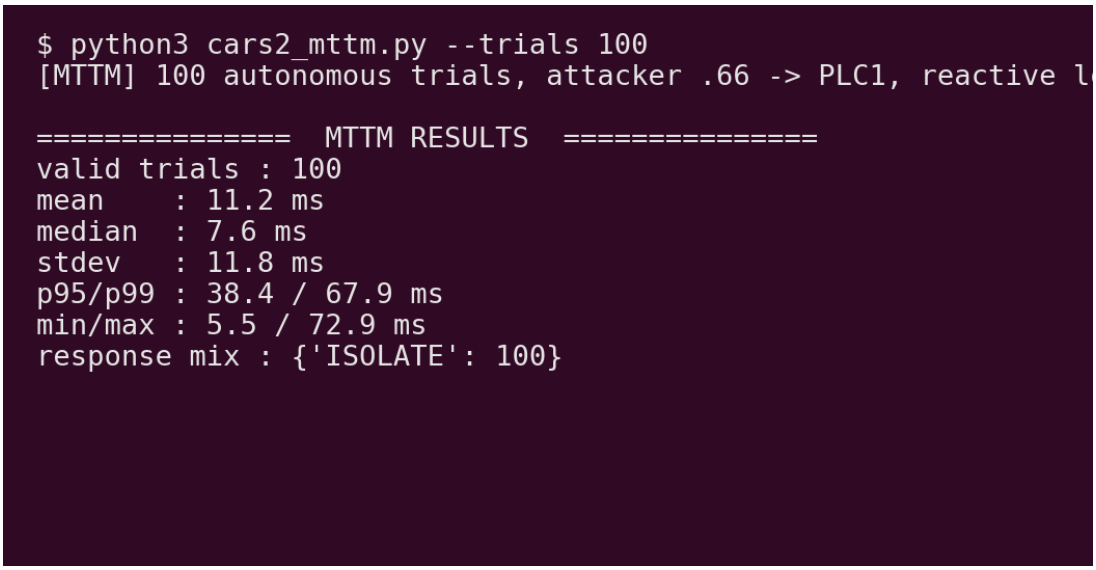


Figure A.10: Reaction window (mean-time-to-mitigate) over one hundred autonomous trials, attack-frame to enforcement on a single clock: median 7.6 ms, mean 11.2 ms, standard deviation 11.8 ms, minimum 5.5 ms, maximum 72.9 ms, every trial ending in ISOLATE.

## A.9 Scripts and code

Each script is presented as its role plus its load-bearing snippet. The lead is the controller, cars_engine.py: the whole evaluation drives its decision path, whose core is `classify()`, mapping a (source, destination, operation) to a tier by walking the rulebook first-match-wins.

```
def classify(src_ip, dst_ip, op=None):
    s, d = role_of(src_ip), role_of(dst_ip)      # map IP -> role from the REGISTRY
    for rsrc, rdst, rop, tier in RULEBOOK:        # rules in priority order
        if (rsrc == "any" or rsrc == s or rsrc == src_ip) and \
           (rdst == "any" or rdst == d or rdst == dst_ip) and \
           (rop  == "any" or rop  == op):
            return tier, s, d                     # first match wins -> its tier
    return "FORBIDDEN", s, d                      # nothing matched -> default deny
```

Listing A.11: The controller decision path (cars_engine.py): first-match rulebook classification.

The role lookup resolves each address to its registered role; the loop returns the tier of the first rulebook row whose source, destination and operation all match, and defaults to FORBIDDEN when none do. This is the exact path the conformance evaluation drives through `/cars/respond`. The remaining load-bearing pieces follow below in two forms: pieces that are pure decision or process logic are given as language-agnostic pseudocode, and pieces whose exact bytes, capture filters or detection rules are themselves the evidence, the protocol frames, the wire captures and the Snort rules, are shown as the deployed excerpt. Each caption links to the real source file.

### Controller: the rest of the decision-to-enforcement path

**Criticality elevation and block duration (`respond()`).** After classification, the destination's criticality weight drives two things: a trusted actuating write (SENSITIVE) to a CRITICAL asset is elevated to FORBIDDEN unless an authorised maintenance window is open, and the quarantine duration is scaled by the same weight.

```
acl = crit_of(dst_ip); dcw = CW.get(acl, 0)          # dst asset weight: 3..0
in_window = time.time() < self.maint_until
elevated = (tier == "SENSITIVE" and dcw >= 3 and not in_window)
if elevated:
    tier = "FORBIDDEN"          # trusted WRITE to a CRITICAL asset -> forbidden
timeout = BLOCK_TIMEOUT + dcw * 15   # CRITICAL 75, HIGH 60, MED 45, LOW 30 s
```

Listing A.12: Grey-zone elevation and criticality-scaled block duration.

**The graded response ladder (`select_response()`).** A tier, the asset weight and a rate flag select one of the seven responses; the safety loop is never enforced against, and higher-criticality assets escalate sooner and skip rungs.

```
if tier == "CRITICAL":    return "REFUSE"        # safety invariant - never enforce
esc = max(1, ESCALATE - dcw); c = state.get("count", 1)
if tier == "OPERATIONAL":
    if flood:
        if dcw >= 3: return "BLOCK"              # flood on CRITICAL asset -> cut
        return "THROTTLE" if c < esc else "BLOCK"
    return "ALLOW"                               # trusted conduit, normal rate
if tier == "SENSITIVE":
    if flood:    return "BLOCK"
    if dcw >= 3: return "BLOCK"                  # CRITICAL asset: skip THROTTLE
    return "THROTTLE" if c < esc else "BLOCK"
if flood:    return "ISOLATE"                    # FORBIDDEN
if dcw >= 3: return "ISOLATE"                    # CRITICAL asset: isolate now
return "BLOCK" if src_count < esc else "ISOLATE"
```

Listing A.13: Criticality-graded, flood-aware response selection.

**The reactive install (`isolate_source()`).** Every enforcement is a real OpenFlow rule stamped with the distinct reactive cookie `0x00ca`, self-expiring via `hard_timeout` and notifying the controller on removal (`SEND_FLOW_REM`) so the defence self-heals when the attack stops. Isolation drops all traffic from the source across every switch.

```
def isolate_source(self, src_ip, timeout=BLOCK_TIMEOUT):
    for d, dp in list(self.datapaths.items()):
        ofp, psr = dp.ofproto, dp.ofproto_parser
        m = psr.OFPMatch(eth_type=0x0800, ipv4_src=src_ip)      # any dst
        dp.send_msg(psr.OFPFlowMod(datapath=dp, cookie=REACTIVE_COOKIE,
            table_id=1, priority=110, match=m, instructions=[],  # [] = drop
            hard_timeout=timeout, flags=ofp.OFPFF_SEND_FLOW_REM))
```

Listing A.14: Source quarantine: a self-healing drop on every switch (`REACTIVE_COOKIE = 0x00CA`).

**Anti-spoof GUARD (`install_guard()`).** Table 0 binds each protected identity to its physical port and MAC; a frame carrying a protected source address that does not arrive on its bound port is dropped before it can reach policy. This is prevention, not detection.

```
for (d, port, mac, ip) in BINDINGS:                # bind identity to port+MAC
    if d == dp.id:
        add(200, OFPMatch(in_port=port, eth_type=0x0800,
                          eth_src=mac, ipv4_src=ip), GOTO)   # legit -> policy
if GUARD_ENABLED:
    for ip in PROTECTED_IPS:
        add(100, OFPMatch(eth_type=0x0800, ipv4_src=ip), [])  # forged src -> drop
add(0, OFPMatch(), GOTO)                            # default -> policy table
```

Listing A.15: Table 0 identity binding and anti-spoof drop.

**Control-API authentication (`_app()`).** Every state-changing endpoint requires a shared token; the read-only status endpoints and the Snort detection feed are excluded so the bridge is unchanged. Leaving the `/cars/respond` feed unauthenticated means a host able to reach it could forge an alert and drive a reactive isolation of its choosing; that exposure, bounded here by the management-plane isolation and closed in a routable deployment by an authenticated or HMAC-signed feed, is analysed in Section 4.12.

```
CONTROL_ENDPOINTS <- { defense, maintenance, reload, reload-a2, block, unblock, restore }
on a request to ‘path‘:
    if path in CONTROL_ENDPOINTS and request token != the shared API token:
        reply 401 Unauthorized              // a state-changing call must carry the token
    // the read-only status endpoints and the /cars/respond sensor feed are excluded
```

Listing A.16: Token gate on the control endpoints, in pseudocode.

## The supporting scripts

**Detection bridge (`snort_bridge.py`).** Snort raises the alerts; the bridge recovers the ICS operation from the alert text, measures its arrival rate, and posts the source, destination, operation and rate to the controller, which alone decides. The brain, not the sensor, holds the policy.

```
for each new Snort alert line:
    match the alert tag  CARS-<proto>-<operation>      // proto in {S7, MODBUS}
    if it matches:
        proto, operation <- the tag’s fields            // recover the industrial operation
        key  <- (source, dest, operation)               // keep read and write distinct
        rate <- arrival rate of this key over the window
        post (source, dest, operation, rate) to the controller   // the brain decides; the bridge
      only reports
```

Listing A.17: Operation recovery from the Snort alert, forwarded to the controller, in pseudocode.

**Flow-integrity self-check (`cars_flow_audit.py`).** A trusted baseline of the immutable policy (Table 0 GUARD and the Table 1 A2 allowlist, cookie `0xa2`) is captured at a clean start; the only additions permitted at runtime are the reactive rules (cookie `0xca`). Anything else appearing on a switch is flagged as drift and surfaced into the controller's decision log.

```
// a rule’s identity = (switch, table, priority, cookie, match) -- not its actions,
// so a tampered action on a policy rule shows as "changed", a bogus rule as "extra".
baseline <- the GUARD rules (table 0) and the allowlist (table 1, cookie 0xa2), captured at a clean
      start
every interval:
    live <- the rules currently installed on each switch
    for each live rule NOT carrying the reactive cookie 0xca:
        if its identity is not in baseline:    report DRIFT (extra or changed)
    for each baseline rule:
        if it is missing from live:            report DRIFT (missing)
    // the only legitimate live additions are the reactive 0xca rules; anything else is audited
```

Listing A.18: Flow identity and the drift model, in pseudocode.

**Process-state remediation (`cars_remediation.py`).** The novel maintain half of block-and-maintain: it detects sensor tampering by process anomaly (a level the bang-bang control law physically cannot produce) and writes the last-good value back over S7, holding the process while the source is quarantined.

```
last_good <- seed level
loop forever:
    level <- read Tank.Level over S7
    if level is tampered (a value the bang-bang control law cannot produce):
        write last_good back to the PLC over S7          // restore the last healthy value
        record a RESTORED event (level, last_good)
    else:
        last_good <- level                                // healthy: advance the last-good value
```

Listing A.19: Anomaly-triggered last-good restore over S7, in pseudocode.

**Latency harness (cars2_mttm.py).** Times the reaction window on a single clock: start the attack, poll for the isolate to appear in the controller's block set, and record attack-frame-to-enforcement.

```
t0 <- now
launch the attack
while not blocked and (now - t0) < 15 s:  wait briefly     // poll for the isolate to appear
mttm <- now - t0                                           // attack frame -> enforcement, on one
      clock
```

Listing A.20: Single-clock mean-time-to-mitigate measurement, in pseudocode.

**Attack tool (cars_fdi_overflow.py).** The false-data injection used in Section 4.4: a compromised host opens an S7 session to the PLC and writes the output area at high rate, blinding the level sensor and jamming the discharge valve. Each write is a CONTROL operation to the output area, which CARS recovers and forbids.

```
c = snap7.client.Client(); c.connect(HOST, 0, 1)   # HOST = PLC 192.168.2.10
zero = bytearray(struct.pack('>f', 0.0))
while time.time() < end:
    c.write_area(PE, 0, 100, zero)   # blind LevelIn %ID100 = 0 (sensor OB30 trusts)
    c.write_area(PA, 0, 104, zero)   # jam DischargeValve %QD104 = 0 (output area 0x82)
```

Listing A.21: The FDI overflow injection (writes to the S7 output area).

## Evaluation and proof harnesses

**Conformance harness (cars_eval.py).** Drives the labelled case set of Section 4.2 through the deployed engine over `/cars/respond`, scores each answer against its intent label, and writes `cars_eval_matrix.csv`. It disarms for the decision-only pass so nothing is enforced on the live conduits, then re-arms.

```
disarm enforcement                              // decision-only pass; re-armed at the end
for each case (source, dest, operation, rate, label):
    response <- ask the deployed engine  (/cars/respond)
    permit   <- response in { ALLOW, MONITOR, REFUSE }
    if label in { legit, flood-exempt }:
        verdict <- permit ? TN : FP             // restricting legitimate work is a false positive
    else if label = attack:
        verdict <- permit ? FN : TP             // permitting an attack is a false negative
    else:
        verdict <- grey                         // grey rows are shown, not scored
```

Listing A.22: Driving each case through the engine and scoring it against intent, in pseudocode.

**Criticality sweep (cars_criticality_proof.sh).** Holds the actor and operation fixed and varies only the target's tier. Part 1 runs disarmed to show the bounded elevation as a pure decision; Part 2 runs armed to measure the tier-scaled quarantine ($30 + 15w$ seconds), healing the reactive rule after each fire.

```
fire(){ curl -s -XPOST $API/respond -H 'Content-Type: application/json' \
       -d "{\"src\":\"$2\",\"dst\":\"$3\",\"proto\":\"$4\",\"op\":\"$5\",\"dpid\":3}"; }
heal(){ for sw in ovsgw ovs1; do sudo ovs-ofctl -O OpenFlow13 --strict del-flows "$sw" "table=1,priority=110"; done; }
arm false     # PART 1 - decision side: bounded elevation, nothing enforced
arm true      # PART 2 - response side: real quarantine, hard_timeout = 30 + cw*15 s
```

Listing A.23: Fire a controlled case, and heal the tier-scaled reactive rule between fires.

**Campaign library (cars_campaign_lib.sh).** The shared helpers every campaign and pivot run sources: arm/disarm through the authenticated API, a cross-layer `snap` of the live state, and `capstart` which opens the three synchronised wire captures.

```
arm(){    curl -s -X POST $API/cars/defense -H "X-CARS-Token: $TOKEN" -d '{"on":true}';  }
disarm(){ curl -s -X POST $API/cars/defense -H "X-CARS-Token: $TOKEN" -d '{"on":false}'; }
capstart(){ sudo tcpdump -i "$PLCIF" -w "$d/plc1_wire.pcap" $filt &     # what reaches the PLC
            sudo tcpdump -i any     -w "$d/of_control.pcap" 'tcp port 6653' &  # the OpenFlow flow_mods
            sudo tcpdump -i snort0  -w "$d/dpi_mirror.pcap" & }          # what the detector sees
```

Listing A.24: Arm/disarm and the three-point capture used across the campaign.

**Wire campaign (cars_wire_campaign.sh, cars_wire_campaign_disarmed.sh).** Runs each attack vector against the armed system (and its disarmed twin), capturing the three wire points and timestamp-aligned state from every layer. The first vector is the control-plane disarm attempt.

```
sudo timeout 240 tcpdump -i "$PLCIF" -w "$D/plc1_wire.pcap" 'host 192.168.2.10 and tcp port 102' &
sudo timeout 240 tcpdump -i any     -w "$D/of_control.pcap" 'tcp port 6653' &
sudo timeout 240 tcpdump -i snort0  -w "$D/dpi_mirror.pcap" &
# V1 CONTROL-PLANE: unauth POST /cars/defense {on:false}  -> expect HTTP 401 + DENIED audit
```

Listing A.25: The three capture points and the control-plane vector.

**Deep end-to-end trial (cars_e2e.sh).** Drives the full response spectrum through the real autonomous chain and records, per scenario, the four independent proofs behind Table 4.5: the frame on the mirror, the Snort alert, the controller decision, and the datapath enforcement.

```
wirecount(){ tshark -r "$OUT/pcap/e2e_full.pcap" -Y "$1" | wc -l; }        # WIRE   : frames on the mirror
pkts(){ ovs-ofctl -O OpenFlow13 dump-flows "$1" table=1 | grep -m1 "$2" \
        | grep -oP 'n_packets=\K[0-9]+'; }                                  # OVS    : enforcement counter moved
# SNORT : delta on /var/log/snort/alert ;  CARS : tier+response in /cars/audit
```

Listing A.26: The four independent ground-truth layers per scenario.

**Packet-drop proof (cars_packet_proof.sh).** Captures the attacker's S7 frames at two synchronised points, the Snort mirror and the PLC's physical port, armed then disarmed, giving the drop-on-the-wire counts of Table 4.6.

```
capture(){   # tag seconds
  sudo timeout "$2" tcpdump -i snort0          "host $PLC1 and tcp port 102" -w "$D/$1_mirror.pcap" &  # DPI sees
  sudo timeout "$2" tcpdump -i enx9c69d331d874 "host $PLC1 and tcp port 102" -w "$D/$1_plc1.pcap"  & } # reaches PLC
arm true; heal; capture armed 15      # then: arm false; capture disarmed 15
```

Listing A.27: Two synchronised capture points: what the detector sees versus what reaches the PLC.

**ICS operation battery (cars_ics_battery.sh).** Fires the full ICS operation spectrum from one trusted operator (.31) at the Modbus unit (.20) and shows the three verdicts the one conduit draws, backing Table 4.7.

```
function probe(operation):
    restore the conduit                    // clear any prior reactive rule
    mark the current end of the decision log
    run ‘operation‘ from the trusted operator
    wait for the new decision to appear
// expected: READ -> OPERATIONAL/ALLOW ; WRITE -> SENSITIVE/THROTTLE ;
//           CONTROL, DIAG, PROGRAM, ILLEGAL -> FORBIDDEN/BLOCK
```

Listing A.28: Fire each operation on the same conduit and await the new decision, in pseudocode.

## Attack clients and the process model

**S7 attack client (s7_write.py).** The S7 tool used across the evaluation. Its output-area write is an actuation CARS classifies as CONTROL; the `--dbspoof` mode pins a false level (the sensor-spoof deception), and further modes cover monitoring, volumetric read floods and a PLC stop.

```
def w(v): c.write_area(PA, 0, 0, bytearray([v & 0xFF]))    # write Q0.x output image (area 0x82) = actuation
# -{}-dbspoof: pin a false level, deceiving both the control loop and the HMI:
payload = struct.pack('>f', a.spoofval)                    # S7 REAL = big-endian 32-bit float
c.db_write(a.db, a.offset, payload)                        # write-var 0x05 -> CARS classifies CONTROL
```

Listing A.29: The output-area write (actuation) and the sensor-spoof variant.

**Modbus attack client (mb_attack.py).** Crafts arbitrary Modbus/TCP function codes, including dangerous and illegal ones, to prove CARS decides on the operation and not the five-tuple. The function code sits at offset seven, exactly where the Snort DPI anchors.

```
def frame(fc, data=b'', tid=1, unit=1):
    pdu  = bytes([fc & 0xFF]) + data
    mbap = struct.pack('>HHHB', tid, 0x0000, len(pdu)+1, unit)   # proto-id 0x0000 @off2 ; FC @off7
    return mbap + pdu
ATTACKS = {'coil':0x05, 'diag':0x08, 'program':0x2B, 'illegal':0x64}  # CONTROL / DIAG / PROGRAM / ILLEGAL
```

Listing A.30: Crafting an arbitrary Modbus function code (FC at offset 7).

**Process model (cars_process.py).** The soft co-simulation of the water-tank bang-bang loop, driving the real Q0.3 relay on PLC1 over S7 from the authorised controller identity. It reasserts the actuator every cycle and reports readback-versus-intent mismatches, which are the moments an attacker's write momentarily reached the relay.

```
each cycle:
    if level <= LOW:        pump <- on             // control law (eq.1)
    else if level >= HIGH:  pump <- off
    level <- level + (FILL if pump else -DRAIN)    // tank dynamics (eq.2)
    write the pump relay Q0.3 = pump               // actuate and re-assert every cycle
    got <- read the relay back
    if got != pump:  interference <- interference + 1   // readback != intent: a foreign write
      reached the relay
```

Listing A.31: The bang-bang control law, the dynamics, and the interference health metric, in pseudocode.

## Gap mitigations: DPI hardening and the process guardian

**DPI reassembly hardening (cars.conf, cars.rules).** The single-packet S7 rules were defeated by TCP fragmentation (Section 4.5.1). Two changes closed it: stream reassembly on the ICS ports, and re-anchoring the operation match to the S7 job header instead of a fixed packet offset.

```
# cars.conf : reassemble the ICS conduits before inspection
preprocessor stream5_global: track_tcp yes, track_udp no, max_tcp 8192
preprocessor stream5_tcp:    policy linux, timeout 60, ports both 102 502
# cars.rules S7 Write-Var, before (fixed offset) -> after (PDU-relative):
#  before: content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth:1;
#  after : content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
```

Listing A.32: Stream reassembly, and the offset-to-relative rule change.

**Fragmentation harness (frag_s7_write.py).** Opens a real allowlisted S7 session and sends a Write-Var either whole or split across TCP segments so the function byte falls in the second, the test behind the before/after of Section 4.5.1.

```
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)   # Nagle off: each send() is its own segment
s.sendall(COTP_CR); s.recv(512); s.sendall(S7_SETUP); s.recv(512)   # real S7 session on an allowlisted conduit
if split <= 0: s.send(WRITEVAR)                            # whole  -> detected -> ISOLATE
else:          s.send(WRITEVAR[:split]); time.sleep(gap); s.send(WRITEVAR[split:])  # fragmented
```

Listing A.33: Sending the Write-Var whole or split at byte N (0x05 pushed into the second segment).

**Low-rate insider tools (hmi_cmd.py, spoof_high.py).** A single authorised HMI command, and a sustained high sensor spoof: the two residual insider actions of Section 4.9.

```
# hmi_cmd.py: one authorised write of the command merker %MB0 (bit 2 = HMI_Stop)
c.write_area(MK, 0, 0, bytearray([0x04]))                    # CARS classifies CONTROL -> ALLOW (trusted role)
# spoof_high.py: pin %ID100 high so Sim.Level (= LevelIn*20) reads ~full
c.write_area(PE, 0, 100, bytearray(struct.pack('>f', 4.5)))    # reported ~90 while the real tank drains
```

Listing A.34: One authorised HMI stop; and pinning the level sensor high.

**Process guardian (cars_process_guardian.py).** The defence-in-depth layer of Section 4.9: an independent, read-only monitor that adds the high-side envelope, symmetric rate and liveness checks the low-side remediation agent lacks, and raises alarms only. It never writes the PLC and never touches enforcement or the remediation agent.

```
on each poll of the level:
    if level > CEILING:               alarm HIGH_LEVEL   // the high excursion the low-side agent
      misses
    if level < FLOOR:                 alarm LOW_LEVEL
    if |level - previous| > MAX_DELTA: alarm RATE         // a physically impossible per-poll jump
    if level has not changed for longer than STALL_TIME:  alarm STALL   // frozen while it should
      cycle = held
```

```
    // alarms only: never writes the PLC, never touches enforcement or the remediation agent
```

Listing A.35: The high-side envelope, rate and liveness checks (alarm-only), in pseudocode.

## A.10 The CARS intelligence dashboard

The dashboard (`cars_dashboard.py`) is a live operator view of the defence rather than a static diagram. It is discovery-driven: a few times a second it polls the controller's REST interface (`/cars/status`, `/cars/hosts`, `/cars/ports`, `/cars/links`, `/cars/guard`, `/cars/audit`) and renders whatever the controller currently knows. The nodes (switches, hosts, the controller, and the modelled north side of IT, DMZ, firewall and Snort), together with their port bindings, link health, the anti-spoof GUARD drop counters and the engine's own decisions, are all pulled from the running system, so the picture is the live state and not a description of it. A separate pane reads the remediation agent's status and event feed from its local files, because that agent runs inside the OT namespace and cannot reach the control plane.

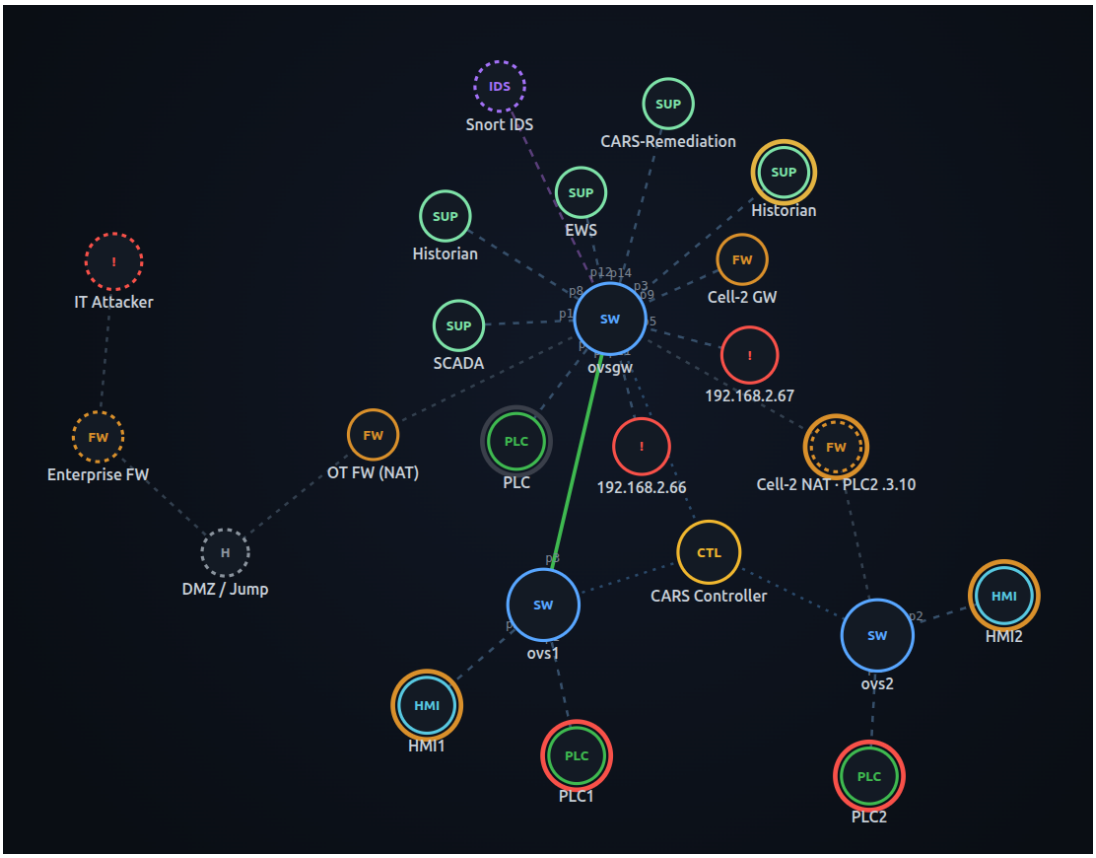


Figure A.11: The CARS dashboard's discovered topology. Switches (`ovsgw`, `ovs1`, `ovs2`) in blue, the supervisory hosts (EWS, SCADA, the historians, CARS-Remediation) and PLCs discovered live with their roles and criticality rings, the CARS controller and Snort IDS, the modelled north side (Enterprise firewall, DMZ/jump, OT NAT) and the Cell-2 NAT path to PLC2. Unregistered hosts (the on-segment attacker seams `192.168.2.66` and `192.168.2.67`) are flagged. Everything shown is polled live from the controller, not drawn by hand.

The centre of the view is the discovered network topology (Figure A.11); the right-hand column carries the live decision feed, filterable by criticality tier, response, operation class and enforcement mode. Each row shows the source and its discovered role, the destination and its criticality tier, the operation class, the criticality-elevated tier and the selected response, colour-coded by severity. It is this feed that makes the engine's reasoning legible in real time: a supervisory read appears as an OPERATIONAL ALLOW, a remediation write as a RESTORE, and an out-of-role actuation as a FORBIDDEN ISOLATE.

Figure 4.3 in Section 4.4 is a capture of this feed during the armed false-data-injection run: the compromised supervisory host `192.168.2.31 (scada)` issuing S7 CONTROL to the CRITICAL PLC1,

each attempt classed FORBIDDEN and met with an ENFORCED ISOLATE. Placed beside the datapath rule of Figure 4.4 and the wire capture of Section 4.5, it is the same attack seen at a third layer, the operator's, completing the cross-layer account: one event visible at once as a policy decision, an installed flow and a dropped packet.

## A.11 Gap remediation evidence

This appendix collects the fresh before-and-after evidence for the late-stage gap remediations of Chapter 4, captured on 7 and 8 August 2026 on the armed testbed and returned to green after each run. The consolidated record with reproduction commands is the companion file `GAP_REMEDIATION_EVIDENCE.md`; the raw artefacts (controller-audit slices, the `cars.rules`/`cars.conf` diffs, the guardian log and the honeypot captures) are in `gap_evidence/`.

**Gap A: DPI defeated by fragmentation, then hardened.** A Write-Var split across two TCP segments evaded the fixed-offset single-packet rule; after enabling stream reassembly and re-anchoring the rules to the S7 job header it is recovered and isolated at more than one split point, with the legitimate loop still permitted.

```
BEFORE (single-packet rules) - fragmented at byte 14, armed:
  192.168.2.31(scada) -> 192.168.2.10(plc) TCP => ALLOW (operational)   [no CONTROL recovered; 0xca=0; write reached PLC]
  (same write sent WHOLE):
  FORBIDDEN 192.168.2.31(scada) -> 192.168.2.10(plc) S7 CONTROL => ISOLATE source 192.168.2.31 75s
AFTER (stream5 reassembly + PDU-relative rules) - same fragmented attack:
  --split 14: FORBIDDEN 192.168.2.31 ... S7 CONTROL => ISOLATE source 192.168.2.31 75s ; flow 0xca nw_src=192.168.2.31
  --split  9: FORBIDDEN 192.168.2.31 ... S7 CONTROL => ISOLATE
  legit loop: OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10 READ/CONTROL => ALLOW   [no new false positive]
```

Listing A.36: Gap A: before (fragmented write evades) and after (caught), from the controller audit.

The fix itself is shown as the unified diff of the deployed Snort configuration and rules against the pre-hardening backup: Listing A.37 adds the TCP stream-reassembly preprocessor on the ICS ports, and Listing A.38 re-anchors every S7 operation rule from a fixed packet offset to a PDU-relative match on the S7 job header.

```
--- /home/msclab/cars_backup_2026-08-07_1715/cars.conf  2026-08-07 17:15:51.404757857 +0100
+++ /etc/snort/cars.conf        2026-08-07 18:57:12.948533211 +0100
@@ -2,4 +2,6 @@
 var EXTERNAL_NET any
 config checksum_mode: none
 config event_queue: max_queue 10 log 5 order_events content_length
+preprocessor stream5_global: track_tcp yes, track_udp no, max_tcp 8192
+preprocessor stream5_tcp: policy linux, timeout 60, ports both 102 502
 include /etc/snort/cars.rules
```

Listing A.37: Gap A: `cars.conf` diff, stream reassembly added on ports 102 and 502.

```
--- /home/msclab/cars_backup_2026-08-07_1715/cars.rules 2026-08-07 17:15:51.404705417 +0100
+++ /etc/snort/cars.rules       2026-08-07 19:12:56.120001291 +0100
@@ -1,4 +1,4 @@
-
alert icmp any any -> 192.168.2.10 any (msg:"CARS-SUSPECT ICMP PLC1"; sid:1000001; rev:5;)
+alert icmp any any -> 192.168.2.10 any (msg:"CARS-SUSPECT ICMP PLC1"; sid:1000001; rev:5;)
 alert icmp any any -> 192.168.2.9  any (msg:"CARS-SUSPECT ICMP HMI1"; sid:1000002; rev:2;)
 alert tcp  any any -> 192.168.2.10 any (msg:"CARS-SUSPECT TCP PLC1"; flags:S; sid:1000003; rev:1;)
 alert tcp  any any -> 192.168.2.9  any (msg:"CARS-SUSPECT TCP HMI1"; flags:S; sid:1000004; rev:1;)
@@ -12,22 +12,22 @@
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coils"; content:"|00 00|"; offset:2; depth:2; content:"|0F|"; offset:7; depth:1; sid:1000022; rev:3;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-regs";  content:"|00 00|"; offset:2; depth:2; content:"|10|"; offset:7; depth:1; sid:1000023; rev:3;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-READ-holding"; content:"|00 00|"; offset:2; depth:2; content:"|03|"; offset:7; depth:1; sid:1000024; rev:3;)
-# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102) ---
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; depth:2; content:"|72|"; offset:7; depth:1; sid:1000040; rev:1;)
+# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102) --- PDU-relative (Gap-2 reassembly fix)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; content:"|72|"; distance:5; within:6; sid:1000040; rev:2;)
 # CC-51 broadened ICS intelligence: dangerous ops
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-DIAG-diagnostics"; content:"|00 00|"; offset:2; depth:2; content:"|08|"; offset:7; depth:1; sid:1000025; rev:1;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-PROGRAM-mei"; content:"|00 00|"; offset:2; depth:2; content:"|2B|"; offset:7; depth:1; sid:1000026; rev:1;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-ILLEGAL-fc"; content:"|00 00|"; offset:2; depth:2; byte_test:1,>,43,7; sid:1000027; rev:1;)
-# PD-3: classic S7comm Write-Var on real PLC1
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth
-       :1; sid:1000041; rev:1;)
+# PD-3: classic S7comm Write-Var on real PLC1 (PDU-relative)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
+       sid:1000041; rev:2;)
 # PD-7: classic S7comm PLC-control on real PLC1 - STOP (0x29) / Control-start (0x28) => DIAG (dangerous)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1;
-        sid:1000042; rev:1;)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1;
-        sid:1000043; rev:1;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|29|"; distance:8; within:9; sid
+       :1000042; rev:2;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|28|"; distance:8; within:9; sid
+       :1000043; rev:2;)
 # showcase: S7 read (func 0x04) => READ => OPERATIONAL (contrast vs write/stop)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|04|"; offset:17; depth:1; sid
-       :1000044; rev:1;)
-# PLC2-2: Cell-2 target .3.10 S7 operation DPI
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth
-       :1; sid:1000045; rev:1;)
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1;
-        sid:1000046; rev:1;)
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|04|"; offset:17; depth:1;
-        sid:1000047; rev:1;)
```

```
-# N2 (audit 07-24): Cell-2 symmetry - Cell-1 has S7 control-start (0x28) DPI (sid 1000043) but Cell-2 lacked it. Add it.
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1;
       sid:1000048; rev:1;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|04|"; distance:8; within:9; sid
       :1000044; rev:2;)
+# PLC2-2: Cell-2 target .3.10 S7 operation DPI (PDU-relative)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
       sid:1000045; rev:2;)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|29|"; distance:8; within:9; sid
       :1000046; rev:2;)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|04|"; distance:8; within:9; sid
       :1000047; rev:2;)
+# N2 Cell-2 symmetry: S7 control-start (0x28) DPI
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|28|"; distance:8; within:9; sid
       :1000048; rev:2;)
```

Listing A.38: Gap A: `cars.rules` diff, S7 rules re-anchored PDU-relative (rev 1 to rev 2).

**Gap B: trusted-insider process guardian.** The high-rate overflow is cut by the volumetric overlay even from a trusted role; the read-only guardian then catches the low-rate denial and the high-side sensor spoof that the low-side remediation agent misses.

```
FLOOD BACKSTOP (trusted .45 driven at high rate):
  192.168.2.45(remediation) -> 192.168.2.10 S7 CONTROL => [FLOOD 51 ops/s] BLOCK conduit 75s  [FDI cut after 77 writes; tank held ~mid-70s]
DEMO 1 - low-rate denial (one authorised HMI_Stop from .45):
  audit:    192.168.2.45(remediation) -> 192.168.2.10 S7 CONTROL => ALLOW (operational)   [network correctly permits]
  guardian: [GUARDIAN] ALARM STALL {level: 40.4, frozen_s: 12.0}                          [process halt caught]
DEMO 2 - spoof-to-high (reported level pinned at 90, real tank drains):
  remediation status: level=90.0, restores unchanged                                      [low-side agent silent = blind]
  guardian: [GUARDIAN] ALARM HIGH_LEVEL {level: 90.0, ceiling: 78.0}  +  ALARM RATE       [high-side caught]
```

Listing A.39: Gap B: the flood backstop and the two guardian alarms.

The guardian writes each alarm to a read-only feed; the stall record from the fresh run is the raw line in Listing A.40. This is a crafted scenario that demonstrates the mechanism rather than broad operational resilience: the flood, envelope and rate thresholds are tuned to this tank and its loop rate, and would need re-tuning for a different plant, loop cadence or operator profile.

```
{"level": 40.4, "frozen_s": 12.0, "event": "PROCESS-ANOMALY", "kind": "STALL", "ts": 1786208299.0824745, "plc": "PLC1"}
```

Listing A.40: Gap B: the guardian alarm feed (`cars_guardian.jsonl`), stall record from the fresh run.

**Gap C: flow-integrity poll window.** A rule injected and deleted inside the ten-second poll evades the auditor; a persistent one is caught within one poll. The single traces below were then repeated thirty times each to quantify the window: the persistent case was caught on every trial at a median of 8.5 seconds, the two-second transient on only five of thirty (the runs whose life straddled a poll boundary). The per-trial values are in `gap_evidence/flowint/persistent.csv` and `transient.csv`.

```
baseline:                   {"ok": 1, "extra": 0}
2 s transient inject+delete:  t+7s ok:1 | t+14s ok:1 | t+21s ok:1     [MISSED - never flipped]
persistent inject (left):     {"ok": 0, "extra": 1}                   [CAUGHT within one poll]
after cleanup:                {"ok": 1, "extra": 0}
30-trial totals:  persistent 30/30 caught, median 8.5 s | transient 5/30 caught, 25/30 evaded
```

Listing A.41: Gap C: transient (mostly missed) versus persistent (always caught) flow-integrity status, with the thirty-trial totals.

The event-driven flow-monitor of Section 4.12 was then run against the identical thirty-trial transient test. It subscribes to the switch's Nicira flow-monitor (`ovs-ofctl monitor <br> watch:`) and, on any change to the policy tables that is not a cookie-`0xca` reactive rule, re-runs the deployed auditor's trusted check at once, so the sub-poll window closes without changing the verdict logic. Every one of the thirty two-second injections was caught, at a median of 0.27 seconds, against five of thirty for the ten-second poll; the per-trial values are in `gap_evidence/flowint/transient_eventdriven.csv`.

```
10 s poll (baseline):     transient  5/30 caught, 25/30 evaded (median 8.5 s when caught)
event-driven monitor:     transient 30/30 caught, median 0.27 s   [sub-poll gap closed]
mechanism: NXST_FLOW_MONITOR watch -> immediate trusted re-check on any non-0xca policy-table change
```

Listing A.42: Gap C remedy: event-driven monitor on the same transient test.

**Gap 2: conduit block at a shared / NAT-collapsed identity.** An IT attacker source-NATed onto the gateway identity `192.168.2.1` was originally cut by a source isolation, which quarantines every host behind the NAT (Section 4.7). The response selection was refined so a shared identity is cut at the conduit instead. Verified on the deployed engine: the same forbidden attempt from `192.168.2.1` now returns `BLOCK` and installs a conduit drop, while a true single host is still source-isolated.

```
gateway .1 (NAT-collapsed) -> CONTROL -> PLC1:  response=BLOCK
  flow: cookie=0xca priority=100,ip,nw_src=192.168.2.1,nw_dst=192.168.2.10 actions=drop   [conduit: only .1->PLC]
unknown .77 (true host)   -> CONTROL -> PLC1:  response=ISOLATE
  flow: cookie=0xca priority=110,ip,nw_src=192.168.2.77 actions=drop                      [source: whole host]
both land zero writes on the PLC; the gateway's non-PLC traffic is untouched by the conduit match.
```

Listing A.43: Gap 2: shared-identity conduit BLOCK versus single-host source ISOLATE, deployed engine.

**Gap D: DEFLECT diverts to the honeypot.** Forcing DEFLECT installs the redirect and the attacker's PLC-bound traffic arrives at the decoy while the real PLC sees nothing.

```
silent-drop baseline:  atkns(.66) ping 192.168.2.10 -> 100% loss                    [dropped; attacker hangs]
force DEFLECT:         response=DEFLECT  action="DEFLECT conduit -> honeypot 192.168.3.99 (deception, self-healing)"
redirect flow:         priority=105 nw_src=192.168.2.66 nw_dst=192.168.2.10 -> set ip_dst=192.168.3.99, goto t2 (n_packets=3)
honeypot capture:      IP 192.168.2.66 > 192.168.3.99: ICMP echo request             [attacker DIVERTED to decoy]
                       ARP, who-has 192.168.2.66 tell 192.168.3.99                    [decoy engaging]
```

Listing A.44: Gap D: silent-drop baseline, the redirect, and the decoy receiving the diverted traffic.

This is a proof of concept for the redirect path, not evidence of robustness. A determined adversary who notices the diversion, fingerprints the decoy or changes tactics is outside what this test exercises; sustaining a convincing interactive decoy is carried as future work in Section 5.3.
The plaintext control channel (G1), state-exhaustion resilience (G3) and the file-based detection bridge under heavy load (G4) are verified but framed as bounded limitations in Section 4.12 rather than tested, for the reasons given there.

## A.12 Process-transparency and availability runs

This section holds the raw artefacts behind the process-transparency claim of Section 4.8 and the reconnection test of Section 4.12, captured on the armed testbed on 7 August 2026 and returned to green after each run.
**Transparency: armed against disarmed under matched load.** The two states were interleaved in short blocks over two hours so that any drift in the hardware-in-the-loop simulation fell on both equally. The per-second tank level, phase label, online flag and restore count are in `availability/interleaved.csv` (7,060 samples); the online-only summary is below. The means and spreads coincide and neither state left the control band, so arming the defence has no measurable effect on the process. As Section 4.8 states, this is strong evidence for the testbed and its hardware-in-the-loop tank rather than a universal guarantee for all industrial processes.

```
disarmed:  n=3528  mean=51.5%  sd=12.8  min=25.5  max=71.3   (band 20-78, no excursion)
armed:     n=3530  mean=50.7%  sd=13.2  min=25.6  max=71.3   (band 20-78, no excursion)
EWS/SCADA legitimate throughput unchanged across both phases.
```

Listing A.45: Transparency: online-only level statistics per phase, from `interleaved.csv`.

**Block-and-maintain and self-heal, measured at the process.** An allowlisted supervisory host launched a false-data injection at the CRITICAL PLC under the armed defence. The write was classified FORBIDDEN and the source isolated after nine packets with the criticality-scaled seventy-five-second timeout; the tank stayed within band for the whole block, the legitimate conduits kept flowing, and the restoration agent was never called. The rule then expired and the source returned to service with no operator action, the install-to-withdrawal spanning seventy-six seconds.

```
t+0.0s   FDI write from 192.168.2.31(scada) -> 192.168.2.10(plc, CRITICAL) => FORBIDDEN
t+~0s    ISOLATE source 192.168.2.31 (cookie 0xca, hard_timeout=75s, both switches)
block    tank level held within band; legit EWS/HMI conduits uninterrupted; restore agent NOT called
t+76s    rule expired (SEND_FLOW_REM); source restored, no operator action; table back to baseline
```

Listing A.46: Block-and-maintain under attack, then self-heal, from the armed run.

**Reconnection jitter, two planes.** Restarting the passive Snort sensor left the live conduit's inter-frame timing indistinguishable from baseline; forcing a controller reconnection by delete-and-re-add opened a roughly twenty-second window with multi-second forwarding stalls and dropped the synchronous S7 session. The per-second timeline (level, legitimate packets per second, controller-attached flag, event marks) is in `availability/jitter_timeline.csv` with the accompanying `hil.pcap`. As Section 4.12 sets out, that window largely reflects the heavy re-add and pipeline re-sync rather than a faithful crash against the `fail_mode=secure` switches, so it is reported as an upper bound and the clean crash-transparency figure is left as future work. This measurement should not be read as a crash-resilience proof or a production-availability guarantee: a fail-secure switch under a forced delete-and-re-add is not the same as a real controller crash, and the deployment implication is left deliberately cautious.

```
Snort restart (passive, out-of-band):  inter-frame mean 4.90 ms vs 4.91 ms baseline, worst gap 33 ms vs 42 ms  [TRANSPARENT]
Controller del/re-add (forced):        ~20 s window, worst forwarding gap 4.2 s, S7 session dropped, manual reconnect  [UPPER BOUND, not a faithful crash]
```

Listing A.47: Reconnection jitter: the two planes contrasted.

## A.13 Statistical visualisations of the evaluation

The figures below plot the measured data behind the Critical Evaluation. Each is generated directly from a captured result file, named in its caption, with no smoothing or illustrative curves; log axes are used where a range spans orders of magnitude.

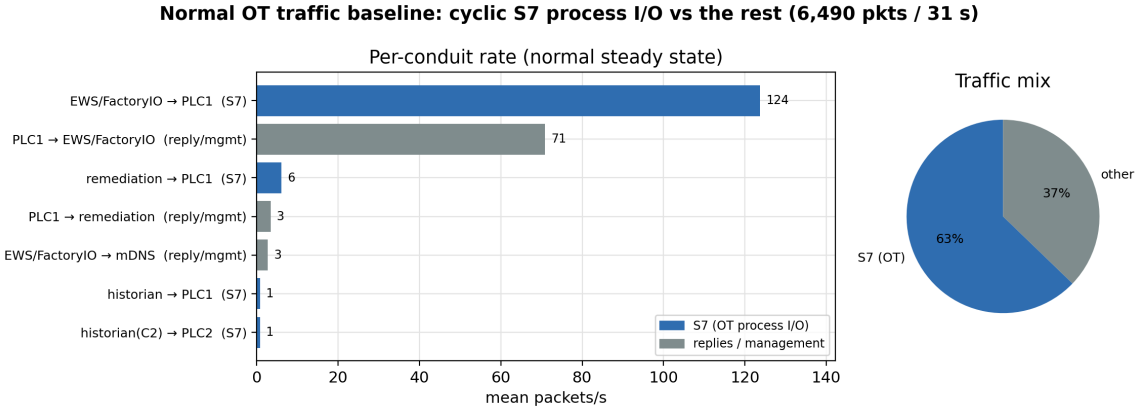


Figure A.12: Normal OT traffic baseline: per-conduit mean packets/s and the protocol mix, from a 31-second steady-state capture (6,490 packets, 9 conduits; the seven carrying more than one packet per second are shown, the two sub-packet reply flows omitted). The cyclic engineering-to-PLC S7 flow dominates; the remainder is PLC replies and background multicast. Source: `results/b1/baseline.pcap` via `b1_analyze.py`.

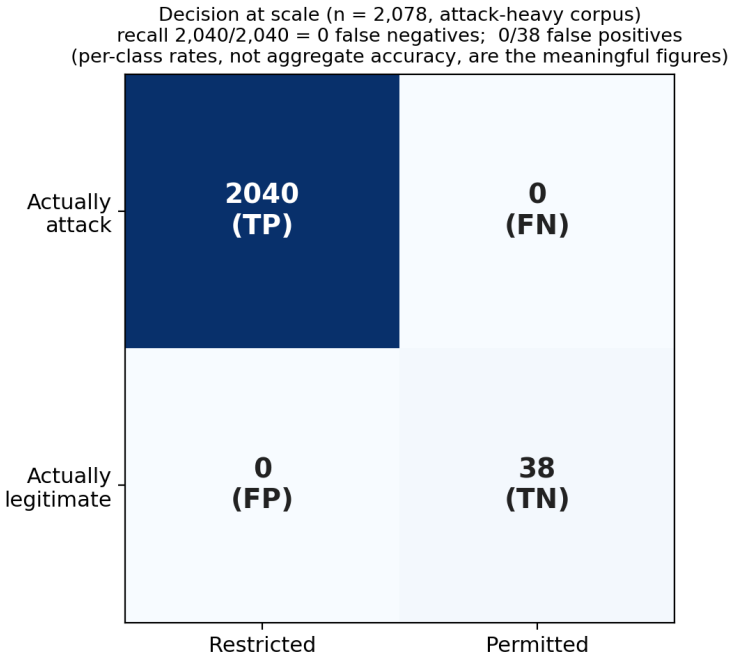


Figure A.13: Decision conformance at scale over the 2,078-case labelled corpus, which is attack-heavy by design (2,040 attacks against 38 legitimate or exempt cases). Every attack was restrained (no false negative) and every legitimate or exempt case permitted (no false positive). Because the decision is deterministic and the corpus is imbalanced, the two per-class error rates, both zero, are the meaningful figures rather than any aggregate. The attack side, where the label and the default-deny share the same registration test, is a coverage-and-robustness result; the zero false positive over the authorised cases is the trust result. Source: `results/vast/vast.csv`.

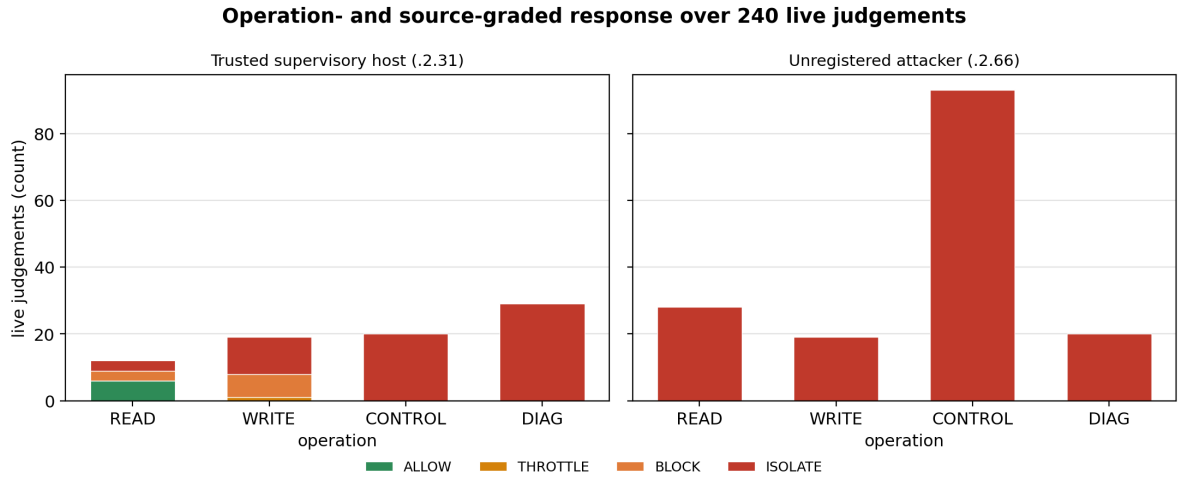


Figure A.14: Operation- and source-graded response over 240 live judgements. The trusted host keeps its reads (with a small tail of persistence-escalated cuts as its offence count rises across the sequence) and is cut on the operations its role must not perform, whereas the unregistered attacker is cut on every operation. Source: `results/critbeh/judgements.csv`.

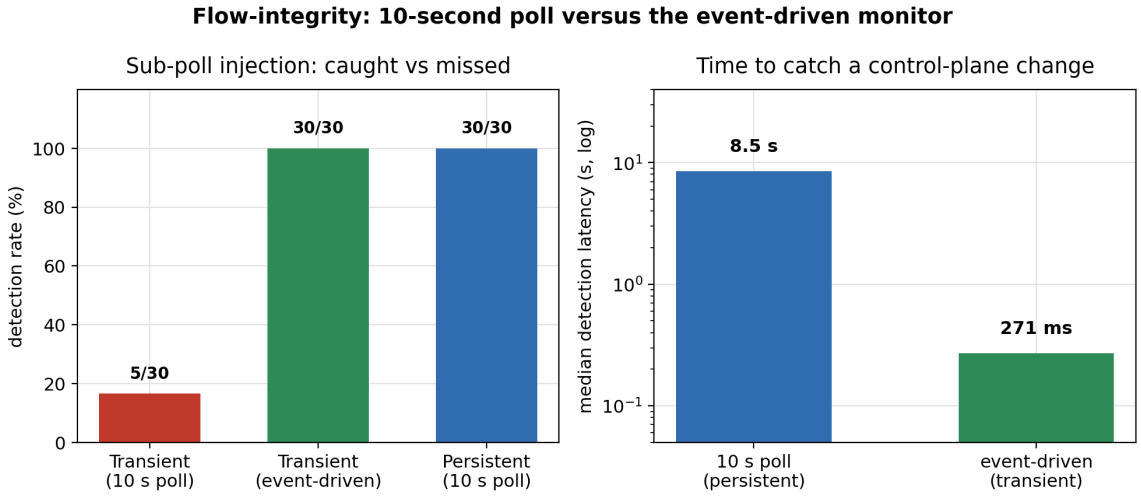


Figure A.15: Flow-integrity, ten-second poll versus the event-driven monitor. Left: a two-second transient injection is missed by the poll on 25 of 30 trials but caught on all 30 by the event-driven monitor; a persistent injection is caught by both. Right: median detection latency, 8.5 s for the poll against 0.27 s for the monitor (log axis). Sources: `persistent.csv` and `transient.csv` for the poll, and `transient_eventdriven.csv` for the event-driven monitor.

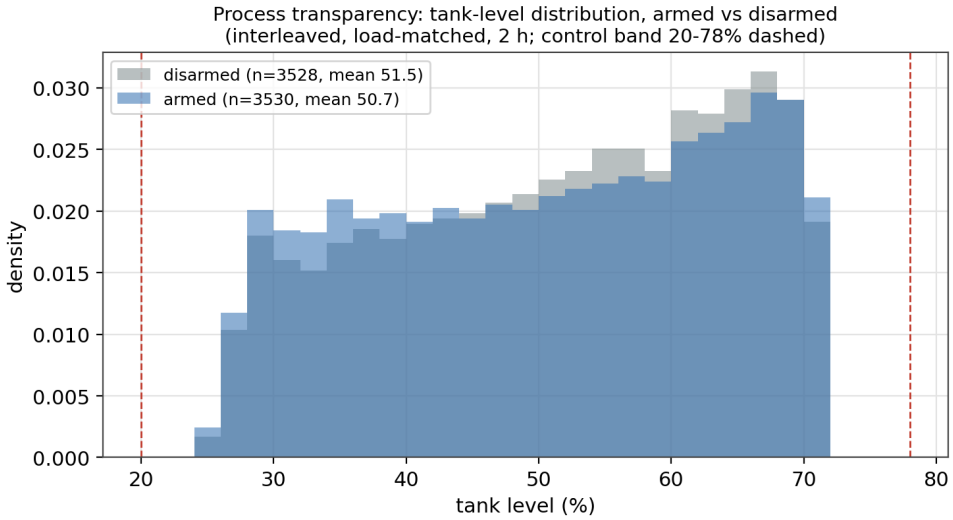


Figure A.16: Process transparency: the tank-level distribution armed against disarmed, over roughly seven thousand interleaved, load-matched per-second samples. The two distributions coincide (means 50.7% armed and 51.5% disarmed) and neither leaves the control band, so arming the defence has no measurable effect on the process. Source: `results/e2/interleaved.csv`.

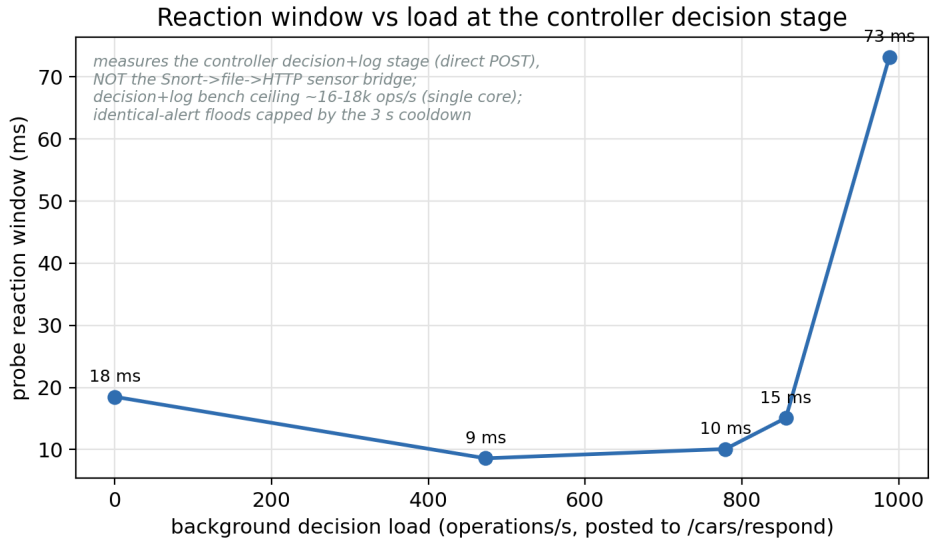


Figure A.17: Reaction window against background load, measured at the controller's decision-and-log stage: both the background and the probe are posted directly to `/cars/respond`, so this characterises the controller (`classify`, `select_response` and the audit write), not the Snort-to-file-to-HTTP sensor bridge. The window holds in the tens of milliseconds to about a thousand operations a second and rises only as this single-threaded stage nears its sixteen-to-eighteen-thousand-a-second bench ceiling. The sensor bridge is a separate and lower ceiling, the acknowledged bottleneck carried to the future work (Section 4.12), and is not what this curve measures. Source: `results/gap1_live/curve.csv`.

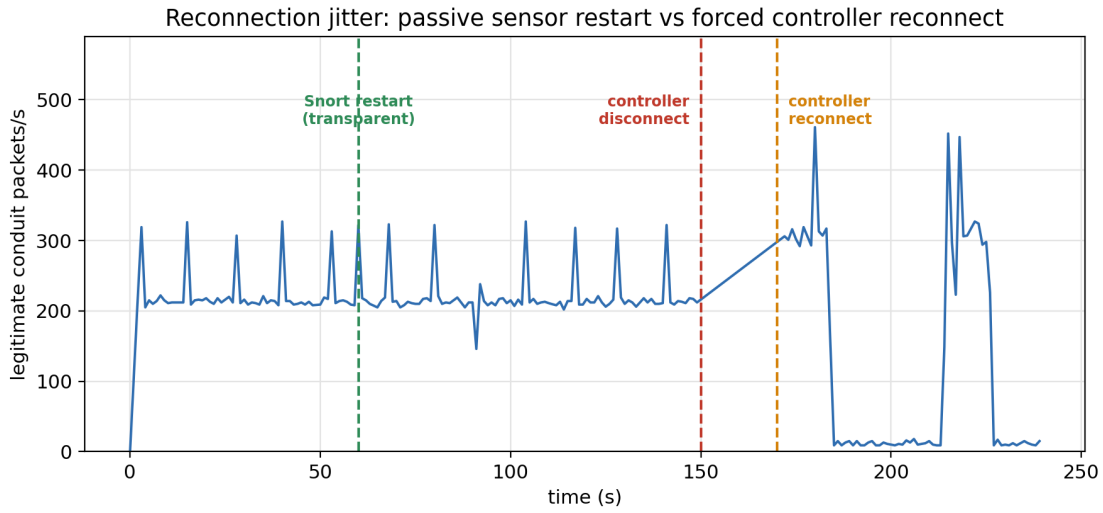


Figure A.18: Reconnection jitter on the legitimate conduit: restarting the passive Snort sensor is transparent (no dip), whereas a forced controller reconnection stalls forwarding. Source: `results/jitter/timeline.csv` (one spurious counter sample removed).

### A.13.1 Long-run and stress-test campaign

The figures below back the long-run and stress test of Section 4.10: four autonomous runs on the one testbed, 440 measured attacks, all enforced. They extend the earlier evidence in duration and repetition, not in network size. The four runs are repeated runs on the same testbed and machine state, not independent replicates; where a median is quoted it is pooled across the runs rather than averaged across per-run medians. The row counts and any exclusions are stated in each caption.

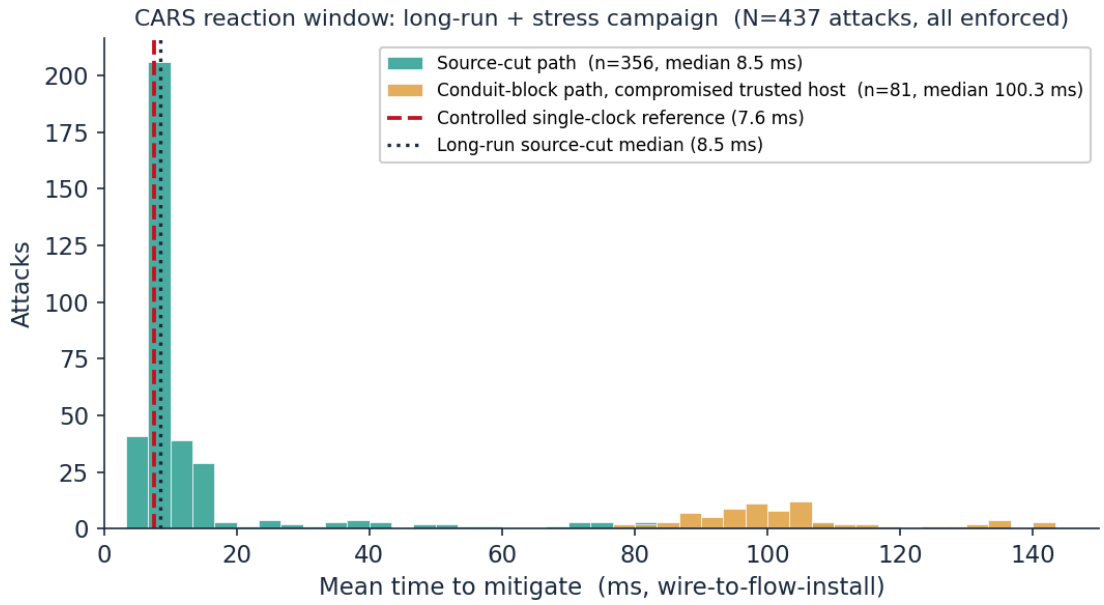


Figure A.19: Reaction window over the long-run and stress campaign. The source-cut path (356 attacks, median 8.5 ms) sits close to the 7.6 ms controlled single-clock reference of Section 4.6; the false-data injections from the compromised trusted host form a separate mode near 100 ms, the conduit-block path that installs after the first packets. 437 of the 440 measured attacks are shown: two under-load DDoS-probe samples, which belong to the load curve rather than the clean reaction window, and one physically impossible negative row from a stale-rule read, are excluded.

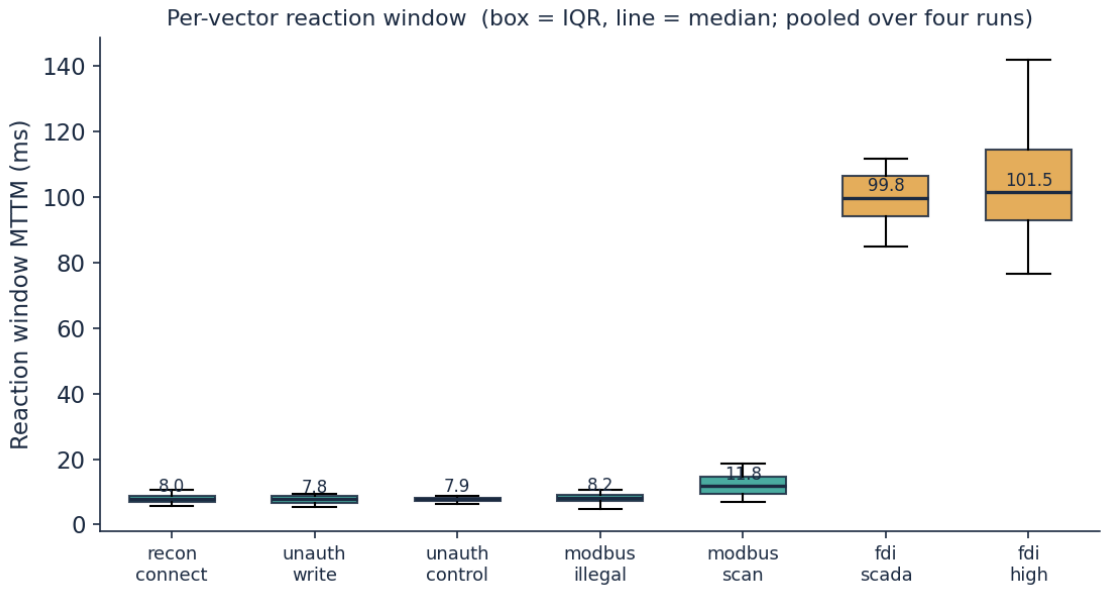


Figure A.20: Per-vector reaction window pooled over the four runs (box is the interquartile range, line the median). The source-cut vectors sit at roughly 7 to 12 ms; the two false-data injection vectors from the compromised trusted host sit near 100 ms because they draw a conduit block rather than an immediate source cut.

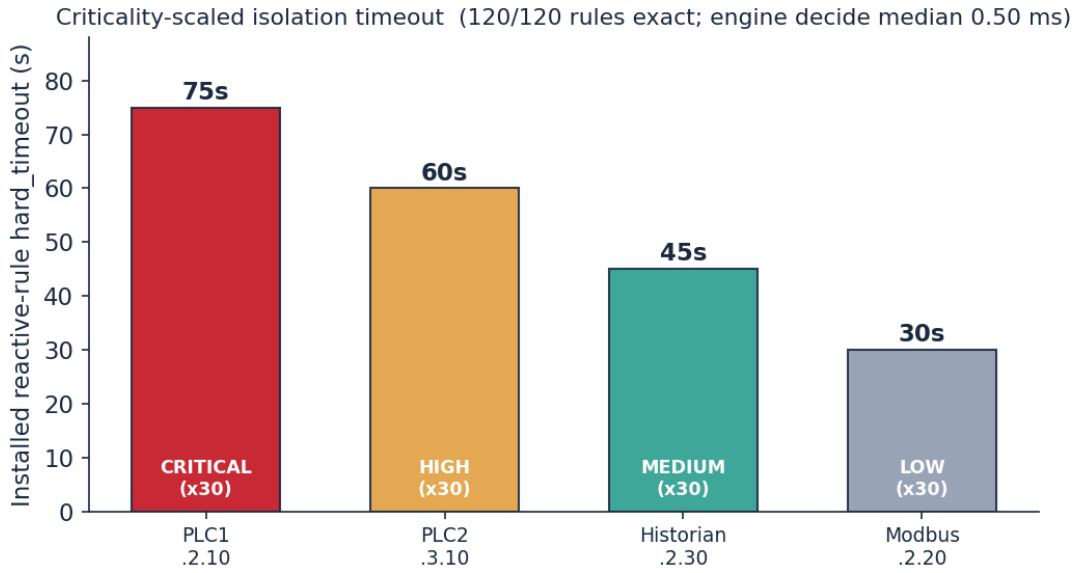


Figure A.21: Criticality-scaled isolation timeout over 120 tier probes across the four runs. Every probe installed 75/60/45/30 s for CRITICAL/HIGH/MEDIUM/LOW with no deviation, at a median engine decide-time of 0.50 ms under the concurrent night-campaign load, against the 0.026 ms of the dedicated clean run of Section 4.6.

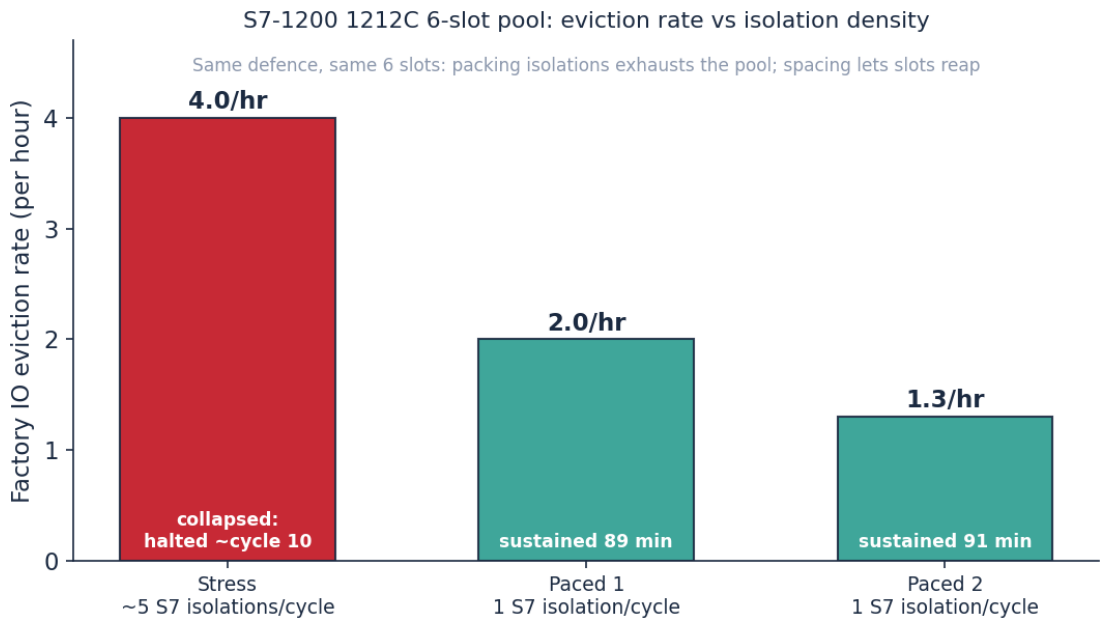


Figure A.22: The S7-1200 (1212C) connection-pool limit. The CPU exposes six dynamic connection resources; repeatedly isolating S7 sessions leaves half-open sessions that consume them. Packing about five isolations into each short cycle evicted the legitimate Factory IO client at 4.0 times an hour and the run could not be sustained; pacing to one isolation per cycle held the process for about ninety minutes at 1.3 to 2.0 evictions an hour. This is the empirical confirmation of the connection-pool caveat in Section 4.12.

That the freeze is the endpoint's limit rather than a defect in the policy is confirmed on the switch: an isolate installs a single drop scoped to the attacker's source address alone, so it cannot match the legitimate conduit. Listing A.48 is a live capture taken on `ovs1` while an isolation was in force, showing the reactive rule matching only `nw_src=192.168.2.77` while the `192.168.2.55` to `192.168.2.10` conduit remained a committed allow with its counter advancing.

```
# Captured live on ovs1 while an ISOLATE was in force, to show the reactive drop is
# scoped to the attacker's source address alone and never touches the legitimate conduit.

# 1) The rule an ISOLATE installs: one source-scoped drop, the attacker's address only.
 cookie=0xca, hard_timeout=75, send_flow_rem, priority=110, ip, nw_src=192.168.2.77  actions=drop

# 2) The legitimate Factory IO / EWS conduit to PLC1 during that same isolation: an intact
#    committed allow, its counter advancing (traffic still forwarded to the PLC).
 cookie=0xa2, table=1, n_packets=2, priority=80, ct_state=+new+trk, tcp,
              nw_src=192.168.2.55, nw_dst=192.168.2.10, tp_dst=102  actions=ct(commit),goto_table:2

# 3) The only drop that mentions the legitimate client is the GUARD identity check, which
#    fires only on a SPOOFED 192.168.2.55 from the wrong port/MAC. n_packets=0: the real
#    client never hits it. The isolate cannot, by construction, cut the legitimate conduit.
 cookie=0x0,  table=0, n_packets=0, priority=100, ip, nw_src=192.168.2.55  actions=drop
```

Listing A.48: The isolate is source-scoped: the reactive drop matches the attacker alone, and the legitimate Factory IO conduit is untouched during the isolation (captured live on `ovs1`).

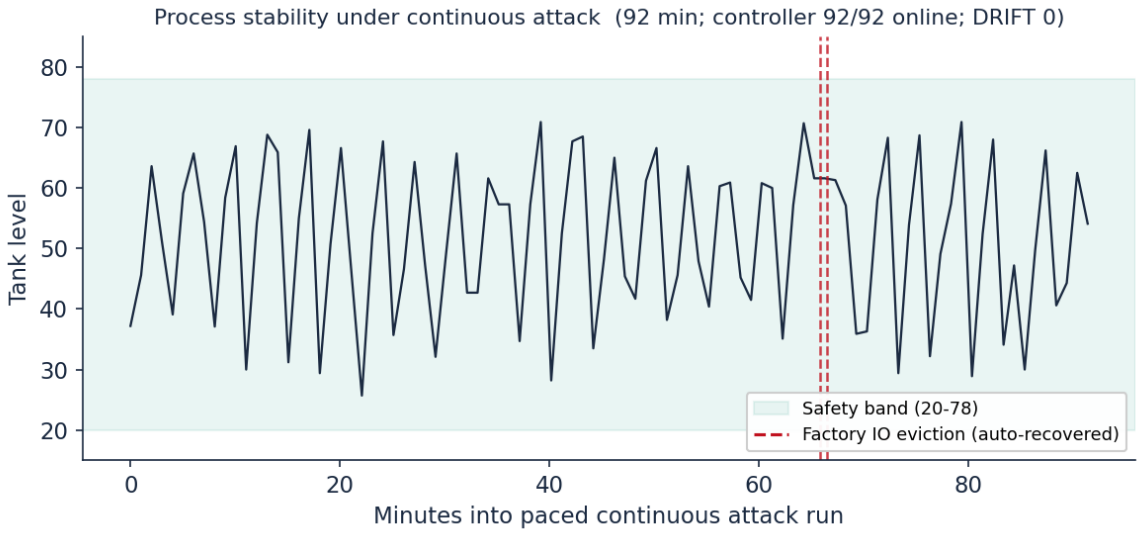


Figure A.23: Process stability under continuous attack: the tank level over the ninety-two one-minute snapshots of the paced run (the same run as the second paced bar of Figure A.22, about ninety-one minutes), held within the 20 to 78 control band throughout. The controller and its remediation agent reported the process online in all ninety-two one-minute snapshots (this is the agent's PLC link sampled once a minute, which does not capture sub-minute client outages) and the flow auditor reported no drift. The two dashed marks are the brief Factory IO client evictions that the liveness guard caught and recovered.